

Linux 操作系统

作者名 2026 年 1 月

28 日

版权所有 © 2026 作者名 出版社名称

摘要

Linux 是一个自由、开放的操作系统，具有高效、安全、稳定等特点，支持多种硬件平台，用户界面友好，网络功能强大，支持多任务、多用户。本书全面介绍了 *Linux* 操作系统的历史与发展、体系结构、版本分类、安装配置、命令行操作、用户权限管理、软件包管理、系统管理、网络配置、脚本编程以及高级应用等内容，帮助读者系统地掌握 *Linux* 操作系统的使用和管理技能。

关键词： *Linux* 操作系统 *RHEL* 内核 发行版

目录

第一章	Linux 概述	1
第 1 节	Linux 的历史与发展	1
第 2 节	Linux 的版权问题	2
第 3 节	Linux 的特点与优势	2
第 4 节	Linux 的应用领域	3
第 5 节	Linux 的体系结构	3
5.1	shell 提示符	3
第 6 节	Linux 的版本	4
6.1	内核版本	5
6.2	发行版本	5
1.6.2.1	RHEL 8	5
1.6.2.2	BaseOS 和 AppStream	6
第二章	Linux 系统安装与配置	7
第 1 节	系统安装前的准备	7
第 2 节	Linux 安装过程	7
第 3 节	系统初始化与基本配置	8
3.1	Linux 开机过程	8
3.2	systemd 初始化进程	9
2.3.2.1	启动级别与 Target 对应关系	9
3.3	systemd 与 System V init 的区别	10
3.4	systemctl 常用命令	10
3.5	timedatectl 命令	10
3.6	clock 命令	13
3.7	重置 root 管理员密码	14
第 4 节	多重引导配置	14

第三章 Linux 命令行基础	15
第 1 节 命令行界面简介	15
1.1 Linux 命令的详细说明	15
第 2 节 文件系统与目录结构	16
2.1 硬盘分区表格式	16
2.2 硬盘分区	17
2.3 分区工具	17
2.4 使用 mkfs 命令建立文件系统	19
2.5 使用 fsck 命令检查和修复文件系统	21
2.6 使用 dd 命令进行文件复制和转换	23
2.7 交换分区管理命令：mkswap、swapon、swapoff	25
3.2.7.1 mkswap 命令	26
3.2.7.2 swapon 命令	27
3.2.7.3 swapoff 命令	27
2.8 磁盘使用情况命令：df	28
2.9 目录磁盘使用情况命令：du	30
2.10 文件系统挂载管理	32
3.2.10.1 mount 命令	32
3.2.10.2 umount 命令	35
3.2.10.3 /etc/fstab 文件	35
2.11 独立硬盘冗余阵列（RAID）	37
2.12 逻辑卷管理器（LVM）	41
2.13 硬盘配额（Disk Quota）	45
2.14 文件权限和属性的记录	49
2.15 文件实际内容的记录	50
2.16 虚拟文件系统（VFS）	50
2.17 Linux 文件系统的特点	50
2.18 常见的硬件设备及其文件名称	51
2.19 Linux 常见目录结构	52
2.20 绝对路径与相对路径	53
2.21 文件与目录的基本概念	55
第 3 节 常用文件与目录操作命令	59
3.1 pwd 命令	60
3.2 clear 命令	60
3.3 echo 命令	61
3.4 chmod 命令	66

3.5	文件系统隐藏属性 (chattr 和 lsattr 命令)	71
3.6	文件访问控制列表 (ACL)	74
3.7	cd 命令	75
3.8	ls 命令	76
3.9	cat 命令	78
3.10	more 命令	79
3.11	less 命令	80
3.12	head 命令	81
3.13	tail 命令	81
3.14	mkdir 命令	82
3.15	rmdir 命令	83
3.16	rm 命令	83
3.17	cp 命令	84
3.18	mv 命令	85
3.19	touch 命令	85
3.20	whereis 命令	86
3.21	whatis 命令	87
3.22	find 命令	87
3.23	grep 命令	89
3.24	dd 命令	91
3.25	cal 命令	92
3.26	man 命令	93
3.27	alias 命令	96
3.28	unalias 命令	98
3.29	history 命令	99
第 4 节	文本编辑	102
第四章	Linux 用户与权限管理	103
第 1 节	用户与组的概念	103
1.1	/etc/passwd 文件	104
1.2	/etc/shadow 文件	105
1.3	/etc/login.defs 文件	106
1.4	/etc/group 文件	108
1.5	/etc/gshadow 文件	109
第 2 节	用户与组管理命令	109
2.1	useradd 命令	110

2.2	userdel 命令	112
2.3	adduser 命令	113
2.4	groupadd 命令	114
2.5	groupmod 命令	116
2.6	groupdel 命令	117
2.7	passwd 命令	118
2.8	su 命令	120
2.9	vipw 命令	121
2.10	vigr 命令	122
2.11	pwck 命令	124
2.12	grpck 命令	125
2.13	id 命令	126
2.14	whoami 命令	128
2.15	newgrp 命令	129
2.16	passwd 命令	130
2.17	chage 命令	132
2.18	usermod 命令	134
2.19	who 命令	136
2.20	last 命令	138
第 3 节	文件权限机制	141
第五章	Linux 软件包管理	143
第 1 节	软件包管理概述	143
第 2 节	RPM 包管理系统	143
2.1	rpm 命令	143
2.2	yum 软件仓库	145
2.3	常见 DNF 命令列表	146
第 3 节	DPKG 包管理系统	146
第 4 节	源码编译安装	146
第六章	Linux 系统管理	149
第 1 节	进程管理	149
1.1	ps 命令	149
1.2	top 命令	153
1.3	htop 命令	154
1.4	pidof 命令	156

1.5	kill 命令	157
1.6	killall 命令	159
1.7	nice 命令	161
1.8	renice 命令	163
1.9	作业管理命令 (jobs、bg、fg)	164
6.1.9.1	jobs 命令	164
6.1.9.2	bg 命令	164
6.1.9.3	fg 命令	165
第 2 节	服务管理	166
2.1	hostnamectl 命令	166
2.2	shutdown 命令	167
2.3	halt 命令	169
2.4	reboot 命令	171
2.5	poweroff 命令	172
第 3 节	磁盘管理	174
第 4 节	系统监控与性能优化	174
4.1	free 命令	175
4.2	uname 命令	176
4.3	dmesg 命令	177
4.4	sosreport 命令	178
第七章	Linux 网络配置与管理	183
第 1 节	网络基础概念	183
第 2 节	网络配置	183
2.1	主机名配置	183
2.2	使用 nmtui 和 nmcli 设置网络	184
7.2.2.1	nmtui 设置网络	184
7.2.2.2	nmcli 设置网络	185
2.3	传统网络配置文件目录	186
第 3 节	网络服务	187
第 4 节	防火墙配置	187
4.1	firewalld 服务详解	187
7.4.1.1	常见的区域名称及默认策略	188
7.4.1.2	使用 firewall-cmd 命令管理防火墙	188
第 5 节	网络工具	195
5.1	wget 命令	195

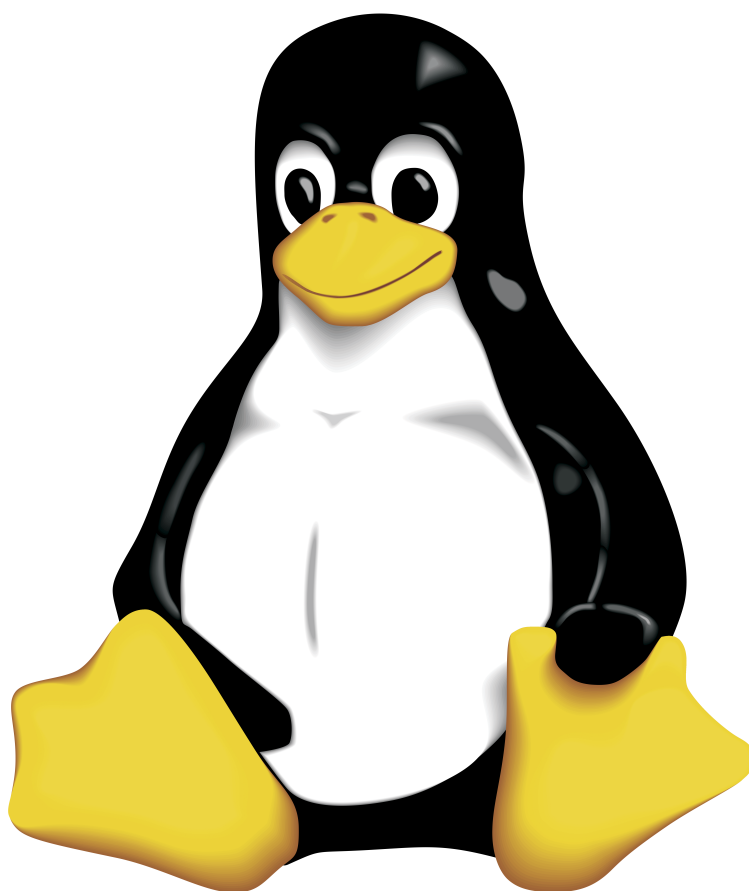
第八章	Linux 脚本编程	199
第 1 节	Shell 脚本基础	199
1.1	变量的定义和引用	199
第 2 节	Shell 编程结构	205
第 3 节	脚本调试与优化	205
第 4 节	实用脚本示例	205
第 5 节	常用命令行工具	205
5.1	grep 命令	205
5.2	正则表达式基础	207
第九章	Linux 高级应用	209
第 1 节	虚拟化技术	209
第 2 节	存储管理	209
第 3 节	安全加固	209
第十章	Linux 学习资源与进阶建议	211
第 1 节	推荐书籍	211
第 2 节	在线教程与文档	211
第 3 节	社区与论坛	211
第 4 节	实践项目建议	211
第 5 节	认证考试（RHCE, LPIC 等）	211

第一章 Linux 概述

第 1 节 Linux 的历史与发展

Linux 操作系统是一个类似 UNIX 的操作系统。Linux 操作系统是 UNIX 在计算机上的完整实现，它的标志是一个名为 Tux 的可爱的小企鹅形象。UNIX 操作系统是 1969 年由肯·莱恩·汤普森（Kenneth Lane Thompson）和丹尼斯·里奇（Dennis Ritchie）在美国贝尔实验室开发的一个操作系统。由于良好且稳定的性能，该操作系统迅速在计算机中得到广泛应用，在随后的几十年中又不断地被改进。

图 1.1: Linux 标志 Tux 小企鹅



1990 年，芬兰人莱纳斯·贝内迪克特·托瓦尔兹（Linus Benedict Torvalds）接触了为教学而设计的 Minix 系统后，开始着手研究编写一个开放的、与 Minix 系统兼容的

操作系统。1991 年 10 月 5 日，莱纳斯在芬兰赫尔辛基大学的一台 FTP 服务器上发布了一个消息。这也标志着 Linux 操作系统诞生。莱纳斯公布了第一个 Linux 的内核 0.02 版本。开始，莱纳斯的兴趣在于了解操作系统的运行原理，因此 Linux 早期的版本并没有考虑最终用户的使用，只是提供了最核心的框架，使得 Linux 开发人员可以享受编制内核的乐趣，但这样也保证了 Linux 操作系统内核的强大与稳定。互联网（Internet）的兴起，使得 Linux 操作系统也十分迅速地发展，很快就有许多程序员加入 Linux 操作系统的编写行列。

随着编程小组的扩大和完整的操作系统基础软件的出现，Linux 开发人员认识到，Linux 已经逐渐变成一个成熟的操作系统。1994 年 3 月，内核 1.0 版本的推出，标志着 Linux 第一个正式版本诞生。

第 2 节 Linux 的版权问题

Linux 是基于 Copyleft（无版权）的软件模式进行发布的。其实 Copyleft 是与 Copyright（版权所有）相对立的新名称，它是 GNU 项目制定的通用公共许可证（General Public License, GPL）。GNU 项目是由理查德·斯托尔曼（Richard Stallman）于 1984 年提出的。他建立了自由软件基金会（Free Software Foundation, FSF），并提出 GNU 计划的目的是开发一个完全自由的、与 UNIX 类似但功能更强大的操作系统，以便为所有的计算机用户提供一个功能齐全、性能良好的基本系统。

第 3 节 Linux 的特点与优势

Linux 操作系统作为一个自由、开放的操作系统，其发展势不可当。它拥有高效、安全、稳定，支持多种硬件平台，用户界面友好，网络功能强大，以及支持多任务、多用户等特点。

Linux 操作系统是多用户多任务的操作系统，允许多个用户同时登录系统，使用系统资源。多用户特性意味着系统可以同时支持多个用户账户，每个用户拥有独立的文件权限和系统资源访问权限，互不干扰。多任务特性则允许系统同时运行多个程序，每个程序都可以独立地占用 CPU 时间片，实现并发执行。这种设计使得 Linux 非常适合服务器环境，能够高效地处理多个用户的请求和多个任务的并发执行。

- 开源与自由软件精神
- 稳定性与可靠性
- 安全性
- 高性能与高并发处理能力
- 灵活性与可定制性

- 广泛的硬件支持

第 4 节 Linux 的应用领域

- 服务器领域
- 嵌入式系统
- 云计算与大数据
- 移动设备（Android 基于 Linux 内核）
- 桌面应用
- 科学计算与人工智能

第 5 节 Linux 的体系结构

Linux 一般由 3 个部分组成：内核（Kernel）、命令解释层（shell 或其他操作环境）、实用工具。

内核是系统的”心脏”，是运行程序、管理磁盘及打印机等硬件设备的核心程序。命令解释层向用户提供一个操作界面，从用户那里接受命令，并且把命令送给内核去执行。由于内核提供的都是操作系统最基本的功能，所以如果内核发生问题，那么整个计算机系统就可能会崩溃。

shell 是系统的用户界面，提供用户与内核进行交互操作的接口。它接收用户输入的命令，并且将命令送入内核去执行。

命令解释层在操作系统内核与用户之间提供操作界面，可以称其为一个解释器。操作系统对用户输入的命令进行解释，再将其发送到内核。Linux 存在几种操作环境，分别是桌面（desktop）、窗口管理器（window manager）和命令行 shell（command line shell）。Linux 操作系统中的每个用户都可以拥有自己的用户操作界面，即根据自己的需求进行定制。

shell 也是一个命令解释器，解释由用户输入的命令，并把命令送到内核。不仅如此，shell 还有自己的编程语言，可用于命令的编辑，它允许用户编写由 shell 命令组成的程序。shell 编程语言具有普通编程语言的很多特点，如它也有循环结构和分支控制结构等。用这种编程语言编写的 shell 程序与其他应用程序具有同样的效果。

5.1 shell 提示符

shell 提示符是 shell 向用户显示的命令输入提示，通常显示在命令行的开头。不同的 Linux 发行版和 shell 类型可能会有不同的提示符格式，但它们通常包含以下信息：

- 用户名：当前登录的用户名称
- 主机名：系统的主机名称
- 当前目录：用户当前所在的工作目录
- 权限级别：通常用不同的符号表示，如 root 用户用 #，普通用户用 \$

常见的 shell 提示符格式：

root 用户提示符示例

```
[root@localhost ~]#
```

普通用户提示符示例

```
[user@localhost ~]$
```

其中：

- root 或 user 是用户名
- localhost 是主机名
- ~ 表示用户的主目录
- # 表示 root 用户，\$ 表示普通用户

用户可以通过修改 shell 配置文件（如 ~/.bashrc）来自定义提示符的格式，使其显示更多或更简洁的信息。

标准的 Linux 操作系统都有一套叫作实用工具的程序，它们是专门的程序，如编辑器、执行标准的计算操作等。用户也可以使用自己的工具。

实用工具可分为以下 3 类：

- 编辑器：用于编辑文件
- 过滤器：用于接收数据并过滤数据
- 交互程序：允许用户发送信息或接收来自其他用户的信息

第 6 节 Linux 的版本

Linux 的版本分为内核版本和发行版本两种。

6.1 内核版本

内核是系统的”心脏”，是运行程序、管理磁盘及打印机等硬件设备的核心程序，提供了一个在裸设备与应用程序间的抽象层。例如，程序本身不需要了解用户的主板芯片集或磁盘控制器的细节就能在高层次上读/写磁盘。

内核的开发和规范一直由莱纳斯领导的开发小组控制着，版本也是唯一的。开发小组每隔一段时间公布新的版本或其修订版，从 1991 年 10 月莱纳斯向世界公开发布的内核 0.0.2 版本（0.0.1 版本功能相当”简陋”，所以没有公开发布），到目前最新的内核 5.10.12 版本，Linux 的功能越来越强大。

Linux 内核的版本号命名是有一定规则的，版本号的格式通常为”主版本号. 次版本号. 修正号”。主版本号和次版本号标志着重要的功能变更，修正号表示较小的功能变更。以 2.6.12 为例，2 代表主版本号，6 代表次版本号，12 代表修正号。

可以到 Linux 内核官方网站下载最新的内核代码。

6.2 发行版本

仅有内核而没有应用软件的操作系统是无法使用的，所以许多公司或社团将内核、源代码及相关的应用程序组织构成一个完整的操作系统，让一般的用户可以简便地安装和使用 Linux，这就是所谓的发行版（Distribution）。一般谈论的 Linux 操作系统便是针对这些发行版的。目前各种发行版超过 300 种，它们的发行版本号各不相同，使用的内核版本号也可能不一样，现在流行的 Linux 操作系统套件有 RHEL、CentOS、Fedora、openSUSE、Debian、Ubuntu 等。

1.6.2.1 RHEL 8

作为面向云环境和企业 IT 的强大企业级 Linux 操作系统，RHEL8 版本于 2019 年 5 月 8 日发布。在 RHEL 7 系列发布约 5 年之后，RHEL 8 在优化诸多核心组件的同时引入了诸多强大的新功能，支持各种工作负载，从而可以让用户轻松驾驭各种环境。

RHEL 8 为”混合云时代”的到来引入了大量新功能，包括用于配置、管理和修复 RHEL 8 的 Red Hat Smart Management 扩展程序，以及包含快速迁移框架、编程语言和诸多开发者工具在内的 Application Streams。

RHEL 8 同时对管理员和管理区域进行了改善，让系统管理员、Windows 管理员更容易访问。此外，通过 Red Hat Enterprise Linux System Roles，Linux 初学者可以更快地自动化执行复杂任务，以及通过 RHEL Web 控制台管理和监控 RHEL 的运行状况。

在安全方面，RHEL 8 内置了对 OpenSSL 1.1.1 和 TLS 1.3 加密标准的支持。它还为 Red Hat 容器工具包提供全面的支持，用于创建、运行和共享容器化应用程序，改进对 ARM 和 POWER 架构、SAP 解决方案和实时应用程序，以及 Red Hat 混合云基础架构的支持。

1.6.2.2 BaseOS 和 AppStream

RHEL 8 提出了一个新的设计理念，将软件仓库分为 BaseOS 和 AppStream 两部分：

BaseOS 以传统 RPM 软件包的形式提供操作系统底层软件的核心集，是基础软件安装库。BaseOS 提供核心操作系统功能，包括系统的基础组件和底层服务。这些软件包具有较长的生命周期，与 RHEL 版本的生命周期一致，确保系统的稳定性和可靠性。BaseOS 中的软件包通常不会频繁更新，主要提供安全补丁和错误修复。

AppStream（应用程序流） 包括额外的用户空间应用程序、运行时语言和数据库，以支持不同的工作负载和用例。AppStream 中的内容有两种格式：

- 熟悉的传统 RPM 格式
- 称为模块（Module）的 RPM 格式扩展

AppStream 采用模块化设计，允许在独立的生命周期中安装其他版本的软件，同时保留核心操作系统软件包。这使用户能够安装同一个程序的多个主要版本，例如同时安装不同版本的 Python 或 Node.js。

通过 AppStream，用户可以更轻松地升级用户空间软件包，而不会影响核心操作系统的稳定性。AppStream 中的软件包通常有更频繁的更新周期，以提供最新的功能和改进。

这种分离设计使得 RHEL 8 能够更好地平衡系统稳定性和应用程序创新性，满足现代企业环境中不同工作负载的需求。

第二章 Linux 系统安装与配置

第 1 节 系统安装前的准备

- 硬件要求与兼容性检查
- 发行版选择建议
- 安装介质制作（U 盘/光盘）
- 分区规划原则

在启动 RHEL 8 安装程序前，需根据实际情况的不同，准备 RHEL 8 DVD 安装映像，同时要分区规划。

对于初次接触 Linux 的用户来说，分区方案越简单越好，所以最好的选择就是为 Linux 准备 3 个分区，即用户保存系统和数据的根分区（/）、启动分区（/boot）和交换分区（swap）。其中，交换分区不用太大，与物理内存同样大小即可；启动分区用于保存系统启动时所需要的文件，一般 500MB 就够了；根分区则需要根据 Linux 操作系统安装后占用资源的大小和所需要保存数据的多少来调整大小（一般情况下，划分 15GB~20GB 就足够了）。

特别注意：如果选择的固件类型为“UEFI”，则 Linux 操作系统至少必须建立 4 个分区：根分区、启动分区、EFI 启动分区（/boot/efi）和交换分区。

当然，对于“Linux 熟手”，或者要安装服务器的管理员来说，这种分区方案就不太适合了。此时，一般会再创建一个/usr 分区，操作系统基本都在这个分区中；还需要创建一个/home 分区，所有的用户信息都在这个分区下；还有/var 分区，服务器的登录文件、邮件、Web 服务器的数据文件都会放在这个分区中。

第 2 节 Linux 安装过程

任何硬盘在使用前都要进行分区。硬盘的分区有两种类型：主分区和扩展分区。RHEL 8 提供了多达 4 种安装方式支持，可以从 CD-ROM/DVD 启动安装、从硬盘安装、从 NFS 服务器安装或者从 FTP/HTTP 服务器安装。

可扩展固件接口（Unified Extensible Firmware Interface, UEFI）启动需要一个独立的分区，它将系统启动文件和操作系统本身隔离，可以更好地保护系统的启动。

UEFI 启动方式支持的硬盘容量更大。传统的基本输入输出系统（Basic Input Output System, BIOS）启动由于受主引导记录（Master Boot Record, MBR）的限制，默认无法引导 2.1TB 以上的硬盘。随着硬盘价格的不断下降，2.1TB 以上的硬盘会逐渐普及，因此 UEFI 启动也是今后主流的启动方式。

- BIOS/UEFI 设置
- 图形化安装界面操作
- 分区方案（MBR/GPT，根分区，swap 分区，/home 分区等）
- 系统引导程序配置（GRUB）
- 用户设置与网络配置

第 3 节 系统初始化与基本配置

- 首次启动与登录
- 网络配置（静态 IP/动态 IP）
- 系统更新与软件源配置
- 语言与时区设置
- 基本硬件驱动安装

3.1 Linux 开机过程

Linux 操作系统的开机过程按以下步骤进行：

1. **BIOS/UEFI 初始化：**计算机启动后，首先执行 BIOS（基本输入输出系统）或 UEFI（统一可扩展固件接口）的自检，检测硬件设备并确定启动设备顺序。
2. **Boot Loader 启动：**从启动设备加载引导加载程序（如 GRUB），引导加载程序负责加载 Linux 内核。
3. **内核加载：**引导加载程序将内核镜像加载到内存并执行，内核开始初始化系统硬件。
4. **内核初始化：**内核检测并初始化各种硬件设备，挂载根文件系统，准备运行用户空间程序。
5. **启动初始化进程：**内核启动第一个用户空间进程（PID 为 1），即初始化进程，负责完成系统的初始化工作。

3.2 systemd 初始化进程

初始化进程作为 Linux 操作系统的第一个进程，需要完成 Linux 操作系统中相关的初始化工作，为用户提供合适的工作环境。

RHEL 8 已经替换了传统的 System V init 初始化进程服务，正式采用全新的 systemd 初始化进程服务。systemd 具有以下特点：

- **并发启动机制：**systemd 采用并行启动服务的方式，大大缩短了开机时间。
- **统一的服务管理：**使用 systemctl 命令管理系统服务，替代了传统的 service 和 chkconfig 命令。
- **依赖关系管理：**自动处理服务间的依赖关系，确保服务按正确顺序启动。
- **按需启动：**支持服务的按需启动，提高系统资源利用率。
- **日志管理：**集成了 journald 日志系统，提供统一的日志管理。

通过这些改进，systemd 显著提升了 Linux 系统的开机速度和管理效率，成为现代 Linux 发行版的标准初始化系统。

目标 (Target) 替代运行级别 RHEL 8 采用 systemd 后，不再使用传统的“运行级别”概念。Linux 操作系统在启动时要进行大量的初始化工作，如挂载文件系统、交换分区和启动各类进程服务等，这些都被视为一个一个的单元 (Unit)。

systemd 用目标 (Target) 代替了 System V init 中运行级别的概念。目标是一组相关单元的集合，用于实现特定的系统状态。例如：

- **multi-user.target：**对应传统的运行级别 3，提供多用户命令行界面
- **graphical.target：**对应传统的运行级别 5，提供图形用户界面
- **rescue.target：**对应传统的单用户模式，用于系统救援
- **poweroff.target：**关机状态
- **reboot.target：**重启状态

这种基于目标的设计更加灵活和可扩展，能够更好地适应现代 Linux 系统的需求。

2.3.2.1 启动级别与 Target 对应关系

System V init 的运行级别与 systemd 的 target 之间存在以下对应关系：

System V 运行级别	描述	对应的 systemd 目标
0	关机	poweroff.target
1	单用户模式	rescue.target
2	多用户模式（无网络）	multi-user.target
3	多用户模式（有网络）	multi-user.target
4	未使用	无直接对应
5	图形界面模式	graphical.target
6	重启	reboot.target
无对应运行级别	紧急模式（最基础的救援模式）	emergency.target

表 2.1: System V init 运行级别与 systemd 目标对应关系

关于 **emergency.target** emergency.target 是 systemd 中的紧急救援模式，它提供了最基础的系统访问环境，比 rescue.target 更加基础，通常用于解决严重的系统问题。

在 emergency.target 模式下：

- 系统只挂载根文件系统为只读模式
- 不启动任何网络服务
- 不启动任何其他系统服务
- 只提供最基本的 shell 环境
- 适合处理严重的文件系统损坏、引导配置错误等问题

相比之下，rescue.target 会挂载所有本地文件系统，启动一些基本服务，提供更完整的救援环境。

3.3 systemd 与 System V init 的区别

3.4 systemctl 常用命令

3.5 timedatectl 命令

timedatectl 命令对 RHEL /CentOS 7 的分布式系统来说，是一个新工具，RHEL 8 仍然沿用。

timedatectl 命令作为 systemd 系统和服务管理器的一部分，代替旧的、传统的、用于基于 Linux 分布式系统的 sysvinit 守护进程的 date 命令。

timedatectl 命令用于管理系统时间和日期设置。timedatectl 命令可以查询和更改系统时钟和设置，可以使用此命令来设置或更改当前的日期、时间和时区，或实现与远程 NTP 服务器的自动系统时钟同步。

特性	System V init	systemd
启动方式	串行启动, 服务按顺序启动	并行启动, 服务同时启动
服务管理	使用 service 命令	使用 systemctl 命令
运行级别	使用 runlevel 概念 (0-6)	使用 target 概念替代运行级别
配置文件	位于 /etc/init.d/, 使用脚本	位于 /etc/systemd/system/, 使用 unit 文件
依赖管理	手动管理服务依赖关系	自动管理服务依赖关系
开机速度	较慢, 因为串行启动	较快, 因为并行启动
日志管理	使用 syslog	集成 journald 日志系统
按需启动	不支持	支持服务的按需启动
系统状态	通过 runlevel 命令查看	通过 systemctl status 命令查看
默认采用	RHEL 7 及之前版本	RHEL 8 及之后版本

表 2.2: systemd 与 System V init 的区别

它提供了以下功能:

- 查询当前系统时间和日期配置
- 设置或更改系统时间和日期
- 管理系统时区设置
- 实现与远程 NTP 服务器的自动系统时钟同步
- 管理硬件时钟设置

查看当前系统时间和日期配置

```
timedatectl
```

查看所有可用的时区

```
timedatectl list-timezones
```

设置系统时区为上海

```
timedatectl set-timezone Asia/Shanghai
```

设置系统时间

命令	功能描述	示例
<code>systemctl status</code>	查看服务状态	<code>systemctl status sshd</code>
<code>systemctl start</code>	启动服务	<code>systemctl start sshd</code>
<code>systemctl stop</code>	停止服务	<code>systemctl stop sshd</code>
<code>systemctl restart</code>	重启服务	<code>systemctl restart sshd</code>
<code>systemctl reload</code>	重新加载服务配置	<code>systemctl reload sshd</code>
<code>systemctl enable</code>	启用服务开机自启	<code>systemctl enable sshd</code>
<code>systemctl disable</code>	禁用服务开机自启	<code>systemctl disable sshd</code>
<code>systemctl is-enabled</code>	检查服务是否开机自启	<code>systemctl is-enabled sshd</code>
<code>systemctl list-units</code>	列出所有活动的单元	<code>systemctl list-units</code>
<code>systemctl list-unit-files</code>	列出所有单元文件	<code>systemctl list-unit-files</code>
<code>systemctl list-dependencies</code>	查看服务依赖关系	<code>systemctl list-dependencies sshd</code>
<code>systemctl isolate</code>	切换到指定目标	<code>systemctl isolate multi-user.target</code>
<code>systemctl get-default</code>	查看默认目标	<code>systemctl get-default</code>
<code>systemctl set-default</code>	设置默认目标	<code>systemctl set-default graphical.target</code>
<code>systemctl poweroff</code>	关机	<code>systemctl poweroff</code>
<code>systemctl reboot</code>	重启	<code>systemctl reboot</code>
<code>systemctl suspend</code>	挂起系统	<code>systemctl suspend</code>
<code>systemctl hibernate</code>	使系统休眠	<code>systemctl hibernate</code>

表 2.3: systemctl 常用命令

```
timedatectl set-time "2024-01-27 14:30:00"
```

```
# 启用NTP时间同步
```

```
timedatectl set-ntp yes
```

```
# 禁用NTP时间同步
```

```
timedatectl set-ntp no
```

```
# 查看NTP同步状态
```

```
timedatectl status
```

注意事项： - timedatectl 命令是 systemd 的一部分，在使用 systemd 的现代 Linux 发行版中可用 - 对于传统的 System V init 系统，需要使用 date 和 hwclock 等命令来管理时间 - 当启用 NTP 同步后，系统会自动从时间服务器获取准确时间

3.6 clock 命令

clock 命令用于从计算机的硬件获得日期和时间，它是 hwclock 命令的别名。在传统的 System V init 系统中，clock 命令是管理硬件时钟的主要工具。

clock 命令的功能：

- 显示硬件时钟的当前时间
- 设置硬件时钟的时间
- 将系统时间同步到硬件时钟
- 将硬件时钟同步到系统时间

```
# 显示硬件时钟的当前时间
```

```
clock
```

```
# 显示硬件时钟的当前时间（使用详细格式）
```

```
clock --show
```

```
# 将系统时间同步到硬件时钟
```

```
clock --systohc
```

```
# 将硬件时钟同步到系统时间
```

```
clock --hctosys
```

设置硬件时钟的时间

```
clock --set --date="2024-01-27 14:30:00"
```

clock 命令的常用选项：

- `-show`：显示硬件时钟的当前时间
- `-set`：设置硬件时钟的时间
- `-date`：指定日期和时间
- `-systohc`：将系统时间同步到硬件时钟
- `-hctosys`：将硬件时钟同步到系统时间
- `-utc`：使用 UTC 时间
- `-localtime`：使用本地时间

使用说明：clock 命令和 hwclock 命令功能完全相同，它们是彼此的别名。在现代 Linux 发行版中，虽然 systemd 提供了 timedatectl 命令来管理时间，但 clock/hwclock 命令仍然可用，用于直接操作硬件时钟。

3.7 重置 root 管理员密码

重启 Linux 主机并出现引导界面时，按“e”键进入内核编辑界面。

在 linux 参数行的最后面追加“rd.break”参数，然后按“Ctrl + X”组合键来运行修改过的内核程序。

大约 30s 后，系统进入紧急救援模式。依次输入以下命令，等待系统重启操作完毕，就可以使用新密码 newredhat 来登录 Linux 操作系统了。

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd
touch ./autorelabel
exit
reboot
```

第 4 节 多重引导配置

Linux 和 Windows 的多重引导（多系统引导）有多种实现方式，常用的有 3 种。

在这 3 种实现方式中，目前用户使用最多的是通过 Linux 的 GRUB 或者 LILO 实现 Windows、Linux 多重引导。

第三章 Linux 命令行基础

第 1 节 命令行界面简介

Linux 命令是对 Linux 操作系统进行管理的命令。对于 Linux 操作系统来说，无论是中央处理器、内存、磁盘驱动器、键盘、鼠标，还是用户等，都是文件。Linux 命令是 Linux 正常运行的核心，与 dos 命令类似。

1.1 Linux 命令的详细说明

1. ”一切皆文件”哲学：Linux 确实将几乎所有资源（包括硬件设备、进程、用户等）都抽象为文件，这是其设计的核心思想之一。这种设计使得系统管理更加统一和灵活。
2. 命令的执行机制：Linux 命令通常通过 shell（如 Bash）解释执行，shell 作为命令解释器，将用户输入的命令转换为内核可理解的指令。
3. 命令的分类：
 - 内置命令：shell 内置的命令（如 cd、echo）
 - 外部命令：独立的可执行文件（如 ls、cp）
 - 系统调用：直接与内核交互的底层命令
4. 与 DOS 命令的区别：虽然两者都是命令行界面，但 Linux 命令在功能、语法和灵活性方面更为强大，支持管道、重定向、通配符等高级特性。
5. 命令的组成：完整的 Linux 命令通常包括命令名、选项和参数三部分，如 `ls -la /home`。
6. 命令的大小写敏感性：在 Linux 操作系统中，命令区分大小写。
7. 命令自动补齐：在命令行中，可以使用“Tab”键来自动补齐命令，即可以只输入命令的前几个字母，然后按“Tab”键补齐。按“Tab”键时，如果系统只找到一个与输入字符相匹配的目录或文件，则自动补齐；如果没有匹配的内容或多个相匹配的名字，系统将发出警鸣声，再按“Tab”键将列出所有相匹配的内容（如果有），以供用户选择。

8. 历史命令：利用向上或向下的方向键，可以翻查曾经执行过的命令，并可以再次执行。
 9. 多命令执行：如果要在一个命令行上输入和执行多条命令，可以使用分号来分隔命令，如“`cd /;ls`”。
 10. 长命令换行：如果要断开一个长命令行，可以使用反斜杠“`\`”。它可以将一个较长的命令分成多行表达，增强命令的可读性。执行后，shell 自动显示提示符“`>`”，表示正在输入一个长命令，此时可继续在新的命令行上输入命令的后续部分。
 11. 后台执行命令：一个文本控制台或一个仿真终端在同一时刻只能执行一个程序或命令。在执行结束前，一般不能进行其他操作。此时可采用在后台执行程序的方式，以释放控制台或终端，使其仍能进行其他操作。要使程序以后台方式执行，只需在要执行的命令后跟上一个“`&`”符号即可，如“`top &`”。
- 终端与 Shell 的概念
 - 常见 Shell（Bash, Zsh, Fish 等）
 - 命令行提示符结构
 - 快捷键与命令历史

第 2 节 文件系统与目录结构

定义 3.1 (文件系统). 文件系统 (*File System*) 是磁盘上有特定格式的一片区域，操作系统可利用文件系统保存和管理文件。

用户在硬件存储设备中执行的文件建立、写入、读取、修改、转存与控制等操作都是依靠文件系统来完成的。文件系统的作用是合理规划硬盘，以满足用户正常的使用需求。

2.1 硬盘分区表格式

硬盘按分区表的格式可以分为主引导记录 (Master Boot Record, MBR) 硬盘与全局唯一标识磁盘分区表 (GUID Partition Table, GPT) 硬盘这两种硬盘格式 (style):

- **MBR 硬盘**: 使用的是旧的传统硬盘分区表格式，其硬盘分区表存储在 MBR 内。MBR 位于硬盘最前端，计算机启动时，使用传统的 BIOS (固化在计算机主板 ROM 芯片上的程序)，BIOS 会先读取 MBR，并将控制权交给 MBR 内的程序代码，然后由此程序代码来继续完成后续的启动工作。MBR 硬盘支持的硬盘最大容量为 2.2 TB (1TB=1024GB)。

- **GPT 硬盘：**一种新的硬盘分区表格式，其硬盘分区表存储在 GPT 内。GPT 位于硬盘的前端，而且它有主分区表与备份分区表，可提供容错功能。使用新式 UEFI BIOS 的计算机，其 BIOS 会先读取 GPT，并将控制权交给 GPT 内的程序代码，然后由此程序代码来继续完成后续的启动工作。GPT 硬盘支持的硬盘容量可以超过 2.2 TB。

2.2 硬盘分区

在数据存储到硬盘之前，该硬盘必须被分割成一个或数个硬盘分区（partition）。在硬盘内有一个称为硬盘分区表（partition table）的区域，用来存储硬盘分区的相关数据，如每一个硬盘分区的起始地址、结束地址、是否为活动（active）的硬盘分区等信息。

硬盘设备是由大量的扇区组成的，每个扇区的容量为 512B，其中第一个扇区最重要。第一个扇区里面保存着主引导记录与硬盘分区表信息。就第一个扇区来讲，主引导记录需要占用 446B，硬盘分区表为 64B，结束符占用 2B。其中硬盘分区表中每记录一个分区信息就需要 16B，这样一来，最多只有 4 个分区信息可以写到第一个扇区中，这 4 个分区就是 4 个主分区。

第一个扇区最多只能创建出 4 个分区，为了解决分区数不够的问题，可以将第一个扇区分区表中 16B（原本要写入主分区信息）的空间拿出来指向另外一个分区（称之为扩展分区）。也就是说，扩展分区其实并不是一个真正的分区，而更像是一个占用 16B 分区表空间的指针——一个指向另外一个分区的指针。用户一般会选择使用 3 个主分区加 1 个扩展分区的方法，在扩展分区中创建出数个逻辑分区，从而满足多分区（大于 4 个）的需求。

扩展分区严格地讲不是一个实际意义的分区，它仅仅是指向下一个分区的指针，这种指针结构将形成一个单向链表。

2.3 分区工具

Linux 系统中常用的分区工具包括 fdisk、cfdisk 和 parted 等，它们可以用来创建、删除、修改硬盘分区。

fdisk 工具： fdisk 硬盘分区工具在 DOS、Windows 和 Linux 中都有相应的应用程序。在 Linux 操作系统中，fdisk 是基于菜单的命令。对硬盘进行分区时，可以在 fdisk 命令后面直接加上要分区的硬盘作为参数。fdisk 是最常用的分区工具之一，适用于 MBR 分区表格式，支持交互式操作。

查看硬盘分区情况

```
fdisk -l
```

对指定硬盘进行分区操作

```
fdisk /dev/sda
```

fdisk 命令的选项及功能：

- -l: 列出所有硬盘的分区情况
- -s <分区>: 显示指定分区的大小（以块为单位）
- -u: 以扇区为单位显示分区表
- -b <大小>: 指定扇区大小（512、1024、2048 或 4096）
- -C <柱面数>: 指定柱面数
- -H <磁头数>: 指定磁头数
- -S <扇区数>: 指定每个磁道的扇区数

fdisk 交互式命令： 在 fdisk 交互式界面中，可以使用以下命令：

- m: 显示帮助信息
- n: 创建新分区
- d: 删除分区
- p: 显示当前分区表
- t: 修改分区类型
- a: 设置活动分区
- w: 保存并退出
- q: 不保存并退出
- o: 创建新的空 DOS 分区表
- s: 创建新的空 Sun 分区表

cfdisk 工具： cfdisk 是一个基于文本界面的分区工具，提供了更友好的交互式操作界面，适用于 MBR 分区表格式。

对指定硬盘进行分区操作

```
cfdisk /dev/sda
```

parted 工具： parted 是一个功能更强大的分区工具，支持 MBR 和 GPT 分区表格式，适用于处理大硬盘和新的分区表格式。

查看硬盘分区情况

```
parted -l
```

对指定硬盘进行分区操作

```
parted /dev/sda
```

Linux 操作系统支持数十种文件系统，常见的文件系统如下：

(1) Ext4: Ext3 的改进版本，作为 RHEL 6 中默认的文件管理系统，它支持的存储容量高达 1EB (1EB=1 073 741 824GB)，且有足够多的子目录。另外，Ext4 文件系统能够批量分配块 (block)，从而极大地提高了读/写效率。

(2) XFS: 一种高性能的日志文件系统，而且是 RHEL 7/8 默认的文件管理系统。它的优势在发生意外宕机后尤其明显，可以快速恢复可能被破坏的文件，而且强大的日志功能只需花费极低的文件权限和属性的信息。它最大可支持的存储容量为 18EB，这几乎满足了所有需求。

2.4 使用 mkfs 命令建立文件系统

mkfs (make filesystem) 命令用于在分区上创建文件系统。

mkfs 命令的基本语法：

```
mkfs [选项] [-t <文件系统类型>] <设备>
```

常用的 mkfs 命令选项：

- -t <文件系统类型>: 指定要创建的文件系统类型，如 ext4、xfs、btrfs 等
- -V: 显示详细的操作过程
- -c: 在创建文件系统之前检查坏块
- -L <标签>: 为文件系统设置卷标
- -b <块大小>: 指定文件系统的块大小 (单位为字节)
- -m <保留百分比>: 指定保留给超级用户的块百分比
- -l: 显示支持的文件系统类型列表
- -I <inode大小>: 指定文件系统的 inode 大小 (单位为字节)，主要用于 ext 文件系统

常用的 mkfs 命令变体： Linux 系统中还提供了一些针对特定文件系统的 mkfs 变体命令，它们是 mkfs 的快捷方式：

- `mkfs.ext2`: 创建 Ext2 文件系统
- `mkfs.ext3`: 创建 Ext3 文件系统
- `mkfs.ext4`: 创建 Ext4 文件系统
- `mkfs.xfs`: 创建 XFS 文件系统
- `mkfs.btrfs`: 创建 Btrfs 文件系统
- `mkfs.vfat`: 创建 FAT 文件系统

mkfs 命令示例：

在 /dev/sda1 分区上创建 Ext4 文件系统

```
mkfs -t ext4 /dev/sda1
```

或使用快捷命令

```
mkfs.ext4 /dev/sda1
```

在 /dev/sda2 分区上创建 XFS 文件系统

```
mkfs -t xfs /dev/sda2
```

或使用快捷命令

```
mkfs.xfs /dev/sda2
```

创建文件系统时检查坏块

```
mkfs.ext4 -c /dev/sda1
```

为文件系统设置卷标

```
mkfs.ext4 -L "data" /dev/sda1
```

指定块大小为 4096 字节

```
mkfs.ext4 -b 4096 /dev/sda1
```

显示支持的文件系统类型列表

```
mkfs -l
```

```
# 指定 inode 大小为 256 字节  
mkfs.ext4 -I 256 /dev/sda1
```

注意事项:

- 在使用 mkfs 命令之前，确保已经对硬盘进行了分区
- mkfs 命令会覆盖分区上的所有数据，使用前请务必备份重要数据
- 对于新硬盘，建议先使用 fdisk 或 parted 等工具进行分区，然后再使用 mkfs 创建文件系统
- 不同的文件系统类型有不同的特点和适用场景，选择时应根据实际需求进行考虑

2.5 使用 fsck 命令检查和修复文件系统

fsck (file system check) 命令主要用于检查文件系统的正确性，并对 Linux 硬盘进行修复。

fsck 命令的基本语法:

```
fsck [选项] <设备>
```

常用的 fsck 命令选项:

- -a: 自动修复文件系统，不询问用户
- -r: 交互式修复文件系统，询问用户
- -y: 对所有问题回答“yes”
- -f: 强制检查，即使文件系统被标记为 clean
- -t <文件系统类型>: 指定要检查的文件系统类型
- -C: 显示检查进度
- -N: 模拟检查，不实际执行修复操作
- -s: 按顺序检查文件系统，而不是并行检查
- -A: 检查 /etc/fstab 文件中列出的所有文件系统
- -d: 显示调试信息
- -P: 当与 -A 一起使用时，同时检查所有文件系统

常用的 fsck 命令变体： Linux 系统中还提供了一些针对特定文件系统的 fsck 变体命令，它们是 fsck 的快捷方式：

- `fsck.ext2`: 检查和修复 Ext2 文件系统
- `fsck.ext3`: 检查和修复 Ext3 文件系统
- `fsck.ext4`: 检查和修复 Ext4 文件系统
- `fsck.xfs`: 检查和修复 XFS 文件系统
- `fsck.btrfs`: 检查和修复 Btrfs 文件系统
- `fsck.vfat`: 检查和修复 FAT 文件系统

fsck 命令示例：

检查 /dev/sda1 分区的文件系统

```
fsck /dev/sda1
```

自动修复文件系统

```
fsck -a /dev/sda1
```

强制检查文件系统

```
fsck -f /dev/sda1
```

检查指定类型的文件系统

```
fsck -t ext4 /dev/sda1
```

或使用快捷命令

```
fsck.ext4 /dev/sda1
```

检查所有文件系统，显示进度，按顺序执行，显示调试信息

```
fsck -s -A -C -d
```

检查所有文件系统，同时执行，自动修复

```
fsck -A -P -a
```

检查所有文件系统，交互式修复

```
fsck -A -r
```


注意事项:

- 不要在已挂载的文件系统上使用 `fsck` 命令，否则可能会导致文件系统损坏
- 对于根文件系统，建议在单用户模式下或从救援光盘启动后进行检查
- 使用 `fsck` 命令修复文件系统时，可能会丢失一些数据，建议在操作前备份重要数据
- 对于 XFS 文件系统，建议使用 `xfs_repair` 命令进行修复，而不是 `fsck.xfs`

2.6 使用 dd 命令进行文件复制和转换

`dd` 命令是 Linux 系统中一个非常强大的命令，主要用于复制文件和转换文件格式，也常用于磁盘操作，如创建磁盘镜像、备份和恢复数据等。`dd` 命令的特点是可以精确控制数据的复制过程，包括块大小、偏移量等参数。

dd 命令的基本语法:

`dd if=<输入文件> of=<输出文件> [选项]`

常用的 dd 命令选项:

- `if=<输入文件>`: 指定输入文件，默认为标准输入
- `of=<输出文件>`: 指定输出文件，默认为标准输出
- `bs=<块大小>`: 指定块大小，默认为 512B
- `count=<块数>`: 指定要复制的块数
- `skip=<块数>`: 从输入文件的指定块数后开始复制
- `seek=<块数>`: 从输出文件的指定块数后开始写入
- `conv=<转换选项>`: 指定转换选项，如 `notrunc`、`noerror`、`sync` 等
- `status=<状态显示>`: 指定状态显示方式，如 `progress`（显示进度）
- `iflag=<输入标志>`: 指定输入文件的标志
- `oflag=<输出标志>`: 指定输出文件的标志

常用的转换选项 (conv 参数):

- notrunc: 不截断输出文件
- noerror: 遇到错误时继续执行
- sync: 将输入数据的每个块填充到 bs 指定的大小
- ucase: 将输入数据转换为大写
- lcase: 将输入数据转换为小写
- swab: 交换输入数据的每对字节
- block: 将输入数据转换为块
- unblock: 将输入数据转换为非块

dd 命令示例:

创建一个 1GB 的空文件

bs=1M 表示块大小为 1MB, count=1024 表示复制 1024 个块, 总共 1GB

```
dd if=/dev/zero of=empty.file bs=1M count=1024
```

备份整个磁盘到镜像文件

if=/dev/sda 表示输入为整个 sda 磁盘, of=sda.img 表示输出到 sda.img 文件

```
dd if=/dev/sda of=sda.img bs=4M
```

从镜像文件恢复磁盘

if=sda.img 表示输入为镜像文件, of=/dev/sda 表示输出到 sda 磁盘

```
dd if=sda.img of=/dev/sda bs=4M
```

备份磁盘分区到镜像文件

```
dd if=/dev/sda1 of=sda1.img bs=4M
```

从镜像文件恢复分区

```
dd if=sda1.img of=/dev/sda1 bs=4M
```

创建一个 USB 启动盘

if=ubuntu.iso 表示输入为 Ubuntu ISO 文件, of=/dev/sdb 表示输出到 USB 设备

```
dd if=ubuntu.iso of=/dev/sdb bs=4M status=progress
```

```
# 测试硬盘读写速度
# if=/dev/zero 表示从空设备读取数据, of=/dev/null 表示写入到空设备 (丢弃数据)
# bs=1M 表示块大小为 1MB, count=1024 表示复制 1024 个块, 总共 1GB
dd if=/dev/zero of=/dev/null bs=1M count=1024

# 复制文件并转换格式
# conv=ucase 表示将输入数据转换为大写
dd if=input.txt of=output.txt conv=ucase

# 跳过输入文件的前 100 块, 从第 101 块开始复制
dd if=input.file of=output.file bs=512 skip=100

# 从输入文件的第 100 块开始复制, 写入到输出文件的第 50 块位置
dd if=input.file of=output.file bs=512 skip=100 seek=50

# 备份MBR (主引导记录)
dd if=/dev/sda of=mbr_backup.img bs=512 count=1

# 测试磁盘读写速度 (使用direct选项绕过缓存, 更真实的速度测试)
time dd if=/dev/zero of=testfile bs=1M count=1000 oflag=direct
time dd if=testfile of=/dev/null bs=1M count=1000 iflag=direct
rm testfile
```

注意事项:

- dd 命令执行时需要小心, 特别是当操作磁盘设备时, 错误的参数可能会导致数据丢失
- 使用 dd 命令备份或恢复磁盘时, 建议在单用户模式下或从救援光盘启动后进行
- 对于大文件或磁盘操作, 建议使用较大的 bs 值 (如 4M 或 8M) 来提高速度
- 使用 status=progress 选项可以在执行过程中显示进度, 避免长时间等待而不知道执行状态
- 当复制磁盘或分区时, 确保目标设备的容量不小于源设备

2.7 交换分区管理命令: mkswap、swapon、swapoff

交换分区 (swap partition) 是 Linux 系统中用于虚拟内存管理的重要组成部分。当系统物理内存不足时, 会将部分内存数据交换到交换分区中, 以释放物理内存空间。以

下是管理交换分区的常用命令：

3.2.7.1 mkswap 命令

mkswap 命令用于在指定设备或文件上创建交换分区。

mkswap 命令的基本语法：

mkswap [选项] <设备或文件> [大小]

常用的 mkswap 命令选项：

- -c：创建交换分区前检查坏块
- -f：强制在指定设备上创建交换分区，即使该设备可能正在使用
- -L <标签>：为交换分区设置标签
- -v0：创建旧版本的交换分区（2.2 内核之前）
- -v1：创建新版本的交换分区（2.2 内核及之后）

mkswap 命令示例：

在 /dev/sda5 分区上创建交换分区

```
mkswap /dev/sda5
```

在 /dev/sda5 分区上创建交换分区并检查坏块

```
mkswap -c /dev/sda5
```

为交换分区设置标签

```
mkswap -L SWAP /dev/sda5
```

创建一个 1GB 的交换文件

首先创建文件

```
fallocate -l 1G /swapfile
```

或使用 dd 命令

```
# dd if=/dev/zero of=/swapfile bs=1M count=1024
```

设置适当的权限

```
chmod 600 /swapfile
```

创建交换分区

```
mkswap /swapfile
```

3.2.7.2 swapon 命令

swapon 命令用于启用交换分区。

swapon 命令的基本语法：

swapon [选项] [设备或文件]

常用的 swapon 命令选项：

- -a: 启用 /etc/fstab 文件中所有的交换分区
- -d: 显示调试信息
- -e: 如果交换分区有错误，则跳过
- -f: 强制启用交换分区，即使该分区可能正在使用
- -p <优先级>: 设置交换分区的优先级（0-32767，值越高优先级越高）
- -s: 显示所有已启用的交换分区的详细信息

swapon 命令示例：

启用 /dev/sda5 交换分区

```
swapon /dev/sda5
```

启用交换文件

```
swapon /swapfile
```

启用所有在 /etc/fstab 中定义的交换分区

```
swapon -a
```

设置交换分区的优先级

```
swapon -p 10 /dev/sda5
```

显示所有已启用的交换分区

```
swapon -s
```

或使用 free 命令查看

```
free -h
```

3.2.7.3 swapoff 命令

swapoff 命令用于禁用交换分区。

swapoff 命令的基本语法：

`swapoff [选项] [设备或文件]`

常用的 swapoff 命令选项：

- `-a`: 禁用所有交换分区
- `-v`: 显示详细信息

swapoff 命令示例：

禁用 /dev/sda5 交换分区

```
swapoff /dev/sda5
```

禁用交换文件

```
swapoff /swapfile
```

禁用所有交换分区

```
swapoff -a
```

2.8 磁盘使用情况命令：df

df 命令用于显示文件系统的磁盘使用情况，包括文件系统的总空间、已用空间、可用空间以及挂载点等信息。

df 命令的基本语法：

`df [选项] [文件或目录]`

常用的 df 命令选项：

- `-a`: 显示所有文件系统，包括大小为 0 的文件系统
- `-B <大小>`: 指定显示时的块大小（如 K、M、G 等）
- `-h`: 以人类可读的格式显示（如 1K、234M、2G 等）
- `-H`: 以 1000 为基数的人类可读格式显示（如 1KB、234MB、2GB 等）
- `-i`: 显示 inode 使用情况，而不是块使用情况
- `-k`: 以 KB 为单位显示
- `-l`: 只显示本地文件系统

- `-m`: 以 MB 为单位显示
- `-t <类型>`: 只显示指定类型的文件系统
- `-T`: 显示文件系统类型
- `-x <类型>`: 排除指定类型的文件系统

df 命令示例:

```
# 显示所有文件系统的磁盘使用情况
# 默认以 KB 为单位
# 第一列: 文件系统
# 第二列: 总大小 (KB)
# 第三列: 已用大小 (KB)
# 第四列: 可用大小 (KB)
# 第五列: 使用率
# 第六列: 挂载点
df
```

```
# 以人类可读的格式显示
# 自动选择合适的单位 (K、M、G 等)
df -h
```

```
# 显示指定文件系统类型的使用情况
df -h -t ext4
```

```
# 排除指定文件系统类型
df -h -x tmpfs -x devtmpfs
```

```
# 显示 inode 使用情况
df -i
```

```
# 显示文件系统类型
df -T
```

```
# 只显示本地文件系统
df -l
```

显示指定目录所在文件系统的使用情况

```
df -h /home
```

注意事项：

- df 命令显示的是文件系统级别的使用情况，而不是单个文件的使用情况
- 对于大型文件系统，使用 df 命令可能会花费一些时间
- 当文件系统使用率接近 100% 时，系统性能可能会下降，应及时清理空间

2.9 目录磁盘使用情况命令：du

du 命令用于显示目录或文件的磁盘使用情况，逐级显示指定目录的每一级子目录占用文件系统数据块的情况。与 df 命令不同，du 命令关注的是具体文件和目录的空间使用情况，而不是整个文件系统的使用情况。

du 命令的基本语法：

```
du [选项] [文件或目录]
```

常用的 du 命令选项：

- -a: 显示所有文件和目录的磁盘使用情况，而不仅仅是目录
- -b: 以字节为单位显示（默认以块为单位）
- -c: 显示所有文件和目录的总大小
- -h: 以人类可读的格式显示（如 1K、234M、2G 等）
- -H: 以 1000 为基数的人类可读格式显示（如 1KB、234MB、2GB 等）
- -k: 以 KB 为单位显示
- -m: 以 MB 为单位显示
- -g: 以 GB 为单位显示
- -l: 计算硬链接的大小，即使它们已在其他地方计数
- -s: 只显示总计，不显示子目录的详细信息
- -t <大小>: 只显示超过指定大小的文件或目录
- -x: 不跨越不同的文件系统，只计算同一文件系统内的文件和目录

- `--max-depth=<n>`: 指定显示的目录深度
- `--exclude=<模式>`: 排除符合指定模式的文件或目录

du 命令示例:

显示当前目录及其子目录的磁盘使用情况

默认以块为单位 (通常为 1024 字节)

`du`

显示当前目录及其子目录的磁盘使用情况, 以人类可读的格式

自动选择合适的单位 (K、M、G 等)

`du -h`

显示指定目录的磁盘使用情况

`du -h /home`

只显示当前目录的总大小, 不显示子目录的详细信息

`du -sh`

显示当前目录及其一级子目录的大小

`du -h --max-depth=1`

显示所有文件和目录的大小, 包括文件

`du -ah`

显示所有文件和目录的大小, 并显示总计

`du -ahc`

只显示超过 100MB 的文件或目录

`du -h --max-depth=2 -t 100M`

排除指定模式的文件或目录

`du -h --exclude="*.log" --exclude="temp"`

显示指定文件的大小

`du -h /etc/passwd`

不跨越不同的文件系统，只计算同一文件系统内的文件和目录

```
du -h -x /
```

结合其他选项使用 -x 选项

```
du -h -x --max-depth=1 /
```

du 命令与 df 命令的区别：

- **df 命令：**显示文件系统级别的使用情况，包括总空间、已用空间、可用空间和挂载点
- **du 命令：**显示具体文件和目录的空间使用情况，从指定的目录或文件开始递归计算
- **计算方式：**df 命令从文件系统的超级块中获取信息，而 du 命令通过遍历文件系统中的文件和目录来计算
- **结果差异：**由于文件系统的保留空间、已删除但仍被进程占用的文件等原因，df 和 du 命令的结果可能会有所不同

注意事项：

- 对于包含大量文件和子目录的目录，du 命令可能会花费较长时间
- 使用 -max-depth 选项可以限制显示的目录深度，提高命令执行速度
- 使用 -exclude 选项可以排除不需要计算的文件或目录，进一步提高执行速度

2.10 文件系统挂载管理

在硬盘新建好文件系统之后，还需要把新建的文件系统挂载到系统上才能使用。把新建的文件系统挂载到系统的过程称为挂载。文件系统挂载到的目录称为挂载点 (Mount Point)。

Linux 操作系统提供了 ‘/mnt’ 和 ‘/media’ 两个专门的挂载点。一般而言，挂载点应该是一个空目录，否则目录中原来的文件将被系统隐藏。通常将光盘和软盘挂载到 ‘/media/cdrom’（或者 ‘/mnt/cdrom’）和 ‘/media/floppy’（或者 ‘/mnt/floppy’）中，其对应的设备文件名分别为 ‘/dev/cdrom’ 和 ‘/dev/fd0’。

文件系统可以在系统引导过程中自动挂载，也可以手动挂载。

3.2.10.1 mount 命令

mount 命令用于将文件系统挂载到指定的挂载点。

mount 命令的基本语法:

mount [选项] [设备] [挂载点]

常用的 mount 命令选项:

- -a: 挂载 /etc/fstab 文件中所有未挂载的文件系统
- -t <类型>: 指定文件系统类型
- -o <选项>: 指定挂载选项, 多个选项用逗号分隔
- -r: 以只读方式挂载
- -w: 以读写方式挂载 (默认)
- -v: 显示详细信息
- -n: 不更新 /etc/mtab 文件

常用的挂载选项 (-o):

- ro: 只读挂载
- rw: 读写挂载
- noexec: 不允许执行挂载分区上的可执行文件
- exec: 允许执行挂载分区上的可执行文件 (默认)
- nosuid: 禁用 setuid 和 setgid 位
- suid: 启用 setuid 和 setgid 位 (默认)
- nodev: 不允许访问设备文件
- dev: 允许访问设备文件 (默认)
- auto: 在使用 mount -a 时自动挂载
- noauto: 在使用 mount -a 时不自动挂载
- user: 允许普通用户挂载
- nouser: 只允许 root 用户挂载 (默认)
- defaults: 使用默认挂载选项 (rw, suid, dev, exec, auto, nouser, async)

- **sync**: 同步写入, 所有操作立即写入磁盘
- **async**: 异步写入, 数据先写入缓存, 然后再写入磁盘 (默认)
- **remount**: 重新挂载已挂载的文件系统

mount 命令示例:

挂载 /dev/sda1 分区到 /mnt 目录

```
mount /dev/sda1 /mnt
```

指定文件系统类型挂载

```
mount -t ext4 /dev/sda1 /mnt
```

以只读方式挂载

```
mount -r /dev/sda1 /mnt
```

指定多个挂载选项

```
mount -o rw,noexec,nosuid /dev/sda1 /mnt
```

挂载光盘

```
mount /dev/cdrom /media/cdrom
```

使用 -t iso9660 选项挂载光盘 (明确指定文件系统类型)

```
mount -t iso9660 /dev/cdrom /media/cdrom
```

挂载 USB 设备

```
mount /dev/sdb1 /media/usb
```

挂载网络文件系统 (NFS)

```
mount server:/share /mnt/nfs
```

挂载 ISO 文件

```
mount -o loop ubuntu.iso /mnt/iso
```

挂载所有未挂载的文件系统

```
mount -a
```

3.2.10.2 umount 命令

umount 命令用于卸载已挂载的文件系统。

umount 命令的基本语法：

umount [选项] [设备或挂载点]

常用的 umount 命令选项：

- -a: 卸载 /etc/mtab 文件中所有已挂载的文件系统
- -l: lazy 卸载，即等待文件系统不再被使用时再卸载
- -f: 强制卸载，即使文件系统仍在被使用
- -v: 显示详细信息

umount 命令示例：

按设备卸载

```
umount /dev/sda1
```

按挂载点卸载

```
umount /mnt
```

卸载所有已挂载的文件系统

```
umount -a
```

lazy 卸载

```
umount -l /mnt
```

强制卸载

```
umount -f /mnt
```

3.2.10.3 /etc/fstab 文件

‘/etc/fstab’ 文件是 Linux 系统中用于配置文件系统挂载信息的配置文件，系统启动时会根据该文件中的配置自动挂载文件系统。

通用唯一识别码 (UUID): 通用唯一识别码 (Universally Unique Identifier, UUID) 为系统中的存储设备提供唯一的标识字符串, 不管这个设备是什么类型的。使用 UUID 挂载设备有以下优势:

- 避免因设备名称变化导致的挂载失败: 自动分配的设备名称 (如 `/dev/sda1`) 并非总是一致的, 它们依赖于启动时内核加载模块的顺序
- 提高系统稳定性: 如果在系统启动时使用盘符挂载, 则可能因找不到设备而加载失败, 而使用 UUID 挂载则不会有这样的问题
- 支持热插拔设备: 对于 USB 等可移动设备, 使用 UUID 可以确保同一块设备始终挂载在同一个目录下

使用 blkid 命令查看设备的 UUID: blkid 命令用于查看块设备的属性, 包括 UUID、文件系统类型等信息。

查看所有块设备的 UUID

```
blkid
```

查看指定设备的 UUID

```
blkid /dev/sda1
```

只显示 UUID

```
blkid -s UUID /dev/sda1
```

以键值对格式显示

```
blkid -o value -s UUID /dev/sda1
```

/etc/fstab 文件的格式:

<设备> <挂载点> <文件系统类型> <挂载选项> <dump> <pass>

各字段的含义:

- <设备>: 要挂载的设备, 可以是设备文件名 (如 `/dev/sda1`)、UUID (如 `UUID=5f2a2b1c-3d4e-5f6g-7h8i-9j0k1l2m3n4o`) 或标签 (如 `LABEL=root`)
- <挂载点>: 文件系统的挂载点目录
- <文件系统类型>: 文件系统的类型, 如 `ext4`、`xfs`、`swap` 等
- <挂载选项>: 挂载选项, 多个选项用逗号分隔

- **<dump>**: dump 命令是否备份该文件系统（0 表示不备份，1 表示备份）
- **<pass>**: fsck 命令检查文件系统的顺序（0 表示不检查，1 表示首先检查，2 表示在检查完所有 1 级文件系统后检查）

/etc/fstab 文件示例：

```
# /etc/fstab
# <设备>          <挂载点>          <文件系统类型>    <挂载选项>
UUID=5f2a2b1c-3d4e-5f6g-7h8i-9j0k1l2m3n4o /                ext4    defaults
UUID=6g7h8i9j-0k1l-2m3n-4o5p-6q7r8s9t0u1v /boot            ext4    defaults
UUID=7h8i9j0k-1l2m-3n4o-5p6q-7r8s9t0u1v2w swap             swap    defaults
/dev/cdrom         /media/cdrom      auto      noauto,user,ro    0 0
/dev/fd0           /media/floppy     auto      noauto,user,rw     0 0
//server/share     /mnt/smb          cifs      username=user,password=pass 0 0
```

注意事项：

- 挂载点必须是一个存在的目录
- 挂载点最好是一个空目录，否则目录中原来的文件将被系统隐藏
- 不要在文件系统正在被使用时卸载它，否则可能会导致数据丢失
- 修改 /etc/fstab 文件时要小心，错误的配置可能会导致系统无法正常启动
- 对于 USB 设备等可移动设备，建议使用 UUID 来标识设备，而不是设备文件名，因为设备文件名可能会变化

2.11 独立硬盘冗余阵列（RAID）

独立硬盘冗余阵列（Redundant Array of Inexpensive Disks, RAID）用于将多个小型硬盘驱动器合并成一个硬盘阵列，以提高存储性能和容错功能。RAID 技术通过数据条带化、镜像或奇偶校验等方式，实现了数据的冗余存储和性能的提升。

RAID 的基本类型： RAID 有多种级别，从概念提出到现在已经发展了 6 个级别，分别是 0、1、2、3、4、5。常用的是 0、1、3、5 这 4 个。

- **RAID 0（条带化）：** 将多个硬盘合并成一个大的硬盘，不具有冗余，并行 I/O，速度最快。RAID 0 也称为带区集。在存放数据时，RAID 0 将数据按硬盘的数量进行分段，然后同时将这些数据写进这些盘中。在所有级别中，RAID 0 的速度是最快的。但是 RAID 0 没有冗余功能，如果一个硬盘（物理）损坏，则所有的数据都无法使用。

- **RAID 1 (镜像):** 把硬盘阵列中的硬盘分成相同的两组, 互为镜像, 当任意硬盘介质出现故障时, 可以利用其镜像上的数据恢复, 从而提高系统的容错能力。对数据的操作仍采用分块后并行传输方式。RAID 1 不仅提高了读写速度, 还加强了系统的可靠性, 其缺点是硬盘的利用率低, 只有 50%。
- **RAID 3:** RAID 3 存放数据的原理和 RAID 0、RAID 1 不同, RAID 3 用一个硬盘来存放数据的奇偶校验位, 数据则分段存储于其余硬盘中。它像 RAID 0 一样, 以并行的方式来存放数据, 但速度没有 RAID 0 快。如果数据盘 (物理) 损坏, 则只要将坏的硬盘换掉, RAID 控制系统会根据校验盘的数据校验位在新盘中重建坏盘上的数据。不过, 如果校验盘 (物理) 损坏, 则全部数据都无法使用。利用单独的校验盘来保护数据虽然没有镜像的安全性高, 但是硬盘利用率得到了很大的提高, 为 $n-1$ 。其中 n 为使用 RAID 3 的硬盘总数量。
- **RAID 5 (分布式奇偶校验):** 向阵列中的硬盘写数据, 奇偶校验数据存放在阵列中的各个盘上, 允许单个硬盘出错。RAID 5 也是以数据的校验位来保证数据的安全, 但它不是以单独硬盘来存放数据的校验位, 而是将数据段的校验位交互存放于各个硬盘上。这样任何一个硬盘损坏, 都可以根据其他硬盘上的校验位来重建损坏的数据。硬盘的利用率为 $n-1$ 。
- **RAID 6 (双重奇偶校验):** 至少需要 4 个硬盘, 提供双重奇偶校验, 可同时承受两个硬盘故障
- **RAID 10 (RAID 1+0):** 结合了 RAID 1 和 RAID 0 的特点, 先镜像再条带化, 提供冗余和性能

软 RAID 与硬 RAID:

- **软 RAID:** 通过软件实现多块硬盘冗余, 不需要专门的硬件设备
- **硬 RAID:** 通过专门的 RAID 卡来实现 RAID 功能

软 RAID 的优缺点:

- **优点:**
 - 配置简单, 管理灵活
 - 不需要额外的硬件投资
 - 对于中小企业来说是一种经济实惠的选择
 - 可以利用现有的服务器硬件
- **缺点:**

- 占用系统 CPU 资源
- 性能相对硬 RAID 较低
- 依赖于操作系统，系统崩溃可能影响 RAID 功能

硬 RAID 的优缺点：

• 优点：

- 性能较高，有专门的处理器处理 RAID 操作
- 独立于操作系统，系统崩溃不影响 RAID 功能
- 提供更多高级功能，如电池备份、热插拔等
- 管理更加专业和集中

• 缺点：

- 成本较高，需要购买专门的 RAID 卡
- 配置相对复杂
- 升级和维护成本高
- 不同厂商的 RAID 卡兼容性可能存在问题

Linux 中的软 RAID 配置： RHEL 提供了对软 RAID 技术的支持。在 Linux 操作系统中建立软 RAID 可以使用 mdadm 工具，方便建立和管理 RAID 设备。

mdadm 命令选项说明： mdadm 命令中凡是以“-”引出的选项，均与“-”加单词首字母的方式等价。例如，“-remove”等价于“-r”，“-add”等价于“-a”。

安装 mdadm 工具：

在 Debian/Ubuntu 系统中

```
apt-get install mdadm
```

在 RHEL/CentOS 系统中

```
dnf install mdadm
```

创建 RAID 设备：

创建 RAID 1 设备（两个硬盘）

```
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb1 /dev/sdc1
```

创建 RAID 5 设备（三个硬盘）

```
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1
```

创建 RAID 10 设备（四个硬盘）

```
mdadm --create /dev/md0 --level=10 --raid-devices=4 /dev/sdb1 /dev/sdc1 /dev/sdd1 /dev/sde1
```

查看 RAID 状态：

查看 RAID 设备状态

```
mdadm --detail /dev/md0
```

查看所有 RAID 设备

```
cat /proc/mdstat
```

管理 RAID 设备：

添加新硬盘到 RAID 设备（作为热备用）

```
mdadm --add /dev/md0 /dev/sde1
```

移除故障硬盘

```
mdadm --fail /dev/md0 /dev/sdb1
```

```
mdadm --remove /dev/md0 /dev/sdb1
```

停止 RAID 设备（需要先卸载）

1. 先卸载 RAID 设备

```
umount /dev/md0
```

2. 再停止 RAID 设备

```
mdadm --stop /dev/md0
```

或使用短选项

```
mdadm -S /dev/md0
```

清除 RAID 设备的超级块信息（完全移除RAID配置）

```
mdadm --misc --zero-superblock /dev/sdb1 /dev/sdc1
```

配置 RAID 自动启动： 在 ‘/etc/mdadm/mdadm.conf’ 文件中添加 RAID 配置，确保系统启动时自动组装 RAID 设备：

生成 mdadm.conf 文件

```
mdadm --detail --scan > /etc/mdadm/mdadm.conf
```

在 RAID 设备上创建文件系统：

```
# 在 RAID 设备上创建 ext4 文件系统
mkfs.ext4 /dev/md0

# 挂载 RAID 设备
mkdir /mnt/raid
mount /dev/md0 /mnt/raid

# 将 RAID 设备添加到 /etc/fstab 中，实现自动挂载
# 在 /etc/fstab 文件中添加以下行
/dev/md0 /mnt/raid ext4 defaults 0 2
```

RAID 级别选择建议：

- 追求性能：RAID 0
- 追求数据安全：RAID 1
- 平衡性能和安全：RAID 5
- 高安全性和性能：RAID 6 或 RAID 10
- 存储空间有限：RAID 1（存储空间利用率 50%
- 预算有限：软 RAID
- 企业级应用：硬 RAID

2.12 逻辑卷管理器（LVM）

逻辑卷管理器（Logical Volume Manager, LVM）允许用户对硬盘资源进行动态调整。LVM 是 Linux 操作系统对硬盘分区进行管理的一种机制，其创建初衷是解决硬盘设备在创建分区后不易修改分区大小的问题。

LVM 的核心理念： 尽管对传统的硬盘分区进行强制扩容或缩容从理论上来讲是可行的，但是可能造成数据丢失。LVM 技术是在硬盘分区和文件系统之间添加了一个逻辑层，它提供了一个抽象的卷组，可以把多块硬盘进行卷组合并。这样一来，用户无须关心物理硬盘设备的底层架构和布局，就可以实现对硬盘分区的动态调整。

LVM 的核心概念：

- **物理卷 (Physical Volume, PV)**: 处于 LVM 的最底层, 可以将其理解为物理硬盘、硬盘分区或者 RAID 硬盘阵列, 是 LVM 的底层存储设备
- **卷组 (Volume Group, VG)**: 建立在物理卷之上, 一个卷组可以包含多个物理卷, 而且在卷组创建之后, 也可以继续向其中添加新的物理卷, 类似于一个存储池
- **逻辑卷 (Logical Volume, LV)**: 用卷组中空闲的资源建立的, 并且逻辑卷在建立后, 可以动态扩展或缩小空间, 类似于传统的分区
- **物理扩展 (Physical Extent, PE)**: 物理卷被分割成的固定大小的块, 是 LVM 中最小的存储单元
- **逻辑扩展 (Logical Extent, LE)**: 逻辑卷被分割成的块, 与物理扩展一一对应

LVM 在生产环境中的应用场景： 在生产环境中, 无法精确地预估每个硬盘分区在日后的使用情况, 因此会导致原先分配的硬盘分区不够用。例如：

- 随着业务量的增加, 用于存放交易记录的数据库目录的体积也随之增加
- 分析并记录用户的行为使日志目录的体积不断变大
- 对较大的硬盘分区进行精简、缩容的情况

可以通过部署 LVM 来解决上述问题, 实现存储资源的动态调整。

LVM 的优缺点：

- **优点：**
 - 可以动态调整逻辑卷大小, 无需重新分区
 - 可以将多个物理硬盘合并为一个卷组, 实现存储空间的统一管理
 - 可以创建快照 (snapshot), 用于备份或测试
 - 可以实现在线迁移数据, 无需停机
 - 提供了更好的存储资源管理灵活性
- **缺点：**
 - 增加了系统复杂性
 - 可能会略微降低存储性能
 - 配置不当可能会导致数据丢失
 - 恢复数据的过程比传统分区复杂

Linux 中的 LVM 配置： 在 Linux 系统中，可以使用以下命令来配置和管理 LVM。

安装 LVM 工具：

在 Debian/Ubuntu 系统中

```
apt-get install lvm2
```

在 RHEL/CentOS 系统中

```
dnf install lvm2
```

创建 LVM 的基本步骤：

1. 创建物理卷

```
pvccreate /dev/sdb1 /dev/sdc1
```

2. 创建卷组

```
vgcreate vg_data /dev/sdb1 /dev/sdc1
```

3. 创建逻辑卷

```
lvcreate -L 100G -n lv_data vg_data
```

4. 格式化逻辑卷

```
mkfs.ext4 /dev/vg_data/lv_data
```

5. 挂载逻辑卷

```
mkdir /mnt/data
```

```
mount /dev/vg_data/lv_data /mnt/data
```

6. 将逻辑卷添加到 /etc/fstab 中，实现自动挂载

在 /etc/fstab 文件中添加以下行

```
/dev/vg_data/lv_data /mnt/data ext4 defaults 0 2
```

管理 LVM：

查看物理卷

```
pvcdisplay
```

```
pvs
```

查看卷组

```
vgdisplay
vgs

# 查看逻辑卷
lvdisplay
lvs

# 扩展逻辑卷
# 1. 先扩展逻辑卷
lvextend -L +50G /dev/vg_data/lv_data
# 2. 再扩展文件系统
resize2fs /dev/vg_data/lv_data

# 缩小逻辑卷（需要先卸载）
# 1. 先卸载逻辑卷
umount /dev/vg_data/lv_data
# 2. 检查文件系统
e2fsck -f /dev/vg_data/lv_data
# 3. 缩小文件系统
resize2fs /dev/vg_data/lv_data 100G
# 4. 再缩小逻辑卷
lvreduce -L 100G /dev/vg_data/lv_data
# 5. 重新挂载逻辑卷
mount /dev/vg_data/lv_data /mnt/data

# 扩展卷组（添加新的物理卷）
pvcreate /dev/sdd1
vgextend vg_data /dev/sdd1

# 移除物理卷（需要先确保其上的数据已迁移）
pvmove /dev/sdb1
vgreduce vg_data /dev/sdb1
pvremove /dev/sdb1

# 创建快照
lvcreate -s -L 10G -n lv_snapshot /dev/vg_data/lv_data
```

```
# 恢复快照
umount /dev/vg_data/lv_data
lvconvert --merge /dev/vg_data/lv_snapshot
mount /dev/vg_data/lv_data /mnt/data
```

LVM 常用命令汇总：

- 物理卷管理：pvcreate、pvdisplay、pvs、pvremove、pvmove
- 卷组管理：vgcreate、vgdisplay、vgs、vgremove、vgextend、vgreduce
- 逻辑卷管理：lvcreate、lvdisplay、lvs、lvremove、lvextend、lvreduce、lvresize
- 文件系统管理：mkfs、mount、umount、resize2fs、e2fsck

LVM 使用建议：

- 对重要数据进行备份，尤其是在进行缩容操作时
- 合理规划卷组和逻辑卷的大小
- 考虑使用 LVM 快照进行备份和测试
- 定期检查 LVM 状态，确保持续存储系统的健康
- 对于生产环境，建议使用 RAID 与 LVM 结合的方式，既提高性能和可靠性，又保持灵活性

2.13 硬盘配额 (Disk Quota)

Linux 是一个多用户的操作系统，为了防止某个用户或组群占用过多的硬盘空间，可以通过硬盘配额 (Disk Quota) 功能限制用户和组群对硬盘空间的使用。在 Linux 操作系统中，可以通过索引结点数和硬盘块区数来限制用户和组群对硬盘空间的使用。

硬盘配额的限制方式：

- 限制用户和组的索引结点数：限制用户和组可以创建的文件数量
- 限制用户和组的硬盘块区数：限制用户和组可以使用的硬盘容量

硬盘配额的类型：

- **软限制 (Soft Limit)：** 当用户使用的空间达到软限制时，系统会发出警告，但仍允许用户继续使用空间
- **硬限制 (Hard Limit)：** 当用户使用的空间达到硬限制时，系统会拒绝用户继续使用空间
- **宽限期 (Grace Period)：** 当用户使用的空间超过软限制但未达到硬限制时，系统给予的宽限时间

Linux 中的硬盘配额配置： 在 Linux 系统中，要使用 Quota（配额）必须有文件系统的支持。假设已经使用了预设支持 Quota 的核心，那么接下来要启动文件系统的支持。

安装配额工具：

在 Debian/Ubuntu 系统中

```
apt-get install quota
```

在 RHEL/CentOS 系统中

```
dnf install quota
```

启用文件系统配额支持： 需要在 `/etc/fstab` 文件中为需要启用配额的文件系统添加配额选项。注意，不建议在根目录下设定 quota。此外 VFAT 文件系统并不支持 Linux Quota 功能。如果是 ext3/ext4/xfs，则支持 Quota。

注意： xfs 文件系统的配额设置不同于 ext4 文件系统的配额设置。

1. ext3/ext4 文件系统：

编辑 `/etc/fstab` 文件，为数据分区添加配额支持

```
/dev/sdb1 /data ext4 defaults,usrquota,grpquota 0 1
```

2. xfs 文件系统：

编辑 `/etc/fstab` 文件，为数据分区添加配额支持

```
/dev/sdc1 /data xfs defaults,usrquota,grpquota 0 1
```

重新挂载文件系统以启用配额：

```
mount -o remount /data
```


创建配额数据库文件：

1. ext3/ext4 文件系统：

```
# 为文件系统创建用户配额数据库文件
quotacheck -cug /data
```

2. xfs 文件系统：

```
# xfs 文件系统不需要创建配额数据库文件
# 只需确保挂载时启用了配额选项
```

编辑用户配额：

1. ext3/ext4 文件系统：

```
# 使用 edquota 命令编辑用户配额
edquota -u username

# 编辑组配额
edquota -g groupname

# 复制配额设置
edquota -p username user1 user2
```

2. xfs 文件系统：

```
# 使用 xfs_quota 命令编辑用户配额
xfs_quota -x -c "limit -u bsoft=10G bhard=15G isoft=1000 ihard=1500 username" /d

# 编辑组配额
xfs_quota -x -c "limit -g bsoft=50G bhard=60G isoft=5000 ihard=6000 groupname" /d
```

查看配额使用情况：

1. ext3/ext4 文件系统：

```
# 查看用户配额使用情况
quota -u username

# 查看组配额使用情况
quota -g groupname

# 查看所有用户的配额使用情况
repquota /data
```

2. xfs 文件系统：

```
# 查看用户配额使用情况
xfs_quota -x -c "quota -u username" /data

# 查看组配额使用情况
xfs_quota -x -c "quota -g groupname" /data

# 查看所有用户的配额使用情况
xfs_quota -x -c "report -u" /data

# 查看所有组的配额使用情况
xfs_quota -x -c "report -g" /data
```

启用和禁用配额：

1. ext3/ext4 文件系统：

```
# 启用配额
aquotaon /data

# 禁用配额
aquotaoff /data
```

2. xfs 文件系统：

```
# xfs 文件系统在挂载时启用配额
# 如需禁用配额，需要重新挂载文件系统
mount -o remount,usrquota=off,grpquota=off /data
```

设置默认配额：

```
# 编辑默认配额模板
edquota -t
```

硬盘配额的使用建议：

- 只在需要限制磁盘使用的文件系统上启用配额
- 合理设置软限制和硬限制，给用户足够的警告时间
- 定期检查配额使用情况，及时调整不合理的配额设置
- 对重要用户和系统用户设置适当的配额，避免影响系统运行
- 结合 LVM 使用，可以更灵活地管理存储空间和配额

2.14 文件权限和属性的记录

日常在硬盘中需要保存的数据实在太多了，因此 Linux 操作系统中有一个名为 super block 的“硬盘地图”。Linux 并不是把文件内容直接写入这个“硬盘地图”中，而是在里面记录整个文件系统的信息。因为如果把所有的文件内容都写入其中，它的体积将变得非常大，而且文件内容的查询与写入速度会变得很慢。

Linux 只是把每个文件的权限与属性记录在索引节点（inode）中，而且每个文件占用一个独立的 inode 表格。该表格的大小默认为 128B，里面记录着如下信息：

- 该文件的访问权限（read、write、execute）
- 该文件的所有者与所属组（owner、group）
- 该文件的大小（size）
- 该文件的创建或内容修改时间（ctime）
- 该文件的最后一次访问时间（atime）
- 该文件的修改时间（mtime）

- 该文件的特殊权限（SUID、SGID、SBIT）
- 该文件的真实数据地址（point）

2.15 文件实际内容的记录

文件的实际内容则保存在 block 中（block 的大小可以是 1KB、2KB 或 4KB），一个 inode 的默认大小仅为 128B（Ext3），记录一个 block 则消耗 4B。当文件的 inode 被写满后，Linux 操作系统会自动分配出一个 block，专门用于像 inode 那样记录其他 block 的信息，这样把各个 block 的内容串到一起，就能够让用户读到完整的文件内容了。

对于存储文件内容的 block，有下面两种常见情况（以 4KB 大小的 block 为例进行说明）：

- **情况 1：**文件很小（如 1KB），但依然会占用一个 block，因此会潜在地浪费 3KB。
- **情况 2：**文件较大（如 5KB），那么会占用两个 block（剩下的 1KB 也要占用一个 block）

2.16 虚拟文件系统（VFS）

计算机系统在发展过程中产生了众多的文件系统，为了使用户在读取或写入文件时不用关心底层的硬盘结构，Linux 内核中的软件层为用户程序提供了一个虚拟文件系统（Virtual File System，VFS）接口，这样用户在操作文件时，实际上是统一对这个虚拟文件系统进行操作。

实际文件系统在 VFS 下隐藏了自己的特性和细节，这样用户在日常使用时会觉得“文件系统都是一样的”，也就可以随意使用各种命令在任何文件系统中进行各种操作了（如使用 cp 命令来复制文件）。

2.17 Linux 文件系统的特点

在 Linux 操作系统中，目录、字符设备、块设备、套接字、打印机等都被抽象成了文件：在 Linux 操作系统中，一切都是文件。既然平时和我们“打交道”的都是文件，那么又应该如何找到它们呢？

Linux 操作系统中的一切都是文件，硬件设备也不例外。既然是文件，就必须有文件名称。系统内核中的 udev 设备管理器会自动规范硬件名称，目的是让用户通过设备文件的名称可以猜出设备大致的属性以及分区信息等。这对于陌生的设备来说特别方便。另外，udev 设备管理器的服务会一直以守护进程的形式运行并侦听内核发出的信号来管理/dev 目录下的设备文件。

2.18 常见的硬件设备及其文件名称

在 Linux 系统中，常见的硬件设备及其对应的文件名称如下：

1. 硬盘设备

- IDE 硬盘：/dev/hd[a-z]（如 /dev/hda、/dev/hdb）
- SATA/SCSI 硬盘：/dev/sd[a-z]（如 /dev/sda、/dev/sdb）
- NVMe 硬盘：/dev/nvme[0-9]n[0-9]+（如 /dev/nvme0n1）

一台主机上可以有多块硬盘，因此系统采用 a~p 来代表 16 块不同的硬盘（默认从 a 开始分配）。

2. 分区设备

- MBR 分区：/dev/sd[a-z][1-4]（主分区或扩展分区，编号从 1 开始，到 4 结束）、/dev/sd[a-z][5-]（逻辑分区，从编号 5 开始）
- GPT 分区：/dev/sd[a-z]p[1-]（如 /dev/sda1、/dev/sda2）
- NVMe 分区：/dev/nvme[0-9]n[0-9]+p[1-]+（如 /dev/nvme0n1p1）

3. 光盘设备

- IDE 光驱：/dev/hd[c-z]（如 /dev/hdc）
- SATA/SCSI 光驱：/dev/sr[0-9] 或 /dev/cdrom（软链接）

4. USB 设备

- USB 存储设备：/dev/sd[a-z]（与 SATA 硬盘相同）
- USB 摄像头：/dev/video[0-9]（如 /dev/video0）
- USB 输入设备：/dev/input/event[0-9] 或特定名称（如 /dev/input/mouse0）

5. 网络设备

- 以太网接口：eth[0-9] 或 en[ox][0-9]（如 eth0、enp0s3）
- 无线网卡接口：wlan[0-9] 或 wlp[0-9]s[0-9]+（如 wlan0、wlp2s0）
- 回环接口：lo

6. 输入设备

- 键盘：/dev/input/event[0-9] 或 /dev/input/keyboard
- 鼠标：/dev/input/event[0-9] 或 /dev/input/mouse[0-9]

- 触摸屏: `/dev/input/event[0-9]` 或 `/dev/input/touchscreen`

7. 输出设备

- 显卡: `/dev/fb[0-9]` (帧缓冲区)、`/dev/dri/card[0-9]` (DRI 设备)
- 音频设备: `/dev/snd/*` (如 `/dev/snd/controlC0`、`/dev/snd/pcmC0D0p`)
- 打印机: `/dev/lp[0-9]` 或 `/dev/usb/lp[0-9]` (USB 打印机)

8. 其他设备

- 随机数生成器: `/dev/random`、`/dev/urandom`
- 零设备: `/dev/zero`
- 空设备: `/dev/null`
- 控制台: `/dev/console`
- 终端: `/dev/tty[0-9]` (虚拟终端)、`/dev/pts/[0-9]` (伪终端)

在 Windows 操作系统中, 想要找到一个文件, 我们要依次进入该文件所在的磁盘分区 (假设这里是 D 盘), 然后进入该分区下的具体目录, 最终找到这个文件。但是在 Linux 操作系统中并不存在 C/D/E/F 等盘, Linux 操作系统中的一切文件都是从根目录 (/) 开始的, 并按照文件系统层次化标准 (Filesystem Hierarchy Standard, FHS) 采用树形结构来存放文件, 以及定义常见目录的用途。

另外, Linux 操作系统中的文件和目录名称是严格区分大小写的。例如, `root`、`rOOt`、`Root`、`rooT` 均代表不同的目录, 并且文件名称中不得包含 “/”

2.19 Linux 常见目录结构

Linux 操作系统中常见的目录名称以及相应的存放内容如下:

目录名称	存放内容
/	根目录，所有文件和目录的起点
/bin	存放系统命令，普通用户和 root 都可以执行
/sbin	存放系统管理命令，只有 root 可以执行
/etc	存放系统配置文件
/home	普通用户的家目录，每个用户有一个以用户名命名的子目录
/root	root 用户的家目录
/var	存放可变数据，如日志文件、邮件、缓存等
/tmp	临时文件目录，系统重启后会清空
/usr	存放应用程序和系统资源
/usr/bin	存放用户命令
/usr/sbin	存放系统管理命令
/usr/local	存放本地安装的软件
/usr/share	存放共享数据，如文档、图标、字体等
/boot	存放启动相关文件，如内核、引导加载程序等
/dev	存放设备文件
/proc	虚拟文件系统，存放系统运行状态信息
/sys	虚拟文件系统，存放硬件设备信息
/lib	存放系统库文件
/lib64	存放 64 位系统库文件
/opt	存放可选的应用程序
/media	挂载可移动媒体设备的目录
/mnt	临时挂载文件系统的目录
/srv	存放服务相关的数据，如网站、FTP 服务的数据等
/lost+found	存放文件系统修复时恢复的文件

2.20 绝对路径与相对路径

在 Linux 操作系统中，文件路径的表示方法有两种：绝对路径和相对路径。

定义 3.2 (绝对路径). 绝对路径是由根目录 (/) 开始写起的文件名或目录名称，例如：
/home/dmtsai/basher。

定义 3.3 (相对路径). 相对路径是相对于目前路径的文件名写法，例如：./home/dmtsai
或 ../../home/dmtsai/ 等。

绝对路径的特点：

- 以根目录 (/) 开头

- 路径完整，不依赖于当前工作目录
- 无论当前在哪个目录，使用绝对路径都能准确定位到文件或目录
- 通常较长，书写起来相对繁琐

相对路径的特点：

- 不以根目录（/）开头
- 路径简短，依赖于当前工作目录
- 使用 `.` 表示当前目录
- 使用 `..` 表示上一级目录
- 当需要在相邻目录间切换时，使用相对路径更加方便

”`.`”和”`..`”特殊目录： 在 Linux 操作系统中，每个目录下都有两个特殊的目录：

- `.`：表示当前目录，是当前路径的相对引用
- `..`：表示上一级目录，是父目录的相对引用

这两个特殊目录在使用相对路径时非常重要：

- 当需要引用当前目录下的文件或子目录时，可以使用 `.` 作为前缀
- 当需要向上级目录导航时，可以使用 `..` 来表示
- 多个 `..` 可以组合使用，例如 `../..` 表示向上两级目录

例 3.1. 假设当前工作目录是 `/home/user`：

- 绝对路径：`/home/user/Documents/file.txt`（完整路径）
- 相对路径：`./Documents/file.txt`（相对于当前目录）
- 相对路径：`../otheruser/Documents/file.txt`（相对于上一级目录）
- Linux 文件系统层次结构（FHS）
- 根目录（/）下主要子目录的功能
- 绝对路径与相对路径
- 文件与目录的基本概念

2.21 文件与目录的基本概念

文件是操作系统用来存储信息的基本结构，是一组信息的集合。文件可通过文件名来唯一地标识。

Linux 文件命名规则：

- 文件名称最长可允许 255 个字符
- 可用字符：A～Z、0～9、.、_、-等符号
- 没有“扩展名”的概念，文件名称和文件种类没有直接关联
- 文件名区分大小写，例如 sample.txt、Sample.txt、SAMPLE.txt 代表不同文件
- 以“.”开头的文件为隐藏文件，需要使用“ls -a”命令才能显示

文件访问权限： Linux 中的每一个文件或目录都包含访问权限，访问权限决定了谁能访问以及如何访问文件和目录。

访问权限的三种类型：

- **所有者权限：** 只允许用户自己访问
- **用户组权限：** 允许一个预先指定的用户组中的用户访问
- **其他用户权限：** 允许系统中的任何用户访问

访问权限的三种程度：

- **读权限：** 允许查看文件内容
- **写权限：** 允许修改文件内容
- **执行权限：** 允许执行文件（如程序）

权限的默认设置： 当创建一个文件时，系统会自动赋予文件所有者读和写的权限，这样可以允许所有者显示文件内容和修改文件。文件所有者可以将这些权限改变为任何想指定的权限。

文件预设权限： `umask` `umask` (user file-creation mode mask) 命令用于设置用户创建文件或目录时的默认权限掩码。它决定了新创建文件和目录的默认权限。

umask 的工作原理： 目录与文件的默认权限是不一样的。我们知道，x 权限对于目录是非常重要的，但是一般文件不应该有执行的权限。因为一般文件通常是用于数据的记录，当然不需要执行的权限。因此，预设的情况如下：

- 若用户建立文件，则预设没有 x 权限，即只有 r、w 这两个权限，也就是最大为 666，预设权限为 -rw-rw-rw-。
- 若用户建立目录，则由于 x 与是否可以进入此目录有关，因此默认所有权限均开放，即 777，预设权限为 drwxrwxrwx。

umask 值指的是该默认值需要减掉的权限（r、w、x 分别对应 4、2、1）。当创建新文件或目录时，系统会先计算默认权限，然后减去 umask 值得到最终的权限：

- 对于文件，默认权限是 666（rw-rw-rw-）
- 对于目录，默认权限是 777（rwxrwxrwx）
- 最终权限 = 默认权限 - umask 值

查看当前 umask 值：

```
# 以数字形式查看umask
umask
```

```
# 以符号形式查看umask
umask -S
```

常见的 umask 值及其含义：

- 0000：无权限限制（不推荐）
- 0002：默认值，允许同组用户写入
- 0022：标准值，禁止同组和其他用户写入
- 0222：禁止所有写入权限
- 0777：禁止所有权限（最严格）

root 的 umask 值默认是 022，这是基于安全的考虑。对于一般用户，通常 umask 值为 002，即保留同组的写入权限。

修改 umask 值:

临时修改umask (仅对当前shell有效)

```
umask 022
```

永久修改umask

在 ~/.bashrc 或 ~/.profile 文件中添加

```
umask 022
```

umask 对权限的影响示例:

例 3.2. 假设 *umask* 值为 022:

- 新文件权限: $666 - 022 = 644$ (*rw-r-r-*)
- 新目录权限: $777 - 022 = 755$ (*rw-r-xr-x*)

假设 *umask* 值为 002:

- 新文件权限: $666 - 002 = 664$ (*rw-rw-r-*)
- 新目录权限: $777 - 002 = 775$ (*rw-rw-r-x*)

umask 的应用场景: umask 与新建文件及目录的默认权限有很大关系。这个属性可以用在服务器上, 尤其是文件服务器 (file server) 上。例如, 在创建 Samba 服务器或者 FTP 服务器时, 显得尤为重要。

- 在多用户环境中, 通过设置合适的 umask 值保护敏感文件
- 在安全要求较高的系统中, 设置更严格的 umask 值
- 在开发环境中, 设置更宽松的 umask 值方便团队协作

注意事项:

- umask 值是从默认权限中减去的, 而不是直接与默认权限进行按位与操作
- umask 值的每一位代表要禁止的权限
- 对于文件, 执行权限不会被默认设置, 需要手动添加
- umask 设置会影响当前 shell 及其子进程, 不会影响其他 shell

权限的表示方式： 每一个文件或目录都有 9 个字符的权限组，分别对应 3 种用户类型（所有者、用户组、其他用户）的读、写、执行权限。

例 3.3. 权限表示示例： `rwxr-xr--`

- 前 3 位 (*rw**x*): 文件所有者的权限（读、写、执行）
- 中间 3 位 (*r-**x*): 用户组成员的权限（读、执行，无写权限）
- 后 3 位 (*r-*): 其他用户的权限（只读，无写和执行权限）

文件属性的含义： 在 Linux 操作系统中，每个文件和目录都有一系列的属性，这些属性决定了文件的类型、访问权限、所有者等信息。

文件属性的组成： 使用 `ls -l` 命令查看文件时，显示的第一列就是文件的属性信息，通常由 10 个字符组成：

例 3.4. 属性示例： `-rwxr-xr--`

- 第 1 位：文件类型
- 第 2-4 位：文件所有者的权限
- 第 5-7 位：文件所属组的权限
- 第 8-10 位：其他用户的权限

文件类型： 第 1 位字符表示文件的类型：

- `-`: 普通文件
- `d`: 目录
- `l`: 符号链接
- `b`: 块设备文件
- `c`: 字符设备文件
- `s`: 套接字文件
- `p`: 管道文件

特殊权限： 在权限位中，除了基本的读、写、执行权限外，还有三个特殊权限：

- **SUID (Set User ID)：** 当执行文件时，以文件所有者的身份运行
- **SGID (Set Group ID)：** 当执行文件时，以文件所属组的身份运行；对于目录，新创建的文件会继承该目录的组
- **SBIT (Sticky Bit)：** 对于目录，只有文件所有者和 root 可以删除文件

文件属性的其他信息： 使用 `ls -l` 命令还可以看到以下文件属性：

- 硬链接数
- 文件所有者
- 文件所属组
- 文件大小
- 文件的最后修改时间
- 文件名

文件属性的查看方法：

查看文件属性

```
ls -l filename
```

查看详细属性（包括inode信息）

```
ls -li filename
```

查看目录属性

```
ls -ld directory
```

第 3 节 常用文件与目录操作命令

- 浏览目录：ls, cd, pwd
- 创建与删除：mkdir, touch, rm, rmdir
- 复制与移动：cp, mv
- 查看文件内容：cat, more, less, head, tail
- 文件属性：chmod, chown, chgrp
- 文件搜索：find, grep

3.1 pwd 命令

pwd (print working directory) 命令用于显示当前工作目录的绝对路径。它是一个内置命令，无需参数即可执行。

显示当前工作目录的绝对路径

pwd

示例输出：

/home/user/Documents

pwd 命令的主要作用：

- 确认当前所在的目录位置
- 在执行复杂操作前验证工作目录
- 在脚本中获取当前目录路径
- 帮助用户了解目录结构

pwd 命令没有常用的选项，直接执行即可显示当前工作目录的完整路径。

3.2 clear 命令

clear 命令用于清除终端屏幕上的内容，使终端回到干净的状态。它是一个内置命令，常用于在执行命令前清理屏幕，提高命令输出的可读性。

清除终端屏幕

clear

使用快捷键Ctrl+L也可以清除屏幕（与clear命令功能相同）

clear 命令的功能：

- 清除终端屏幕上的所有内容
- 保持当前的工作目录和 shell 环境不变
- 光标位置重置到屏幕左上角

注意事项：

- clear 命令只是清除屏幕显示，不会删除任何实际数据
- 清除后，使用向上方向键仍然可以查看之前的命令历史
- 不同终端模拟器对 clear 命令的处理可能略有差异
- 在某些终端中，clear 命令可能只是将屏幕内容向上滚动，而不是真正清除

3.3 echo 命令

echo 命令用于在命令行终端输出字符串或变量提取后的值。它是一个内置命令，常用于输出文本、显示变量值、生成文件内容等。

echo 命令的功能：

- 输出字符串到终端
- 显示变量的值
- 生成文件内容
- 在脚本中输出调试信息
- 输出特殊字符

输出字符串

```
echo "Hello, World!"
```

输出字符串（不使用引号）

```
echo Hello, World!
```

输出变量的值

```
name="John"
```

```
echo $name
```

输出多个变量

```
name="John"
```

```
age=30
```

```
echo "Name: $name, Age: $age"
```

输出带特殊字符的字符串

```
echo "This is a \"quoted\" word"
```

输出多行文本

```
echo -e "Line 1\nLine 2\nLine 3"
```

输出带制表符的文本

```
echo -e "Name\tAge\tCity"
```

不输出换行符

```
echo -n "Enter your name: "

# 输出转义字符
echo -e "Hello\nWorld"

# 输出到文件（覆盖）
echo "Hello, World!" > output.txt

# 输出到文件（追加）
echo "Hello, World!" >> output.txt

# 输出环境变量的值
echo $HOME
echo $PATH
echo $USER

# 输出命令替换的结果
echo "Current directory: $(pwd)"
echo "Current date: $(date)"

# 输出算术运算的结果
echo $((10 + 20))
echo $((5 * 6))
```

echo 命令的常用选项：

- -n: 不输出换行符
- -e: 启用转义字符的解释
- -E: 禁用转义字符的解释（默认）

转义字符说明：使用-e 选项时，echo 命令支持以下转义字符：

- \n: 换行符
- \t: 制表符
- \r: 回车符
- \\: 反斜杠

- \": 双引号
- \': 单引号
- \b: 退格符
- \a: 响铃符
- \c: 不输出换行符（与-n 类似）
- \v: 垂直制表符
- \0NNN: 八进制 ASCII 码（NNN 为八进制数）
- \xHH: 十六进制 ASCII 码（HH 为十六进制数）

使用转义字符

```
echo -e "Line 1\nLine 2\nLine 3"
echo -e "Name:\tAge:\tCity:"
echo -e "Hello\b\b\bWorld"
echo -e "This is a \"quoted\" word"
echo -e "Path: C:\\\\Users\\\\John"
echo -e "Octal: \101\102\103"
echo -e "Hex: \x41\x42\x43"
```

变量输出: echo 命令常用于输出变量的值，包括环境变量和自定义变量：

输出环境变量

```
echo $HOME          # 输出用户主目录
echo $PATH           # 输出PATH环境变量
echo $USER           # 输出当前用户名
echo $SHELL          # 输出当前shell路径
echo $PWD            # 输出当前工作目录
```

输出自定义变量

```
name="Alice"
age=25
echo "Name: $name, Age: $age"
```

输出数组元素

```
fruits=("apple" "banana" "orange")
echo ${fruits[0]} # 输出第一个元素
echo ${fruits[@]} # 输出所有元素
```

命令替换：echo 命令可以与命令替换结合使用，输出命令的执行结果：

```
# 使用$()进行命令替换
echo "Current directory: $(pwd)"
echo "Current date: $(date)"
echo "Current user: $(whoami)"

# 使用反引号进行命令替换（不推荐）
echo "Current directory: `pwd`"
echo "Current date: `date`"
```

算术运算：echo 命令可以输出算术运算的结果：

```
# 基本算术运算
echo $((10 + 20))      # 输出30
echo $((50 - 20))      # 输出30
echo $((5 * 6))        # 输出30
echo $((20 / 4))       # 输出5
echo $((20 % 3))       # 输出2

# 复杂算术运算
echo $((10 + 20 * 2))  # 输出50
echo $(((10 + 20) * 2)) # 输出60

# 位运算
echo $((5 | 3))        # 输出7
echo $((5 & 3))         # 输出1
echo $((5 ^ 3))        # 输出6
```

文件操作：echo 命令常用于创建文件或向文件写入内容：

```
# 创建新文件（覆盖）
echo "Hello, World!" > output.txt

# 追加内容到文件
echo "Hello, World!" >> output.txt

# 创建多行文件
echo -e "Line 1\nLine 2\nLine 3" > output.txt
```

创建空文件

```
echo -n > empty.txt
```

创建配置文件

```
echo -e "[Settings]\nname=John\nage=30" > config.ini
```

使用说明： - echo 命令是 shell 的内置命令，用于输出文本或变量值 - 默认情况下，echo 命令会在输出后添加换行符 - 使用 -n 选项可以禁止输出换行符 - 使用 -e 选项可以解释转义字符 - 使用重定向操作符（> 和 >>）可以将输出写入文件

注意事项：

- 输出包含空格的字符串时，建议使用引号括起来
- 输出变量时，变量名前要加 \$ 符号
- 在脚本中，echo 命令常用于输出调试信息
- 不同 shell 的 echo 命令可能略有差异
- 输出特殊字符时，注意转义字符的使用
- 使用 -e 选项时，要确保转义字符的正确使用

echo 与 printf 的比较：

- **echo：** 简单易用，适合基本输出
- **printf：** 功能更强大，支持格式化输出
- echo 命令更简单，但 printf 命令更灵活
- printf 命令支持格式化字符串，如 %d、%s 等
- 在需要精确控制输出格式时，建议使用 printf 命令

常见用途：

- 在脚本中输出提示信息
- 显示变量的值进行调试
- 生成配置文件
- 创建日志文件
- 输出命令执行结果
- 在管道中传递数据

3.4 chmod 命令

chmod (change mode) 命令用于修改文件或目录的访问权限。它是 Linux 系统中最常用的权限管理命令之一。

权限的表示方法:

数字表示法: 使用 3 位八进制数字表示权限，每一位数字对应一种用户类型的权限:

- 第一位: 文件所有者的权限
- 第二位: 文件所属组的权限
- 第三位: 其他用户的权限

数字表示法是指将读取 (r)、写入 (w) 和执行 (x) 分别以数字 4、2、1 来表示，没有授予的部分表示为 0，然后把授予的权限相加:

- 4: 读权限 (r)
- 2: 写权限 (w)
- 1: 执行权限 (x)
- 0: 没有权限

例 3.5. 权限数字计算示例:

- $rx = 4 + 2 + 1 = 7$
- $rw = 4 + 2 + 0 = 6$
- $r-x = 4 + 0 + 1 = 5$
- $r- = 4 + 0 + 0 = 4$
- $-wx = 0 + 2 + 1 = 3$
- $-w- = 0 + 2 + 0 = 2$
- $-x = 0 + 0 + 1 = 1$
- $-- = 0 + 0 + 0 = 0$

例 3.6. 完整权限数字表示示例:

- 755: *rw~~x~~r-xr-x* (所有者可读可写可执行, 其他用户可读可执行)
- 644: *rw-r-r-* (所有者可读可写, 其他用户只读)
- 700: *rw~~x~~---* (只有所有者可读可写可执行)
- 777: *rw~~x~~rw~~x~~rw~~x~~* (所有用户都可读可写可执行)

符号表示法 (文字表示法): 使用符号表示权限的修改, 格式为: [u/g/o/a] [/=/][r/w/x]+, 其中:

1. 用户类型 (使用 4 种字符来表示不同的用户):

- u: user, 表示所有者
- g: group, 表示所属组
- o: others, 表示其他用户
- a: all, 表示以上 3 种用户

2. 权限类型 (使用下面 3 种字符的组合来设置操作权限):

- r: read, 读
- w: write, 写
- x: execute, 执行

3. 操作符 (操作符包括以下 3 种):

- +: 添加某种权限
- -: 减去某种权限
- =: 赋予给定权限并取消原来的权限

例 3.7. 符号表示法示例:

- `chmod ux file+`: 给所有者添加执行权限
- `chmod g-rw file`: 从所属组移除读和写权限
- `chmod o=rx file`: 给其他用户设置读和执行权限, 并取消原来的权限
- `chmod aw file+`: 给所有用户添加写权限
- `chmod ug=rw file`: 给所有者和所属组设置读和写权限
- `chmod u=rw,g=r,o=r file`: 给所有者设置读和写权限, 给所属组和其他用户设置只读权限

chmod 命令的基本用法:

使用数字表示法修改权限

```
chmod 755 filename
```

使用符号表示法修改权限

```
chmod u+rw filename      # 给所有者添加读、写、执行权限
```

```
chmod g+rx filename      # 给所属组添加读、执行权限
```

```
chmod o+r filename       # 给其他用户添加读权限
```

```
chmod a-w filename       # 移除所有用户的写权限
```

```
chmod u=rw,g=r,o=r filename # 设置所有者可读可写，其他用户只读
```

递归修改目录及其内容的权限

```
chmod -R 755 directory
```

添加特殊权限

```
chmod u+s filename       # 添加SUID权限
```

```
chmod g+s filename       # 添加SGID权限
```

```
chmod +t directory       # 添加SBIT权限
```

移除特殊权限

```
chmod u-s filename       # 移除SUID权限
```

```
chmod g-s filename       # 移除SGID权限
```

```
chmod -t directory       # 移除SBIT权限
```

chmod 命令的常用选项:

- -R: 递归修改目录及其所有子目录和文件的权限
- -f: 强制执行，忽略错误信息
- -v: 显示权限修改的详细信息
- --reference=RFILE: 以参考文件的权限为标准修改目标文件的权限

权限修改示例:

示例1: 将文件设置为所有者可读可写可执行，其他用户可读可执行

```
chmod 755 script.sh
```

示例2: 将文件设置为所有用户只读

```
chmod 644 document.txt
```

示例3: 递归修改目录及其内容的权限

```
chmod -R 755 project/
```

示例4: 给文件添加执行权限

```
chmod +x script.sh
```

示例5: 给所有者添加写权限

```
chmod u+w file.txt
```

示例6: 移除其他用户的执行权限

```
chmod o-x script.sh
```

示例7: 设置与参考文件相同的权限

```
chmod --reference=file1.txt file2.txt
```

特殊权限的使用: 特殊权限也可以采用数字表示法。SUID、SGID 和 SBIT 权限分别为 4、2 和 1。使用 `chmod` 命令设置文件权限时,可以在普通权限的数字前面加上一位数字来表示特殊权限。

- **SUID** (4000): 当执行文件时,以文件所有者的身份运行,常用于需要特殊权限的命令(如 `passwd`)
- **SGID** (2000): 当执行文件时,以文件所属组的身份运行;对于目录,新创建的文件会继承该目录的组
- **SBIT** (1000): 对于目录,只有文件所有者和 `root` 可以删除文件,常用于 `/tmp` 目录

例 3.8. 特殊权限的数字表示:

- 4: *SUID* 权限
- 2: *SGID* 权限
- 1: *SBIT* 权限
- 0: 没有特殊权限

例 3.9. 特殊权限设置示例:

添加SUID权限

```
chmod 4755 filename
```

添加SGID权限

```
chmod 2755 filename
```

添加SBIT权限

```
chmod 1777 directory
```

添加SUID和SGID权限

```
chmod 6755 filename
```

添加SGID和SBIT权限

```
chmod 3777 directory
```

注意事项：

- 只有文件所有者和 root 用户可以修改文件的权限
- 权限的修改需要谨慎，特别是对于系统文件
- 递归修改权限时要注意不要影响到不需要修改的文件
- 使用符号表示法时，要注意权限修改的顺序
- 特殊权限的使用需要了解其安全 implications
- 对于目录，执行权限是进入目录的必要条件
- 对于文件，执行权限是执行文件的必要条件

权限与指令间的关系： 权限对于用户来说非常重要，因为权限可以限制用户能不能读取/建立/删除/修改文件或目录。

不同的操作需要不同的权限：

- **读取文件：** 需要文件的读权限（r）
- **修改文件：** 需要文件的写权限（w）
- **执行文件：** 需要文件的执行权限（x）
- **进入目录：** 需要目录的执行权限（x）

- 列出目录内容：需要目录的读权限（r）
- 在目录中创建文件：需要目录的写权限（w）和执行权限（x）
- 删除目录中的文件：需要目录的写权限（w）和执行权限（x），以及文件的写权限（对于某些文件系统）

权限的正确设置对于系统安全至关重要，不合理的权限设置可能导致：

- 敏感文件被未授权用户访问
- 系统文件被意外修改
- 恶意程序被执行
- 数据丢失或损坏

因此，在设置文件和目录权限时，应遵循最小权限原则：只授予用户完成任务所需的最小权限。

常见错误：

- 忘记使用-R选项递归修改目录权限
- 错误的权限数字，导致权限设置不当
- 对系统文件设置过于宽松的权限，存在安全风险
- 混淆符号表示法的语法

3.5 文件系统隐藏属性（chattr 和 lsattr 命令）

在 Linux 的 ext2/ext3/ext4 文件系统下，除基本的 r、w、x 权限外，还可以设定系统隐藏属性。设置系统隐藏属性使用 chattr 命令，使用 lsattr 命令可以查看隐藏属性。

chattr 命令： chattr（change attributes）命令用于设置文件系统的隐藏属性。

基本语法

chattr [选项] [属性] 文件或目录

常用选项

- R：递归处理所有子文件和子目录
- v：设置文件版本号
- f：强制执行，忽略错误信息

常用的隐藏属性： `chattr` 这些属性共有以下 8 种：

- **a**: append only, 系统只允许在这个文件之后追加数据, 不允许任何进程覆盖或截断这个文件。如果目录具有这个属性, 则系统将只允许在这个目录下建立和修改文件, 而不允许删除任何文件。
- **b**: 不更新文件或目录的最后存取时间。
- **c**: 将文件或目录压缩后存放。
- **d**: no dump, 将文件或目录排除在操作之外。
- **i**: immutable, 不得任意改动文件或目录。
- **s**: secure deletion, 保密性地删除文件或目录, 即硬盘空间被全部收回。
- **S**: synchronous, 即时更新文件或目录。
- **u**: undeletable, 预防意外删除。
- **t**: no tail-merging, 不允许尾部合并
- **j**: journaled, 启用 ext3/ext4 的日志功能

chattr 命令示例：

设置文件为不可变

```
chattr +i file.txt
```

设置文件为只可追加

```
chattr +a file.txt
```

移除不可变属性

```
chattr -i file.txt
```

递归设置目录及其内容为不可变

```
chattr -R +i directory/
```

设置多个属性

```
chattr +ai file.txt
```

lsattr 命令： lsattr (list attributes) 命令用于查看文件系统的隐藏属性。

基本语法

lsattr [选项] 文件或目录

常用选项

-R: 递归显示所有子文件和子目录的属性

-a: 显示所有文件（包括隐藏文件）

-d: 显示目录本身的属性，而不是目录内容

-v: 显示文件版本号

lsattr 命令示例：

查看文件的隐藏属性

lsattr file.txt

递归查看目录及其内容的隐藏属性

lsattr -R directory/

查看目录本身的隐藏属性

lsattr -d directory/

查看所有文件（包括隐藏文件）的隐藏属性

lsattr -a

隐藏属性的应用场景：

- a属性：适用于日志文件，只允许追加内容，防止意外删除或修改
- i属性：适用于系统配置文件，防止意外修改
- d属性：适用于临时文件，不需要备份
- S属性：适用于重要数据，确保数据及时同步到磁盘
- u属性：适用于需要可恢复性的文件

注意事项：

- 只有 root 用户可以设置和修改文件的隐藏属性
- 隐藏属性只在 ext2/ext3/ext4 文件系统上有效

- 某些隐藏属性可能会影响文件的正常操作，使用时需谨慎
- 设置了*i*属性的文件，即使是 root 用户也无法删除或修改
- 设置了*a*属性的文件，只能追加内容，不能修改或删除

3.6 文件访问控制列表（ACL）

文件访问控制列表（Access Control List，ACL）是一种更细粒度的权限控制机制，允许对指定的用户或用户组设置文件或目录的操作权限。

ACL 的特点：

- 权限是针对某一类用户设置的
- 可以对某个指定的用户进行单独的权限控制
- 如果针对某个目录设置了 ACL，则目录中的文件会继承其 ACL
- 若针对文件设置了 ACL，则文件不再继承其所在目录的 ACL

查看 ACL： 使用`getfacl`命令查看文件或目录的 ACL：

查看文件的ACL

```
getfacl file.txt
```

查看目录的ACL

```
getfacl directory/
```

设置 ACL： 文件的 ACL 提供的是在所有者、所属组、其他用户的读/写/执行权限之外的特殊权限控制，使用 `setfacl` 命令可以针对单一用户或用户组、单一文件或目录来进行读/写/执行权限的控制。其中，针对目录文件需要使用-R 递归选项；针对普通文件可以使用-m 选项；如果想要删除某个文件的 ACL，则可以使用-b 选项。

使用`setfacl`命令设置文件或目录的 ACL：

设置文件的ACL，允许用户user1读写

```
setfacl -m u:user1:rw file.txt
```

设置文件的ACL，允许用户组group1读取

```
setfacl -m g:group1:r file.txt
```

设置目录的ACL，允许用户user1读写执行

```
setfacl -m u:user1:rwX directory/
```

递归设置目录及其内容的ACL

```
setfacl -R -m u:user1:rwX directory/
```

删除 ACL: 使用setfacl命令删除文件或目录的 ACL:

删除文件的特定用户ACL

```
setfacl -x u:user1 file.txt
```

删除文件的特定用户组ACL

```
setfacl -x g:group1 file.txt
```

删除文件的所有ACL

```
setfacl -b file.txt
```

递归删除目录及其内容的所有ACL

```
setfacl -R -b directory/
```

ACL 的应用场景:

- 需要对特定用户设置不同权限的场景
- 多用户共享文件或目录的场景
- 复杂的权限管理需求

注意事项:

- 只有 root 用户可以设置和修改文件的 ACL
- ACL 只在支持的文件系统上有效
- 某些文件系统可能需要启用 ACL 支持
- 常用的 ls 命令看不到 ACL 信息,却可以看到文件权限的最后一个点"."变成了"+",这就意味着该文件已经设置了 ACL

3.7 cd 命令

cd (change directory) 命令用来在不同的目录中进行切换。它是一个内置命令,用于改变当前工作目录。

切换到指定目录

```
cd /path/to/directory
```

切换到用户的家目录

```
cd
```

```
cd ~
```

切换到上一级目录

```
cd ..
```

切换到上一次的工作目录

```
cd -
```

cd 命令的特点:

- 用户在登录系统后，会处于用户的”家目录”（\$HOME）中
- 家目录一般以/home 开始，后接用户名（如/home/user）
- root 用户的家目录为/root
- 可以使用绝对路径或相对路径指定目标目录
- 支持特殊符号：
 - ”.” 代表当前目录
 - ”..” 代表当前目录的父目录
 - ” ” 代表用户的家目录（主目录）
 - ”-” 代表上一次的工作目录
- 不带任何参数的”cd” 命令相当于”cd ”，即将目录切换到用户的家目录

cd 命令是 Linux 中最常用的命令之一，是用户在文件系统中导航的基础工具。

3.8 ls 命令

ls (list) 命令用来列出文件或目录信息。它是一个常用的文件系统导航工具，可以显示指定目录中的文件和子目录。

列出当前目录中的文件和子目录

```
ls
```

列出详细信息（包括权限、所有者、大小、修改时间等）

```
ls -l
```

显示隐藏文件（以.开头的文件）

```
ls -a
```

以人类可读的格式显示文件大小

```
ls -h
```

按修改时间排序（最新的在前）

```
ls -t
```

递归列出子目录中的内容

```
ls -R
```

ls 命令的常用选项：

- -l: 显示详细信息
- -a: 显示所有文件（包括隐藏文件）
- -A: 显示所有文件（包括隐藏文件，但不显示. 和..）
- -h: 以人类可读的格式显示文件大小
- -t: 按修改时间排序
- -c: 按状态改变时间排序
- -R: 递归显示子目录内容
- -i: 显示 inode 号
- -S: 按文件大小排序
- -r: 反向排序
- -F: 在文件名后添加类型指示符（/表示目录，* 表示可执行文件，@ 表示符号链接）
- -C: 按列格式显示文件列表
- -d: 只显示目录本身，不显示目录中的内容

- -g: 类似-l, 但不显示所有者信息

ls 命令是 Linux 中最基本、最常用的命令之一, 通过不同的选项组合, 可以满足各种文件浏览需求。

3.9 cat 命令

cat (concatenate) 命令主要用于滚动显示文件内容, 或将多个文件合并成一个文件。它是一个常用的文件内容查看工具。

通常使用 cat 命令查看文件内容, 但是 cat 命令的输出内容不能分页显示, 要查看超过一屏的文件内容, 需要使用 more 或 less 等其他命令。如果在 cat 命令中没有指定参数, 则 cat 会从标准输入 (键盘) 中获取内容。

显示单个文件的内容

```
cat file.txt
```

显示多个文件的内容

```
cat file1.txt file2.txt
```

将多个文件合并到一个新文件

```
cat file1.txt file2.txt > combined.txt
```

将内容追加到文件末尾

```
cat file.txt >> existing.txt
```

从标准输入读取内容并显示

```
cat
```

cat 命令的常用选项:

- -n: 显示行号
- -b: 显示行号 (但不包括空白行)
- -s: 将连续的空白行压缩为一个
- -A: 显示所有字符 (包括控制字符)

cat 命令是 Linux 中最基本的文件内容查看命令之一, 适用于查看小型文件的完整内容。

3.10 more 命令

more 命令通常用于分屏显示文件内容。在使用 cat 命令时，如果文件内容太长，则用户只能看到文件的最后一部分。这时可以使用 more 命令一页一页地分屏显示文件内容。

查看文件内容（分屏显示）

```
more file.txt
```

从指定行开始查看

```
more +10 file.txt
```

分页显示文件内容，并指定每屏显示10行

```
more -10 file.txt
```

more 命令的操作键：

- Enter 键：向下移动一行
- Space 键：向下移动一页
- Q 键：退出 more 命令
- B 键：向上移动一页
- /键：搜索关键字
- n 键：查找下一个匹配项

在大部分情况下，可以不加任何选项直接执行 more 命令查看文件内容。执行 more 命令后，进入 more 状态，可以使用上述操作键进行导航。

more 命令经常在管道中被调用，以实现各种命令输出内容的分屏显示。例如：

查看目录内容并分屏显示

```
ls -la | more
```

查看系统日志并分屏显示

```
tail -n 100 /var/log/syslog | more
```

3.11 less 命令

less 命令是 more 命令的改进版，比 more 命令的功能强大。more 命令只能向下翻页，而 less 命令不但可以向下、向上翻页，还可以前后左右移动。

查看文件内容（分屏显示，支持上下滚动）

```
less file.txt
```

查看文件内容并显示行号

```
less -N file.txt
```

在管道中使用less

```
echo "Hello\nWorld\nLinux" | less
```

less 命令的操作键：

- Enter 键：向下移动一行
- Space 键：向下移动一页
- B 键：向上移动一页
- 方向键：向前、后、左、右移动
- Q 键：退出 less 命令
- /键：搜索关键字，输入要查找的单词或字符后按 Enter 键，less 会高亮显示第一个匹配项
- n 键：查找下一个匹配项
- N 键：查找上一个匹配项
- G 键：跳转到文件末尾
- g 键：跳转到文件开头

less 命令的搜索功能：less 命令还支持在一个文本文件中进行快速查找。先按”/”键，再输入要查找的单词或字符，按 Enter 键后，less 命令会在文本文件中进行快速查找，并把找到的第一个搜索目标高亮显示。如果希望继续查找，就再次按”/”键，再按”Enter”键即可。

执行 less 命令后，进入 less 状态，可以使用上述操作键进行导航。less 命令的功能比 more 命令更加丰富，是查看文件内容的常用工具。

3.12 head 命令

head 命令用于显示文件的开头部分，默认情况下只显示文件前 10 行的内容。

head 命令的常用选项：

- -n: 指定显示的行数。若 num 为负值，则表示从倒数 |num| 行后面的所有行不显示
- -c: 指定显示的字节数

查看文件的前10行（默认）

```
head file.txt
```

查看文件的前5行（使用-n选项）

```
head -5 file.txt
```

```
head -n 5 file.txt
```

查看文件的前20行

```
head -20 file.txt
```

```
head -n 20 file.txt
```

查看文件的前100个字节（使用-c选项）

```
head -c 100 file.txt
```

查看文件中除了最后3行以外的所有行（使用-n负值）

```
head -n -3 file.txt
```

3.13 tail 命令

tail 命令用于显示文件内容的末尾部分，默认情况下，只显示文件内容的末尾 10 行。

tail 命令的常用选项：

- -n num: 显示指定文件内容的末尾 num 行
- -c num: 显示指定文件内容的末尾 num 个字符
- -n+num: 从第 num 行开始显示指定文件的内容
- -f: 持续刷新文件内容，实时显示文件的最新变化

tail 命令”最强悍”的功能是可以持续刷新一个文件的内容，想要实时查看最新日志文件时，这个功能特别有用。使用-f 选项可以实时监控文件的变化，当文件有新内容添加时，会自动显示出来。

查看文件的最后10行（默认）

```
tail file.txt
```

查看文件的最后5行（使用-n选项）

```
tail -5 file.txt
```

```
tail -n 5 file.txt
```

查看文件的最后20行

```
tail -20 file.txt
```

```
tail -n 20 file.txt
```

查看文件的最后100个字符（使用-c选项）

```
tail -c 100 file.txt
```

从第20行开始显示文件内容（使用-n+num格式）

```
tail -n+20 file.txt
```

持续刷新文件内容，实时显示最新变化（使用-f选项）

```
tail -f file.txt
```

实时查看系统日志

```
tail -f /var/log/syslog
```

3.14 mkdir 命令

mkdir 命令用于创建一个目录。目录名可以为相对路径，也可以为绝对路径。

mkdir 命令的常用选项：

- -p: 在创建目录时，如果父目录不存在，则同时创建该目录及该目录的父目录

在当前目录创建一个新目录

```
mkdir new_directory
```

创建多个目录

```
mkdir dir1 dir2 dir3
```

创建嵌套目录（使用-p选项）

```
mkdir -p parent/child/grandchild
```

使用绝对路径创建目录

```
mkdir /path/to/new_directory
```

3.15 rmdir 命令

rmdir 命令用于删除空目录。目录名可以为相对路径，也可以为绝对路径。但所删除的目录必须为空目录。

rmdir 命令的常用选项：

- -p: 在删除目录时，一同删除父目录，但父目录中必须没有其他目录及文件

删除一个空目录

```
rmdir empty_directory
```

删除多个空目录

```
rmdir dir1 dir2 dir3
```

删除目录及其空的父目录（使用-p选项）

```
rmdir -p parent/child/grandchild
```

3.16 rm 命令

rm 命令用于删除文件或目录。默认情况下，rm 命令只能删除文件，不能删除目录。

rm 命令的常用选项：

- -a: 删除所有文件，包括隐藏文件
- -f: 强制删除，不提示确认
- -i: 删除前提示用户确认
- -R: 递归删除目录及其内容

删除一个文件

```
rm file.txt
```

删除多个文件

```
rm file1.txt file2.txt file3.txt
```

删除目录及其所有内容（使用-R选项）

```
rm -R directory
```

强制删除文件（使用-f选项）

```
rm -f file.txt
```

删除前提示确认（使用-i选项）

```
rm -i file.txt
```

删除所有文件，包括隐藏文件（使用-a选项）

```
rm -a *
```

3.17 cp 命令

cp 命令用于复制文件或目录。可以将一个或多个文件复制到指定的目标文件或目录中。这个命令是非常重要的，不同身份执行这个命令会有不同的结果产生，尤其是-a、-p 选项，对于不同身份来说，差异非常大。

在预设的条件中，cp 命令的源文件与目的文件的权限是不同的，目的文件的拥有者通常会命令操作者本身。想要复制文件给其他用户，也必须要注意文件的权限（包含读、写、执行以及文件拥有者等），否则，其他用户还是无法对你给的文件进行修改。

cp 命令的常用选项：

- -p: 保留源文件的属性（权限、时间戳等）
- -r: 递归复制目录及其内容
- -i: 交互式复制，覆盖前提示确认
- -a: 相当于-pdr，保留所有属性并递归复制

复制文件

```
cp source.txt destination.txt
```

复制文件到目录

```
cp file.txt directory/
```

复制多个文件到目录

```
cp file1.txt file2.txt file3.txt directory/
```

复制目录（使用-r选项）

```
cp -r source_directory destination_directory
```

保留属性复制（使用-p选项）

```
cp -p file.txt file_backup.txt
```

交互式复制（使用-i选项）

```
cp -i file.txt file.txt
```

保留所有属性并递归复制（使用-a选项）

```
cp -a source_dir target_dir
```

3.18 mv 命令

mv 命令主要用于文件或目录的移动或改名。

mv 命令的常用选项：

- -i: 交互式操作，覆盖前提示确认
- -f: 强制覆盖，不提示确认
- -v: 详细模式，显示移动过程

移动文件到另一个目录

```
mv file.txt directory/
```

文件改名

```
mv old_name.txt new_name.txt
```

移动目录

```
mv source_dir destination_dir
```

交互式移动（使用-i选项）

```
mv -i file.txt directory/
```

强制移动（使用-f选项）

```
mv -f file.txt directory/
```

3.19 touch 命令

touch 命令用于建立文件或更新文件的修改日期。

touch 命令的常用选项:

- -a: 只更新文件的访问时间
- -m: 只更新文件的修改时间
- -d: 指定文件的时间戳
- -t: 使用 [[CC]YY]MMDDhhmm[.ss] 格式指定时间戳

创建新文件

```
touch new_file.txt
```

更新文件的修改日期

```
touch existing_file.txt
```

同时更新多个文件的时间戳

```
touch file1.txt file2.txt file3.txt
```

只更新访问时间（使用-a选项）

```
touch -a file.txt
```

只更新修改时间（使用-m选项）

```
touch -m file.txt
```

指定时间戳（使用-d选项）

```
touch -d "2023-12-31 23:59:59" file.txt
```

3.20 whereis 命令

whereis 命令用来寻找命令的可执行文件所在的位置。

whereis 命令的常用选项:

- -b: 只查找可执行文件
- -m: 只查找手册页
- -s: 只查找源代码文件
- -u: 查找不常见的条目

查找命令的可执行文件、手册页和源代码

```
whereis ls
```

只查找可执行文件（使用-b选项）

```
whereis -b ls
```

只查找手册页（使用-m选项）

```
whereis -m ls
```

查找多个命令的位置

```
whereis ls grep find
```

3.21 whatis 命令

whatis 命令用于获取命令简介。它从某个程序的使用手册中抽出一行简单的介绍性文件，帮助用户迅速了解这个程序的具体功能。

获取命令简介

```
whatis ls
```

获取多个命令的简介

```
whatis ls grep find
```

获取系统调用的简介

```
whatis open read write
```

3.22 find 命令

find 命令用于查找文件。它的功能非常强大。

find 命令的常用选项：

- -name: 按文件名查找
- -type: 按文件类型查找（f: 普通文件，d: 目录，l: 符号链接，b: 块设备，c: 字符设备，p: 命名管道，s: 套接字）
- -size: 按文件大小查找，-size n 表示查找大小为 n 块的文件（一块为 512B），+n 表示大于 n 块，-n 表示小于 n 块，nc 表示 n 个字符
- -mtime: 按修改时间查找
- -ctime: 按更改时间查找

- -user: 按文件所有者查找
- -group: 按文件所属组查找
- -inum: 按 inode 号查找
- -perm: 按文件权限查找
- -newer: 查找比指定文件更新的文件
- -exec: 对找到的文件执行命令, 表示找到的文件,
; 表示命令结束
- -ok: 对找到的文件执行命令, 执行前会询问确认
- -print: 打印找到的文件路径 (默认选项)

在当前目录查找名为test.txt的文件

```
find . -name "test.txt"
```

查找所有普通文件

```
find . -type f
```

查找大于1MB的文件

```
find . -size +1M
```

查找最近7天修改的文件

```
find . -mtime -7
```

查找属于root用户的文件

```
find . -user root
```

查找具有执行权限的文件

```
find . -perm /u+x
```

查找文件并执行命令 (删除找到的文件)

```
find . -name "*.tmp" -exec rm {} \;
```

按文件所属组查找

```
find . -group users
```

显式使用-print选项（默认行为）

```
find . -name "*.txt" -print
```

按inode号查找文件

```
find . -inum 12345
```

按更改时间查找（最近7天更改的文件）

```
find . -ctime -7
```

按文件权限查找（精确匹配644权限）

```
find . -perm 644
```

查找比指定文件更新的文件

```
find . -newer reference.txt
```

使用-exec执行命令（删除找到的文件，无确认）

```
find . -name "*.tmp" -exec rm {} \;
```

使用-ok执行命令（删除找到的文件，执行前确认）

```
find . -name "*.tmp" -ok rm {} \;
```

后台运行find命令（添加&符号）

```
find . -name "*.log" -size +1M -exec gzip {} \; &
```

注意事项：由于 find 命令在执行过程中将消耗大量资源，所以建议以后台方式运行（在命令末尾添加 & 符号）。

3.23 grep 命令

grep 命令用于查找文件中包含指定字符串的行。

grep 命令的常用选项：

- -i: 忽略大小写
- -n: 显示行号
- -c: 统计匹配行的数量
- -l: 只显示包含匹配的文件名
- -h: 不显示文件名（当搜索多个文件时）

- -v: 反向查找，显示不包含指定字符串的行
- -r: 递归查找子目录
- -A: 显示匹配行及其后 n 行
- -B: 显示匹配行及其前 n 行
- -C: 显示匹配行及其前后 n 行
- -E: 使用扩展正则表达式

在文件中查找指定字符串

```
grep "pattern" file.txt
```

忽略大小写查找

```
grep -i "pattern" file.txt
```

显示行号

```
grep -n "pattern" file.txt
```

反向查找

```
grep -v "pattern" file.txt
```

递归查找子目录

```
grep -r "pattern" directory/
```

显示匹配行及其前后2行

```
grep -C 2 "pattern" file.txt
```

统计匹配行的数量

```
grep -c "pattern" file.txt
```

只显示包含匹配的文件名

```
grep -l "pattern" *.txt
```

不显示文件名（搜索多个文件时）

```
grep -h "pattern" *.txt
```

使用正则表达式符号（^表示行开始，\$表示行结尾）

```
grep "^pattern" file.txt # 查找以pattern开头的行
grep "pattern$" file.txt # 查找以pattern结尾的行
```

查找包含空格的字符串

```
grep "hello world" file.txt # 使用双引号
grep 'hello world' file.txt # 使用单引号
```

过滤掉带"#"的注释行和空白行

```
grep -v "^#" file.txt | grep -v "^$"
```

使用说明：在 `grep` 命令中，字符“^”表示行的开始，字符“\$”表示行的结尾。如果要查找的字符串中带有空格，则可以用单引号或双引号标注。

命令比较：`grep` 命令和 `find` 命令的差别在于，`grep` 命令是在文件中搜索满足条件的行，而 `find` 命令是在指定目录下根据文件的相关信息查找满足指定条件的文件。

3.24 dd 命令

`dd` 命令是一个比较重要而且有特色的命令，它能够让用户按照指定大小和数量的数据块来复制文件的内容。当然如果需要，还可以在复制过程中转换其中的数据。

`dd` 命令的功能也绝不仅限于复制文件这么简单。如果想把光驱设备中的光盘制作成 iso 映像文件，在 Windows 操作系统中需要借助于第三方软件才能做到，但在 Linux 操作系统中可以直接使用 `dd` 命令来压制映像文件，将它变成一个可立即使用的 iso 映像文件。

Linux 操作系统中有一个名为 `/dev/zero` 的设备文件，因为这个文件不会占用系统存储空间，却可以提供无穷无尽的数据，所以可以使用它作为 `dd` 命令的输入文件来生成一个指定大小的文件。

`dd` 命令的常用选项：

- `if`：输入文件（默认为标准输入）
- `of`：输出文件（默认为标准输出）
- `bs`：数据块大小（默认 512 字节）
- `count`：复制的数据块数量
- `seek`：输出文件开始时跳过的块数
- `skip`：输入文件开始时跳过的块数
- `conv`：转换方式（如 `notrunc`, `noerror`, `sync` 等）

- status: 显示进度信息 (如 status=progress)

基本复制文件

```
cp file1.txt file2.txt
```

使用dd复制文件 (更精确控制)

```
dd if=file1.txt of=file2.txt bs=1M count=1
```

创建指定大小的空文件 (1GB)

```
dd if=/dev/zero of=emptyfile.img bs=1M count=1024
```

将光驱制作成iso映像文件

```
dd if=/dev/cdrom of=disc_image.iso bs=2048 status=progress
```

备份MBR (主引导记录)

```
dd if=/dev/sda of=mbr_backup.img bs=512 count=1
```

克隆磁盘分区

```
dd if=/dev/sda1 of=/dev/sdb1 bs=4M status=progress
```

测试磁盘读写速度

```
time dd if=/dev/zero of=testfile bs=1M count=1000 oflag=direct
```

```
time dd if=testfile of=/dev/null bs=1M count=1000 iflag=direct
```

```
rm testfile
```

注意事项: - dd 命令的参数顺序不影响执行结果, 但建议使用”if= 输入文件 of= 输出文件” 的格式以提高可读性 - 执行 dd 命令时要特别小心, 尤其是操作磁盘设备时, 错误的参数可能导致数据丢失 - 使用 status=progress 选项可以在执行过程中显示复制进度

3.25 cal 命令

cal 命令用于显示指定月份或年份的日历。它是一个简单但实用的工具, 可以帮助用户快速查看日期信息。

cal 命令的使用规则:

- 不带任何参数: 显示当前月份的日历
- 一个参数: 表示年份 (范围为 1~9999), 显示该年份的完整日历
- 两个参数: 第一个表示月份, 第二个表示年份, 显示指定月份的日历

显示当前月份的日历

```
cal
```

显示2024年的完整日历

```
cal 2024
```

显示2024年12月的日历

```
cal 12 2024
```

cal 命令的常用选项：

- -1: 只显示当前月份（默认）
- -3: 显示上个月、当前月和下个月的日历
- -y: 显示当前年份的完整日历
- -j: 使用儒略日（一年中的第几天）显示日历
- -m: 以周一为一周的开始

显示上个月、当前月和下个月的日历

```
cal -3
```

显示当前年份的完整日历

```
cal -y
```

使用儒略日显示当前月份的日历

```
cal -j
```

以周一为一周的开始显示日历

```
cal -m
```

使用说明：cal 命令的输出会自动调整格式，使其在终端中显示美观。当指定年份时，会显示该年份的所有月份；当指定月份和年份时，会只显示该月份的日历。

3.26 man 命令

man 命令用于列出命令的帮助手册。它是一个非常重要的工具，可以帮助用户快速了解 Linux 命令的详细用法、选项和参数。

man 命令的功能：

- 查看命令的详细帮助文档
- 了解命令的选项、参数和用法
- 查看配置文件格式和系统调用
- 学习 Linux 系统管理知识

查看命令的帮助手册

```
man ls
```

查看配置文件的帮助手册

```
man passwd
```

查看系统调用的帮助手册

```
man 2 open
```

查看所有章节中包含指定关键字的帮助手册

```
man -a printf
```

在帮助手册中搜索关键字

```
man -k keyword
```

查看帮助手册的路径

```
man -w ls
```

man 命令的常用选项:

- -a: 显示所有匹配的手册页
- -d: 显示调试信息
- -f: 显示命令的简要说明（相当于 `whatis` 命令）
- -k: 搜索包含关键字的帮助手册（相当于 `apropos` 命令）
- -K: 在所有手册页中搜索关键字
- -w: 显示手册页的路径
- -M: 指定手册页的搜索路径
- -P: 指定使用的分页器

手册页章节：man 命令的手册页分为多个章节，每个章节包含不同类型的内容：

- 1: 用户命令
- 2: 系统调用
- 3: 库函数
- 4: 设备和特殊文件
- 5: 文件格式和配置文件
- 6: 游戏
- 7: 杂项
- 8: 系统管理命令

查看用户命令的手册页（默认章节1）

```
man ls
```

查看系统调用的手册页（章节2）

```
man 2 open
```

查看库函数的手册页（章节3）

```
man 3 printf
```

查看配置文件格式的手册页（章节5）

```
man 5 passwd
```

查看系统管理命令的手册页（章节8）

```
man 8 iptables
```

手册页结构：典型的 man 手册包含以下几部分：

- **NAME:** 命令的名字
- **SYNOPSIS:** 名字的概要，简单说明命令的使用方法
- **DESCRIPTION:** 详细描述命令的使用，如各种参数（选项）的作用
- **SEE ALSO:** 列出可能要查看的其他相关的手册页条目
- **AUTHOR、COPYRIGHT:** 作者和版权等信息

man 命令操作键：在 man 命令的帮助手册中，可以使用以下按键进行导航：

- Enter 键：向下移动一行
- Space 键：向下移动一页
- B 键：向上移动一页
- 方向键：向前、后、左、右移动
- Q 键：退出 man 命令
- /键：搜索关键字
- n 键：查找下一个匹配项
- N 键：查找上一个匹配项
- G 键：跳转到手册页末尾
- g 键：跳转到手册页开头

使用说明：man 命令是 Linux 系统中最重要帮助工具之一，几乎所有命令都有详细的帮助手册。通过 man 命令，用户可以快速了解命令的详细用法、选项和参数，是学习和使用 Linux 命令的必备工具。

注意事项：

- 某些命令可能在多个章节中都有手册页，可以使用 man -a 命令查看所有章节
- 如果不确定命令的完整名称，可以使用 man -k 命令搜索关键字
- man 命令的帮助手册通常使用 less 分页器显示，支持 less 命令的所有操作键
- 对于初学者，建议先使用 man -f 命令查看命令的简要说明，再使用 man 命令查看详细帮助

3.27 alias 命令

alias 命令用于创建命令的别名。它允许用户为常用的命令或命令组合创建简短的名称，提高命令行操作的效率。

alias 命令的功能：

- 创建命令别名
- 简化常用命令的输入
- 为复杂命令组合创建快捷方式

- 提高命令行操作效率

创建简单别名

```
alias ll='ls -l'
alias la='ls -a'
alias grep='grep --color=auto'
```

创建复杂别名

```
alias update='sudo apt update && sudo apt upgrade'
alias backup='tar -czvf backup.tar.gz /home/user'
```

查看所有已定义的别名

```
alias
```

查看特定别名的定义

```
alias ll
```

使用说明: - alias 命令是 shell 的内置命令, 用于创建命令别名 - 别名只在当前 shell 会话中有效, 退出 shell 后会失效 - 要使别名永久生效, 需要将其添加到 shell 配置文件中 (如 ~/.bashrc) - 别名可以包含参数, 但参数只能在命令末尾使用 - 如果别名与原命令同名, 别名会覆盖原命令

永久别名配置: 要使别名永久生效, 需要将其添加到 shell 配置文件中:

编辑bash配置文件

```
nano ~/.bashrc
```

在文件末尾添加别名

```
alias ll='ls -l'
alias la='ls -a'
alias grep='grep --color=auto'
```

保存并退出后, 执行以下命令使配置生效

```
source ~/.bashrc
```

注意事项:

- 别名只在当前 shell 会话中有效, 需要永久配置才能在所有会话中使用
- 如果别名包含空格, 需要使用引号括起来

- 别名可以嵌套，但要注意避免循环引用
- 在脚本中，别名默认是禁用的，需要使用 `shopt -s expand_aliases` 启用
- 要在别名中使用参数，需要使用函数而不是别名

3.28 unalias 命令

`unalias` 命令用于取消别名的定义。它允许用户删除不再需要的别名，或临时禁用某个别名。

`unalias` 命令的功能：

- 删除已定义的别名
- 删除所有别名
- 临时禁用别名

删除特定别名

```
unalias ll
```

删除多个别名

```
unalias ll la grep
```

删除所有别名

```
unalias -a
```

临时使用原命令（在命令前加反斜杠）

```
\ls
```

使用原命令的完整路径

```
/bin/ls
```

`unalias` 命令的常用选项：

- `-a`：删除所有已定义的别名

使用说明： - `unalias` 命令是 shell 的内置命令，用于删除别名 - 删除别名后，原命令恢复可用 - 使用 `unalias -a` 会删除所有已定义的别名 - 要临时使用原命令而不删除别名，可以在命令前加反斜杠

注意事项：

- 删除别名不会影响 shell 配置文件中的定义

- 要永久删除别名，需要从配置文件中删除并重新加载配置
- 临时使用原命令的方法：在命令前加反斜杠或使用完整路径
- unalias 命令只能删除当前 shell 会话中的别名

alias 与 unalias 的使用场景：

- **日常使用：**为常用命令创建别名，提高效率
- **临时调整：**创建临时别名用于特定任务
- **环境配置：**在配置文件中定义永久别名
- **冲突解决：**使用 unalias 删除冲突的别名
- **调试：**临时禁用别名以使用原命令

3.29 history 命令

history 命令用于显示用户最近执行的命令，可以保留的历史命令数和环境变量 HISTSIZE 有关。只要在编号前加“!”，就可以重新运行 history 中显示出的命令行。

history 命令的功能：

- 显示最近执行的命令历史
- 查看指定数量的历史命令
- 执行历史命令
- 清空命令历史
- 修改历史命令

显示所有历史命令

```
history
```

显示最近10条历史命令

```
history 10
```

显示最近20条历史命令

```
history 20
```

执行历史命令中的第100条命令

```
!100
```

执行上一条命令

!!

执行上一条以"ls"开头的命令

!ls

清空命令历史

history -c

删除历史文件

history -w

history 命令的常用选项:

- -c: 清空当前历史命令列表
- -d: 删除指定编号的历史命令
- -a: 将当前会话的历史命令追加到历史文件
- -n: 从历史文件中读取尚未读取的历史命令
- -r: 读取历史文件并追加到当前历史列表
- -w: 将当前历史列表写入历史文件
- -p: 展开历史命令但不执行
- -s: 将命令添加到历史列表而不执行

历史命令执行: 在 history 命令的输出中, 每条命令都有一个编号, 可以使用以下方式重新执行历史命令:

- **!n:** 执行编号为 n 的历史命令
- **!!:** 执行上一条命令
- **!-n:** 执行倒数第 n 条命令
- **!string:** 执行最近一条以 string 开头的命令
- **!?string?:** 执行最近一条包含 string 的命令
- **ôldnew:** 将上一条命令中的 old 替换为 new 并执行

执行编号为100的历史命令

!100

执行上一条命令

!!

执行倒数第3条命令

!-3

执行最近一条以"ls"开头的命令

!ls

执行最近一条包含"grep"的命令

!?grep?

将上一条命令中的"file1"替换为"file2"并执行

^file1^file2

环境变量配置: history 命令的行为可以通过以下环境变量进行配置:

- **HISTSIZE**: 设置内存中保存的历史命令数量
- **HISTFILESIZE**: 设置历史文件中保存的历史命令数量
- **HISTFILE**: 指定历史文件的路径 (默认为 ~/.bash_history)
- **HISTCONTROL**: 控制哪些命令被保存到历史中
- **HISTIGNORE**: 指定不保存到历史的命令模式
- **HISTTIMEFORMAT**: 设置历史命令的时间戳格式

在 ~/.bashrc 中添加以下配置

```
export HISTSIZE=1000
```

```
export HISTFILESIZE=2000
```

```
export HISTCONTROL=ignoredups:erasedups
```

```
export HISTIGNORE="ls:cd:pwd:exit"
```

```
export HISTTIMEFORMAT="%F %T "
```

HISTCONTROL 的值:

- **ignoredups**: 忽略重复的命令

- **erasedups:** 删除所有重复的命令
- **ignorespace:** 忽略以空格开头的命令
- **ignoreboth:** 相当于 ignorespace:ignoredups

使用说明: - history 命令是 shell 的内置命令，用于查看和执行历史命令 - 历史命令保存在 ~/.bash_history 文件中 - 使用上下方向键可以浏览历史命令 - 使用 Ctrl+R 可以搜索历史命令 - 历史命令在多个 shell 会话之间共享

注意事项:

- 历史命令文件在 shell 退出时才会更新
- 敏感信息（如密码）不应保存在历史命令中
- 可以在命令前加空格来避免命令被保存到历史中
- 使用 history -c 可以清空当前会话的历史命令
- 要删除历史文件，需要删除 ~/.bash_history 文件

历史命令搜索:

- **Ctrl+R:** 反向搜索历史命令
- **Ctrl+S:** 正向搜索历史命令
- **history | grep:** 使用 grep 搜索历史命令

使用grep搜索历史命令

```
history | grep "ssh"
```

使用Ctrl+R交互式搜索

按下Ctrl+R后，输入要搜索的关键字

第 4 节 文本编辑

- Vi/Vim 编辑器基础
- Nano 编辑器使用
- 常用编辑命令与快捷键

第四章 Linux 用户与权限管理

第 1 节 用户与组的概念

用户账户是用户的身份标识。用户通过用户账户可以登录系统，并访问已经被授权的资源。系统依据账户来区分属于每个用户的文件、进程、任务，并给每个用户提供特定的工作环境（如用户的工作目录、shell 版本以及图形化的环境配置等），使每个用户都能各自不受干扰地独立工作。

Linux 操作系统下的用户账户分为两种：普通用户账户和超级用户账户（root）。普通用户账户在系统中只能进行普通工作，只能访问他们拥有的或者有权限执行的文件。超级用户账户也叫管理员账户，它的任务是对普通用户和整个系统进行管理。超级用户账户对系统具有绝对的控制权，能够对系统进行一切操作，如操作不当很容易造成系统损坏。

因此即使系统只有一个用户使用，也应该在超级用户账户之外再建立一个普通用户账户，在用户进行普通工作时以普通用户账户登录系统。

在 Linux 操作系统中，为了方便管理员的管理和用户的工作，产生了组的概念。组是具有相同特性的用户的逻辑集合，使用组有利于系统管理员按照用户的特性组织和管理用户，提高工作效率。有了组，在进行资源授权时可以把权限赋予某个组，组中的成员即可自动获得这种权限。一个用户账户可以同时是多个组的成员，其中某个组是该用户的主组（私有组），其他组为该用户的附属组（标准组）。

root 用户的 UID 为 0；系统用户的 UID 从 1 到 999；普通用户的 UID 可以在创建时由管理员指定，如果不指定，则用户的 UID 默认从 1000 开始顺序编号。系统用户是一些标准账户，此类账户的 shell 为/sbin/nologin，代表无本地登录权限。在 Linux 操作系统中，创建用户账户的同时也会创建一个与用户同名的组，该组是用户的主组。普通组的 GID 默认也从 1000 开始编号。

用户账户信息和组信息分别存储在用户账户文件和组文件中。

- 用户（User）与用户 ID（UID）
- 组（Group）与组 ID（GID）
- 主要组与附加组
- /etc/passwd, /etc/shadow, /etc/group 文件解析

1.1 /etc/passwd 文件

/etc/passwd 文件是 Linux 系统中最重要用户账户信息文件之一，它存储了系统中所有用户的基本信息。该文件是一个文本文件，所有用户都可以读取，但只有 root 用户可以修改。

/etc/passwd 文件的每一行代表一个用户账户，每行包含 7 个字段，字段之间用冒号 (:) 分隔。格式如下：

用户名:密码:UID:GID:用户信息:主目录:登录shell

各字段的含义：

- **用户名：**用户的登录名称，长度通常为 1-32 个字符
- **密码：**用户密码的占位符，实际密码存储在/etc/shadow 文件中，此处通常显示为 x
- **UID：**用户 ID，唯一标识系统中的每个用户
- **GID：**用户主组的组 ID
- **用户信息：**用户的描述信息，如全名、电话号码等
- **主目录：**用户登录后的默认工作目录，通常为/home/用户名
- **登录 shell：**用户登录后使用的 shell 程序，如/bin/bash、/bin/sh 等

示例：

```
root:x:0:0:root:/root:/bin/bash
```

```
user1:x:1000:1000:User One:/home/user1:/bin/bash
```

注意事项：

- /etc/passwd 文件对所有用户可读，因此密码字段不存储实际密码
- 实际密码存储在/etc/shadow 文件中，只有 root 用户可读
- 修改/etc/passwd 文件前应先备份
- 不建议直接编辑/etc/passwd 文件，应使用 useradd、usermod 等命令

1.2 /etc/shadow 文件

/etc/shadow 文件是 Linux 系统中存储用户密码信息的文件，该文件只有 root 用户可以读取和修改，确保了用户密码的安全性。与/etc/passwd 文件不同，/etc/shadow 文件存储了用户的加密密码以及密码相关的策略信息。

/etc/shadow 文件的每一行代表一个用户账户，每行包含 9 个字段，字段之间用冒号 (:) 分隔。格式如下：

用户名:加密密码:最后修改时间:最小密码年龄:最大密码年龄:密码警告时间:密码不活动时间:账

各字段的含义：

- **用户名：**用户的登录名称，必须与/etc/passwd 文件中的用户名一致
- **加密密码：**用户密码的加密值，如果为 * 或! 表示该账户被锁定
- **最后修改时间：**从 1970 年 1 月 1 日到密码最后修改日期的天数
- **最小密码年龄：**密码修改后最少要使用的天数，0 表示可以随时修改
- **最大密码年龄：**密码有效的最大天数，超过此天数必须修改密码
- **密码警告时间：**密码过期前多少天开始警告用户
- **密码不活动时间：**密码过期后多少天账户被禁用
- **账户过期时间：**从 1970 年 1 月 1 日到账户过期日期的天数，空表示永不过期
- **保留字段：**保留字段，目前未使用

示例：

```
root:$6$rounds=656000$salt$hash:18570:0:99999:7:::
user1:$6$rounds=656000$salt$hash:18571:0:99999:7:::
```

字段说明：

- **加密密码：**使用 SHA-512 等加密算法加密的密码值
- **最后修改时间：**18570 表示从 1970 年 1 月 1 日开始的第 18570 天
- **最小密码年龄：**0 表示可以随时修改密码
- **最大密码年龄：**99999 表示约 273 年，实际可视为永不过期
- **密码警告时间：**7 表示密码过期前 7 天开始警告

- **密码不活动时间：**空表示密码过期后不禁用账户
- **账户过期时间：**空表示账户永不过期

注意事项：

- `/etc/shadow` 文件只有 `root` 用户可以读取和修改
- 加密密码字段为 `*` 或 `!` 时表示账户被锁定
- 修改 `/etc/shadow` 文件前应先备份
- 不建议直接编辑 `/etc/shadow` 文件，应使用 `passwd`、`usermod` 等命令
- 密码策略可以通过 `/etc/login.defs` 文件配置

1.3 `/etc/login.defs` 文件

`/etc/login.defs` 文件是 Linux 系统中用于配置用户和密码策略的配置文件，该文件定义了创建用户和设置密码时的默认值和策略。该文件对 `useradd`、`usermod`、`passwd` 等命令的行为产生影响，是系统管理员管理用户账户和密码策略的重要配置文件。

`/etc/login.defs` 文件的主要配置项：

- **CREATE_HOME：**是否在创建用户时创建主目录，默认为 `yes`
- **MAIL_DIR：**用户邮件目录的路径，默认为 `/var/spool/mail`
- **MAIL_FILE：**用户邮件文件的路径，默认为 `/var/spool/mail/$USER`
- **PASS_MAX_DAYS：**密码有效的最大天数，默认为 `99999`
- **PASS_MIN_DAYS：**密码修改后最少要使用的天数，默认为 `0`
- **PASS_MIN_LEN：**密码的最小长度，默认为 `5`
- **PASS_WARN_AGE：**密码过期前多少天开始警告，默认为 `7`
- **UID_MIN：**普通用户 `UID` 的最小值，默认为 `1000`
- **UID_MAX：**普通用户 `UID` 的最大值，默认为 `60000`
- **GID_MIN：**普通组 `GID` 的最小值，默认为 `1000`
- **GID_MAX：**普通组 `GID` 的最大值，默认为 `60000`
- **SYS_UID_MIN：**系统用户 `UID` 的最小值，默认为 `201`
- **SYS_UID_MAX：**系统用户 `UID` 的最大值，默认为 `999`

- **SYS_GID_MIN**: 系统组 GID 的最小值, 默认为 201
- **SYS_GID_MAX**: 系统组 GID 的最大值, 默认为 999
- **ENCRYPT_METHOD**: 密码加密方法, 如 SHA512、MD5 等
- **USERGROUPS_ENAB**: 是否在创建用户时创建同名组, 默认为 yes

示例配置:

密码策略配置

```
PASS_MAX_DAYS    99999
PASS_MIN_DAYS     0
PASS_MIN_LEN      5
PASS_WARN_AGE     7
```

UID和GID范围配置

```
UID_MIN          1000
UID_MAX          60000
GID_MIN          1000
GID_MAX          60000
SYS_UID_MIN      201
SYS_UID_MAX      999
SYS_GID_MIN      201
SYS_GID_MAX      999
```

用户创建配置

```
CREATE_HOME      yes
USERGROUPS_ENAB yes
MAIL_DIR         /var/spool/mail
```

密码加密方法

```
ENCRYPT_METHOD    SHA512
```

使用说明: - /etc/login.defs 文件是全局配置文件, 影响所有用户创建和密码设置操作 - 修改配置后, 新创建的用户和密码设置将使用新的配置 - 已存在的用户不会受到影响 - 配置文件中的注释以 # 开头 - 空行和注释行会被忽略

注意事项:

- 修改/etc/login.defs 文件前应先备份

- 确保 UID 和 GID 范围不与现有用户和组冲突
- 密码策略应根据安全需求合理配置
- 修改密码加密方法可能影响现有用户的密码验证
- 建议在测试环境中验证配置后再应用到生产环境

1.4 /etc/group 文件

/etc/group 文件是 Linux 系统中存储组信息的文件，该文件存储了系统中所有组的基本信息。该文件是一个文本文件，所有用户都可以读取，但只有 root 用户可以修改。

/etc/group 文件的每一行代表一个组，每行包含 4 个字段，字段之间用冒号 (:) 分隔。格式如下：

组名:组密码:GID:组成员列表

各字段的含义：

- **组名：**组的名称，长度通常为 1-32 个字符
- **组密码：**组密码的占位符，实际组密码存储在/etc/gshadow 文件中，此处通常显示为 x 或为空
- **GID：**组 ID，唯一标识系统中的每个组
- **组成员列表：**属于该组的用户列表，多个用户用逗号分隔

示例：

```
root:x:0:
wheel:x:10:user1,user2
users:x:100:
```

注意事项：

- /etc/group 文件对所有用户可读
- 组密码很少使用，通常不设置
- 组成员列表只包含附加组成员，不包含主组成员
- 修改/etc/group 文件前应先备份
- 不建议直接编辑/etc/group 文件，应使用 groupadd、groupmod 等命令

1.5 /etc/gshadow 文件

/etc/gshadow 文件是 Linux 系统中存储组密码信息的文件，该文件只有 root 用户可以读取和修改，确保了组密码的安全性。与/etc/group 文件不同，/etc/gshadow 文件存储了组的加密密码以及组相关的策略信息。

/etc/gshadow 文件的每一行代表一个组，每行包含 4 个字段，字段之间用冒号 (:) 分隔。格式如下：

组名:加密密码:组管理员:组成员列表

各字段的含义：

- **组名：**组的名称，必须与/etc/group 文件中的组名一致
- **加密密码：**组密码的加密值，如果为 * 或! 表示该组被锁定或无密码
- **组管理员：**可以管理该组的用户列表，多个用户用逗号分隔
- **组成员列表：**属于该组的用户列表，多个用户用逗号分隔

示例：

```
root:*::  
wheel:$6$rounds=656000$salt$hash:admin1,admin2  
users:!::
```

注意事项：

- /etc/gshadow 文件只有 root 用户可以读取和修改
- 组密码很少使用，大多数系统不启用组密码功能
- 加密密码字段为 * 或! 时表示该组被锁定或无密码
- 修改/etc/gshadow 文件前应先备份
- 不建议直接编辑/etc/gshadow 文件，应使用 gpasswd 等命令

第 2 节 用户与组管理命令

- 用户管理：useradd, usermod, userdel, passwd
- 组管理：groupadd, groupmod, groupdel
- 权限查看与修改：id, su, sudo, chmod, chown, chgrp
- 用户信息查看：who, w, last, lastlog

2.1 useradd 命令

useradd 命令用于创建新的用户账户。该命令会在系统中创建新用户，并自动创建与用户同名的组作为用户的主组。useradd 命令会为新用户创建主目录，并复制/etc/skel目录下的配置文件到用户的主目录中。

useradd 命令的功能：

- 创建新的用户账户
- 自动创建与用户同名的组
- 创建用户的主目录
- 复制默认配置文件到用户主目录
- 设置用户的登录 shell

创建新用户

```
useradd username
```

创建新用户并指定UID

```
useradd -u 1001 username
```

创建新用户并指定主组

```
useradd -g groupname username
```

创建新用户并指定主目录

```
useradd -d /home/custom username
```

创建新用户并指定登录shell

```
useradd -s /bin/bash username
```

创建新用户并添加注释

```
useradd -c "User Description" username
```

创建新用户但不创建主目录

```
useradd -M username
```

创建新用户并设置密码

```
useradd -p encrypted_password username
```


创建系统用户

`useradd -r username`

`useradd` 命令的常用选项:

- `-u`: 指定用户的 UID
- `-g`: 指定用户的主组
- `-G`: 指定用户的附加组, 多个组用逗号分隔
- `-d`: 指定用户的主目录
- `-m`: 创建用户的主目录 (默认选项)
- `-M`: 不创建用户的主目录
- `-n`: 不创建与用户同名的组
- `-s`: 指定用户的登录 shell
- `-c`: 添加用户的注释信息
- `-p`: 指定用户的加密密码
- `-r`: 创建系统用户
- `-e`: 指定账户的过期日期, 格式为 YYYY-MM-DD
- `-f`: 指定密码过期后多少天账户被禁用。如果为 0, 账户过期后将立即被禁用; 如果为-1, 账户过期后, 将不被禁用, 即永不过期
- `-k`: 指定骨架目录, 默认为 /etc/skel

使用说明: - `useradd` 命令需要 root 权限才能执行 - 创建用户后, 需要使用 `passwd` 命令为用户设置密码 - 默认情况下, `useradd` 会创建与用户同名的组 - 使用 `-n` 选项时, 不会创建与用户同名的组 - 用户的主目录默认为 /home/用户名 - 默认的登录 shell 为 /bin/bash - /etc/skel 目录下的文件会被复制到新用户的主目录中

注意事项:

- 创建用户后, 用户账户默认是锁定的, 需要设置密码才能登录
- 指定 UID 时, 确保该 UID 未被其他用户使用
- 指定主目录时, 确保该目录不存在或为空
- 创建系统用户时, 通常不创建主目录
- 删除用户时, 用户的主目录不会被自动删除

2.2 userdel 命令

userdel 命令用于删除用户账户。该命令会从系统中删除指定的用户账户,包括/etc/passwd、/etc/shadow、/etc/group 等文件中的相关记录。默认情况下, userdel 命令不会删除用户的主目录和邮件文件。

userdel 命令的功能:

- 删除用户账户
- 删除用户的主目录
- 删除用户的邮件文件
- 从组文件中删除用户记录

删除用户账户

```
userdel username
```

删除用户账户及其主目录

```
userdel -r username
```

强制删除用户账户, 即使用户正在登录

```
userdel -f username
```

删除用户账户及其主目录和邮件文件

```
userdel -r -f username
```

userdel 命令的常用选项:

- -r: 删除用户的主目录和邮件文件
- -f: 强制删除用户账户, 即使用户正在登录
- -Z: 删除用户的 SELinux 用户映射

使用说明: - userdel 命令需要 root 权限才能执行 - 默认情况下, userdel 不会删除用户的主目录和邮件文件 - 使用-r 选项可以删除用户的主目录和邮件文件 - 如果用户正在登录系统, 需要先让用户退出或使用-f 选项强制删除 - 删除用户后, 该用户拥有的文件不会被自动删除, 文件的所有者会变为 UID - 也可以直接删除/etc/passwd 和/etc/shadow 文件中要删除的用户对应的行来删除账户 - 但使用 userdel 命令是更安全推荐的方法, 因为它会同时更新所有相关文件

注意事项:

- 删除用户前，应先备份用户的重要数据
- 删除用户后，该用户的主目录和邮件文件将无法恢复
- 删除用户不会删除该用户拥有的文件，这些文件会保留在系统中
- 如果用户正在运行进程，删除用户可能导致进程出现问题
- 建议在删除用户前，先确认该用户没有重要的进程在运行
- 删除用户后，该用户的 UID 可能被重新分配给新用户

2.3 adduser 命令

adduser 命令用于创建用户账户。adduser 命令实际上是 useradd 命令的友好版本，它提供了交互式的用户创建方式，可以自动创建用户的主目录、设置初始密码等。在大多数 Linux 发行版中，adduser 是一个 Perl 脚本，它调用 useradd 命令来完成实际的创建工作。

adduser 命令的功能：

- 创建用户账户
- 创建用户的主目录
- 创建与用户同名的组
- 设置用户的初始密码
- 复制/etc/skel 目录下的文件到用户主目录

创建用户账户（交互式）

```
adduser username
```

创建系统用户

```
adduser --system username
```

创建用户账户并指定主目录

```
adduser --home /custom/home username
```

创建用户账户并指定登录shell

```
adduser --shell /bin/zsh username
```

创建用户账户并禁用登录

```
adduser --disabled-login username
```

adduser 命令的常用选项:

- `-system`: 创建系统用户
- `-home`: 指定用户的主目录
- `-shell`: 指定用户的登录 shell
- `-disabled-login`: 创建用户但禁用登录
- `-disabled-password`: 创建用户但不设置密码
- `-gecos`: 添加用户的注释信息
- `-ingroup`: 指定用户的主组
- `-no-create-home`: 不创建用户的主目录
- `-encrypt-home`: 加密用户的主目录

使用说明: - adduser 命令需要 root 权限才能执行 - adduser 命令提供交互式界面, 会提示输入密码、全名等信息 - 默认情况下, adduser 会创建用户的主目录 - 默认情况下, adduser 会创建与用户同名的组 - 默认的登录 shell 为 `/bin/bash` - `/etc/skel` 目录下的文件会被复制到新用户的主目录中 - adduser 命令比 useradd 命令更友好, 适合新手使用

注意事项:

- adduser 命令在不同 Linux 发行版中可能有所不同
- 在某些发行版中, adduser 是 useradd 的别名
- 创建用户后, 用户账户默认是解锁的, 可以立即登录
- 指定主目录时, 确保该目录不存在或为空
- 创建系统用户时, 通常不创建主目录
- adduser 命令会自动设置用户的初始密码

2.4 groupadd 命令

groupadd 命令用于创建新的用户组。该命令会在 `/etc/group` 和 `/etc/gshadow` 文件中添加新的组记录。创建组后, 可以将用户添加到该组中, 以使用户获得该组的权限。

groupadd 命令的功能:

- 创建新的用户组

- 指定组的 GID
- 创建系统组
- 创建加密的组密码

创建新组

```
groupadd groupname
```

创建新组并指定GID

```
groupadd -g 1005 groupname
```

创建系统组

```
groupadd -r groupname
```

创建新组并指定GID范围

```
groupadd -K GID_MIN=1000 -K GID_MAX=2000 groupname
```

创建新组并强制使用指定的GID

```
groupadd -o -g 1005 groupname
```

groupadd 命令的常用选项:

- -g: 指定组的 GID
- -r: 创建系统组
- -o: 允许使用非唯一的 GID
- -f: 如果组已存在, 则成功退出
- -K: 覆盖/etc/login.defs 中的默认值
- -p: 为组设置加密密码

使用说明: - groupadd 命令需要 root 权限才能执行 - 默认情况下, 组的 GID 从 1000 开始顺序编号 - 系统组的 GID 从 1 到 999 - 如果不指定 GID, 系统会自动分配一个可用的 GID - 创建组后, 可以使用 usermod 命令将用户添加到该组 - 组的密码通常不使用, 因为用户可以通过 sudo 等工具获得组权限 - addgroup 命令是 groupadd 命令的友好版本, 提供了更交互式的组创建方式 - 在大多数 Linux 发行版中, addgroup 是一个脚本, 它调用 groupadd 命令来完成实际的创建工作

注意事项:

- 指定 GID 时，确保该 GID 未被其他组使用
- 创建系统组时，通常 GID 小于 1000
- 组名只能包含字母、数字、下划线和连字符
- 组名不能以连字符开头
- 组名的长度通常限制为 32 个字符
- 删除组前，应先确认没有用户使用该组作为主组

2.5 groupmod 命令

groupmod 命令用于修改用户组的属性。该命令可以修改组的名称、GID 等属性，是管理用户组的重要工具。

groupmod 命令的功能：

- 修改组的名称
- 修改组的 GID
- 将组设置为系统组
- 修改组的密码

修改组名

```
groupmod -n newgroupname oldgroupname
```

修改组的GID

```
groupmod -g 1006 groupname
```

修改组名和GID

```
groupmod -n newgroupname -g 1006 oldgroupname
```

强制使用指定的GID

```
groupmod -o -g 1006 groupname
```

修改组的密码

```
groupmod -p encrypted_password groupname
```

groupmod 命令的常用选项：

- -g: 指定组的 GID

- -n: 修改组的名称
- -o: 允许使用非唯一的 GID
- -p: 为组设置加密密码
- -R: 指定 chroot 目录

使用说明: - groupmod 命令需要 root 权限才能执行 - 修改组的 GID 时, 应确保新 GID 未被其他组使用 - 如果使用-o 选项, 可以允许使用非唯一的 GID - 修改组名时, 会同时更新/etc/group 和/etc/gshadow 文件中的记录 - 修改组的 GID 时, 可能需要同时更新使用该组的文件和目录的权限

注意事项:

- 修改组的 GID 前, 应先确认没有用户使用该组作为主组
- 修改组名时, 会影响所有使用该组的文件和目录
- 修改组的 GID 可能会影响系统的安全性
- 建议在修改组属性前备份相关文件

2.6 groupdel 命令

groupdel 命令用于删除用户组。该命令会从/etc/group 和/etc/gshadow 文件中删除指定的组记录。默认情况下, groupdel 命令不会删除组的主目录。

groupdel 命令的功能:

- 删除用户组
- 从/etc/group 文件中删除组记录
- 从/etc/gshadow 文件中删除组记录

删除组

```
groupdel groupname
```

删除多个组

```
groupdel groupname1 groupname2 groupname3
```

groupdel 命令的常用选项: groupdel 命令没有常用的选项, 只需要指定要删除的组名。

使用说明: - groupdel 命令需要 root 权限才能执行 - 删除组前, 应先确认没有用户使用该组作为主组 - 如果有用户的主组是该组, groupdel 命令会失败 - 删除组后, 该组的权限将不再有效 - 删除组不会影响用户的个人文件

注意事项:

- 删除组前，应先确认没有用户使用该组作为主组
- 如果有用户的主组是该组，需要先将用户的主组改为其他组
- 删除组后，该组的 GID 可能被重新分配给新组
- 删除组不会删除用户的个人文件
- 建议在删除组前，先备份重要的组信息
- 系统组通常不建议删除

2.7 gpasswd 命令

gpasswd 命令用于管理用户组的密码和组成员。该命令可以设置组密码、添加或删除组成员、指定组管理员等。gpasswd 命令是管理组的重要工具，提供了比 groupadd 和 groupmod 更细粒度的组管理功能。

gpasswd 命令的功能：

- 设置组密码
- 添加组成员
- 删除组成员
- 指定组管理员
- 启用或禁用组成员管理

设置组密码

```
gpasswd groupname
```

添加组成员

```
gpasswd -a username groupname
```

删除组成员

```
gpasswd -d username groupname
```

指定组管理员

```
gpasswd -A admin1,admin2 groupname
```

禁用组成员管理

```
gpasswd -r groupname
```


允许组成员管理

```
gpasswd -R groupname
```

添加多个组成员

```
gpasswd -M user1,user2 groupname
```

gpasswd 命令的常用选项:

- -a: 添加用户到组
- -d: 从组中删除用户
- -A: 指定组管理员
- -M: 设置组成员列表
- -r: 删除组密码
- -R: 禁用组密码
- -l: 锁定组
- -u: 解锁组
- -Q: 查询组信息

使用说明: - gpasswd 命令需要 root 权限才能执行 - 设置组密码后, 用户可以使用 newgrp 命令切换到该组 - 组管理员可以添加或删除组成员, 无需 root 权限 - 添加组成员时, 如果用户不是该组的成员, 会被添加到组中 - 删除组成员时, 如果用户是该组的成员, 会被从组中移除

注意事项:

- 组密码很少使用, 因为用户可以通过 sudo 等工具获得组权限
- 组管理员只能管理他们被指定为管理员的组
- 添加组成员时, 确保用户存在
- 删除组成员时, 确保用户是该组的成员
- 建议谨慎使用组密码, 因为它可能会带来安全风险

2.8 su 命令

su 命令用于切换用户身份。该命令可以让当前用户在不退出登录的情况下，切换到其他用户身份，例如从 root 管理员切换到普通用户，或从普通用户切换到 root 管理员（需要输入 root 密码）。

su 命令的功能：

- 切换用户身份
- 以其他用户身份执行命令
- 切换到 root 用户
- 保持当前环境变量
- 加载目标用户的环境变量

切换到指定用户

```
su username
```

切换到root用户

```
su
```

切换到root用户（完全切换环境）

```
su -
```

以其他用户身份执行命令

```
su -c "command" username
```

以root身份执行命令

```
su -c "command"
```

切换到指定用户并加载其环境变量

```
su - username
```

su 命令的常用选项：

- -: 切换用户时，同时加载目标用户的环境变量
- -c: 执行指定的命令后退出
- -l: 与-l选项相同，加载目标用户的环境变量

- -m: 切换用户时, 保持当前的环境变量
- -s: 指定要使用的 shell

使用说明: - su 命令需要输入目标用户的密码才能切换 - root 用户切换到其他用户时, 不需要输入密码 - 使用 su -命令会完全切换到目标用户的环境, 包括工作目录、环境变量等 - 使用 su 命令只会切换用户身份, 不会切换工作目录和环境变量 - 执行完命令后, 可以使用 exit 命令退出当前用户身份, 返回原用户 - su 命令与用户名之间的“-”非常重要, 它表示完全切换到新用户, 包括环境变量、工作目录等所有信息 - 强烈建议在切换用户身份时添加“-”, 以确保获得正确的用户环境 - 从 root 管理员切换到普通用户不需要密码验证, 这是系统为了方便管理员操作而设计的 - 从普通用户切换到 root 管理员需要进行密码验证, 这是一个必要的安全检查, 防止未授权的用户获得 root 权限

注意事项:

- 切换到 root 用户时, 应谨慎操作, 避免误操作导致系统损坏
- 普通用户切换到 root 用户需要输入 root 密码, 应确保密码的安全性
- 长时间使用 root 权限可能会带来安全风险, 建议完成操作后及时退出
- 在脚本中使用 su 命令时, 应注意密码输入的问题
- 对于频繁的管理操作, 建议使用 sudo 命令代替 su 命令

2.9 vipw 命令

vipw 命令用于安全地编辑/etc/passwd 文件。该命令会锁定文件, 防止其他用户同时修改, 并且在编辑完成后检查文件的语法正确性, 确保文件格式不会被破坏。vipw 命令是系统管理员管理用户账户的重要工具。

vipw 命令的功能:

- 安全编辑/etc/passwd 文件
- 锁定文件, 防止并发修改
- 检查文件语法正确性
- 备份原始文件
- 编辑/etc/shadow 文件 (使用-s 选项)

编辑/etc/passwd文件

vipw

编辑/etc/shadow文件

`vipw -s`

显示编辑的文件路径

`vipw -h`

强制编辑，即使文件被锁定

`vipw -f`

`vipw` 命令的常用选项：

- `-s`: 编辑/etc/shadow 文件
- `-h`: 显示编辑的文件路径
- `-f`: 强制编辑，即使文件被锁定
- `-q`: 安静模式，不显示提示信息

使用说明： - `vipw` 命令需要 `root` 权限才能执行 - `vipw` 命令会使用系统默认的编辑器（通常是 `vi` 或 `vim`） - 编辑完成后，`vipw` 命令会检查文件的语法正确性 - 如果文件格式有错误，`vipw` 命令会提示并允许用户重新编辑 - `vipw` 命令会自动创建文件的备份，以防编辑出错

注意事项：

- 使用 `vipw` 命令编辑用户信息时，应确保输入的格式正确
- 手动修改/etc/passwd 文件可能会导致系统故障，建议使用 `vipw` 命令
- 编辑/etc/shadow 文件时，应特别注意密码字段的格式
- 编辑完成后，应验证修改是否正确
- 建议在编辑前备份原始文件，以防意外情况

2.10 `vigr` 命令

`vigr` 命令用于安全地编辑/etc/group 文件。该命令与 `vipw` 命令类似，会锁定文件，防止其他用户同时修改，并且在编辑完成后检查文件的语法正确性，确保文件格式不会被破坏。`vigr` 命令是系统管理员管理用户组的重要工具。

`vigr` 命令的功能：

- 安全编辑/etc/group 文件

- 锁定文件，防止并发修改
- 检查文件语法正确性
- 备份原始文件
- 编辑/etc/gshadow 文件（使用-s 选项）

编辑/etc/group文件

`vigr`

编辑/etc/gshadow文件

`vigr -s`

显示编辑的文件路径

`vigr -h`

强制编辑，即使文件被锁定

`vigr -f`

`vigr` 命令的常用选项：

- -s: 编辑/etc/gshadow 文件
- -h: 显示编辑的文件路径
- -f: 强制编辑，即使文件被锁定
- -q: 安静模式，不显示提示信息

使用说明： - `vigr` 命令需要 root 权限才能执行 - `vigr` 命令会使用系统默认的编辑器（通常是 vi 或 vim） - 编辑完成后，`vigr` 命令会检查文件的语法正确性 - 如果文件格式有错误，`vigr` 命令会提示并允许用户重新编辑 - `vigr` 命令会自动创建文件的备份，以防编辑出错

注意事项：

- 使用 `vigr` 命令编辑组信息时，应确保输入的格式正确
- 手动修改/etc/group 文件可能会导致系统故障，建议使用 `vigr` 命令
- 编辑/etc/gshadow 文件时，应特别注意密码字段的格式
- 编辑完成后，应验证修改是否正确
- 建议在编辑前备份原始文件，以防意外情况

2.11 pwck 命令

pwck 命令用于检查/etc/passwd 和/etc/shadow 文件的完整性和正确性。该命令会检查文件的语法格式、用户 ID 的唯一性、主目录的存在性等，是系统管理员维护用户账户完整性的重要工具。

pwck 命令的功能：

- 检查/etc/passwd 文件的完整性
- 检查/etc/shadow 文件的完整性
- 验证用户 ID 的唯一性
- 验证主目录的存在性
- 检查 shell 的有效性
- 检测文件格式错误

检查/etc/passwd和/etc/shadow文件

pwck

详细模式，显示所有检查结果

pwck -r

只读模式，不修改文件

pwck -s

强制修复错误

pwck -y

pwck 命令的常用选项：

- -r: 详细模式，显示所有检查结果
- -s: 只读模式，不修改文件
- -y: 自动回答 yes，修复所有错误
- -q: 安静模式，只显示错误信息

使用说明： - pwck 命令需要 root 权限才能执行 - 执行 pwck 命令会检查用户账户的完整性 - 如果发现错误，pwck 命令会提示并询问是否修复 - 使用-y 选项可以自动修复所有错误 - 建议定期执行 pwck 命令，确保用户账户信息的完整性

注意事项：

- 执行 pwck 命令可能会修改用户账户信息，应谨慎操作
- 建议在执行 pwck 命令前备份/etc/passwd 和/etc/shadow 文件
- 修复错误时，应确保了解修改的内容
- 定期执行 pwck 命令可以防止用户账户信息损坏
- 系统启动时，通常会自动执行 pwck 命令检查用户账户

2.12 grpck 命令

grpck 命令用于检查/etc/group 和/etc/gshadow 文件的完整性和正确性。该命令会检查文件的语法格式、组 ID 的唯一性、组名的有效性等，是系统管理员维护用户组完整性的重要工具。

grpck 命令的功能：

- 检查/etc/group 文件的完整性
- 检查/etc/gshadow 文件的完整性
- 验证组 ID 的唯一性
- 验证组名的有效性
- 检测文件格式错误
- 检查组成员的有效性

检查/etc/group和/etc/gshadow文件
grpck

详细模式，显示所有检查结果
grpck -r

只读模式，不修改文件
grpck -s

强制修复错误
grpck -y

grpck 命令的常用选项：

- -r：详细模式，显示所有检查结果

- -s: 只读模式, 不修改文件
- -y: 自动回答 yes, 修复所有错误
- -q: 安静模式, 只显示错误信息

使用说明: - grpck 命令需要 root 权限才能执行 - 执行 grpck 命令会检查用户组的完整性 - 如果发现错误, grpck 命令会提示并询问是否修复 - 使用-y 选项可以自动修复所有错误 - 建议定期执行 grpck 命令, 确保用户组信息的完整性

注意事项:

- 执行 grpck 命令可能会修改用户组信息, 应谨慎操作
- 建议在执行 grpck 命令前备份/etc/group 和/etc/gshadow 文件
- 修复错误时, 应确保了解修改的内容
- 定期执行 grpck 命令可以防止用户组信息损坏
- 系统启动时, 通常会自动执行 grpck 命令检查用户组

2.13 id 命令

id 命令用于显示用户的 UID、GID 和所属的组信息。该命令可以显示当前用户或指定用户的身份信息, 是系统管理员和用户查询用户身份的重要工具。

id 命令的功能:

- 显示用户的 UID (用户 ID)
- 显示用户的 GID (组 ID)
- 显示用户所属的所有组
- 显示用户的用户名
- 显示组的名称
- 显示安全上下文 (SELinux)

显示当前用户的身份信息

```
id
```

显示指定用户的身份信息

```
id username
```


只显示UID

```
id -u
```

只显示GID

```
id -g
```

只显示所有GID

```
id -G
```

只显示用户名

```
id -un
```

只显示组名

```
id -gn
```

显示所有组名

```
id -Gn
```

显示安全上下文

```
id -Z
```

id 命令的常用选项:

- -u: 只显示 UID
- -g: 只显示 GID
- -G: 只显示所有 GID
- -n: 与-u、-g、-G 一起使用，显示名称而不是数字
- -Z: 显示安全上下文 (SELinux)
- -r: 显示实际 ID 而不是有效 ID
- -a: 显示所有信息

使用说明: - id 命令不需要 root 权限，可以被任何用户执行 - 执行 id 命令会显示用户的完整身份信息 - 使用不同的选项可以获取不同类型的身份信息 - id 命令常用于脚本中，获取用户的 UID 或 GID - 显示的 GID 中，第一个是用户的主组 GID，后面的是附加组 GID

注意事项:

- 如果指定的用户不存在，id 命令会返回错误信息
- 在 SELinux 启用的系统中，id -Z 选项会显示安全上下文
- 实际 ID（real ID）是用户登录时的 ID，有效 ID（effective ID）是当前进程的 ID
- id 命令的输出格式可能会因系统版本不同而略有差异
- 对于系统管理任务，id 命令是一个快速获取用户身份信息的有用工具

2.14 whoami 命令

whoami 命令用于显示当前登录用户的用户名。该命令是一个简单但实用的工具，常用于脚本和命令行中，快速确认当前执行命令的用户身份。

whoami 命令的功能：

- 显示当前登录用户的用户名
- 不需要任何参数
- 执行速度快，输出简洁

显示当前用户的用户名

```
whoami
```

whoami 命令的常用选项：whoami 命令没有常用的选项，直接执行即可显示当前用户的用户名。

使用说明： - whoami 命令不需要 root 权限，可以被任何用户执行 - 执行 whoami 命令会立即显示当前登录用户的用户名 - whoami 命令常用于脚本中，获取当前执行脚本的用户身份 - 与 id 命令不同，whoami 命令只显示用户名，不显示 UID、GID 等详细信息 - whoami 命令的输出格式非常简洁，只包含用户名

注意事项：

- whoami 命令总是显示当前有效的用户身份
- 在使用 su 命令切换用户后，whoami 命令会显示切换后的用户
- whoami 命令是一个标准的 Unix/Linux 命令，在所有系统中都可用
- 对于简单的用户身份确认，whoami 命令比 id 命令更快捷

2.15 newgrp 命令

`newgrp` 命令用于切换用户的当前组身份。该命令可以将用户的当前组切换到指定的组，使用户获得该组的权限。`newgrp` 命令是管理用户组权限的重要工具。

`newgrp` 命令的功能：

- 切换用户的当前组身份
- 启动一个新的 shell 会话
- 验证组密码（如果设置）
- 临时获得其他组的权限

切换到指定组

```
newgrp groupname
```

切换到指定组并保留当前环境

```
newgrp - groupname
```

切换到主组

```
newgrp
```

`newgrp` 命令的常用选项：

- `-:`：切换组时，保留当前的环境变量
- `-c`：执行指定的命令后退出
- `-l`：与`-`选项相同，保留当前环境

使用说明： - `newgrp` 命令需要用户是指定组的成员 - 如果组设置了密码，需要输入组密码才能切换 - 切换组后，会启动一个新的 shell 会话 - 在新的 shell 会话中，用户的有效 GID 会被设置为指定组的 GID - 退出新的 shell 会话后，会返回原来的组身份

注意事项：

- 切换到其他组需要用户是该组的成员
- 如果组设置了密码，应确保组密码的安全性
- 切换组后，会启动一个新的 shell，需要使用 `exit` 命令退出
- 对于临时需要其他组权限的情况，`newgrp` 命令非常有用
- 频繁切换组可能会导致 shell 层级过多，应注意管理

2.16 passwd 命令

passwd 命令用于设置或修改用户密码。该命令可以修改当前用户的密码，也可以修改其他用户的密码（需要 root 权限）。passwd 命令还会检查密码的强度，确保密码符合系统的安全策略。

passwd 命令的功能：

- 设置用户密码
- 修改用户密码
- 锁定用户账户
- 解锁用户账户
- 设置密码过期时间
- 显示密码状态

修改当前用户的密码

```
passwd
```

修改指定用户的密码（需要root权限）

```
passwd username
```

锁定用户账户

```
passwd -l username
```

解锁用户账户

```
passwd -u username
```

删除用户密码（使账户无密码）

```
passwd -d username
```

设置密码过期时间

```
passwd -x 30 username
```

显示密码状态

```
passwd -S username
```

强制用户下次登录时修改密码

```
passwd -e username
```

强制更新密码（即使密码过期）

```
passwd -f username
```

passwd 命令的常用选项：

- -l: 锁定用户账户
- -u: 解锁用户账户
- -d: 删除用户密码
- -e: 设置密码过期时间，强制用户下次登录时修改密码
- -n: 设置密码的最小使用天数
- -x: 设置密码的最大使用天数
- -w: 设置密码过期前的警告天数
- -i: 设置密码过期后的账户禁用天数
- -S: 显示密码状态
- -k: 仅当密码过期时才更新密码
- -f: 强制操作，在密码过期时强制更新密码
- -a: 显示所有用户的密码状态
- -stdin: 从标准输入读取密码

使用说明： - 普通用户只能修改自己的密码 - 修改其他用户的密码需要 root 权限 - 修改密码时需要输入当前密码进行验证 - 新密码需要输入两次进行确认 - 密码不会在屏幕上显示 - 系统会检查密码的强度，确保密码符合安全策略 - 锁定账户后，用户无法登录，但可以使用 SSH 密钥等其他方式 - 解锁账户后，用户可以正常登录 - 使用 passwd 命令锁定账户时，密码字段前面会加上"!!" - 使用 usermod 命令锁定账户时，密码字段前面会加上"!" - 可以使用 grep 命令查看/etc/shadow 文件来确认账户是否被锁定 - 也可以通过修改/etc/passwd 或/etc/shadow 文件，在密码字段的第一个字符前加上"*" 来锁定账户 - 在需要恢复时，只要删除"*" 即可 - 如果只是禁止用户账户登录系统，可以将其启动 shell 设置为/bin/false 或/dev/null

密码验证与安全： - 普通用户修改口令时，passwd 命令会首先询问原来的口令，只有验证通过才可以修改 - root 用户为用户指定口令时，不需要知道原来的口令 - 为了系

统安全，用户应选择包含字母、数字和特殊符号组合的复杂口令，且口令长度应至少为 8 个字符 - 如果密码复杂度不够，系统会提示”无效的密码：密码未通过字典检查-它基于字典单词” - 处理方法：

- 再次输入刚才输入的简单密码，系统也会接受
- 更改为符合要求的密码，例如，P@ssw02d 包含大小写字母、数字、特殊符号等 8 位字符组合

注意事项：

- 密码应包含大小写字母、数字和特殊字符
- 密码长度应不少于 8 个字符
- 不要使用常见的字典单词作为密码
- 不要使用生日、电话号码等个人信息作为密码
- 定期更换密码，提高账户安全性
- 不要在多个账户中使用相同的密码
- 锁定账户前，应先确认该用户没有重要的进程在运行
- 删除密码后，用户可以无密码登录，这会带来安全风险
- 建议定期检查用户的密码状态，确保账户安全

2.17 chage 命令

chage 命令用于更改用户密码过期信息。该命令可以修改用户密码的过期时间、最小使用天数、最大使用天数、警告天数等密码策略相关的信息。chage 命令需要 root 权限才能修改其他用户的密码过期信息。

chage 命令的功能：

- 修改用户密码的过期时间
- 设置密码的最小使用天数
- 设置密码的最大使用天数
- 设置密码过期前的警告天数
- 设置密码过期后的账户禁用天数
- 显示用户的密码过期信息

- 强制用户下次登录时修改密码

显示用户的密码过期信息

```
chage -l username
```

修改用户密码的最大使用天数

```
chage -M 90 username
```

修改用户密码的最小使用天数

```
chage -m 7 username
```

修改密码过期前的警告天数

```
chage -W 14 username
```

修改密码过期后的账户禁用天数

```
chage -I 7 username
```

修改账户的过期日期

```
chage -E 2026-12-31 username
```

强制用户下次登录时修改密码

```
chage -d 0 username
```

chage 命令的常用选项:

- -l: 显示用户的密码过期信息
- -d: 设置密码的最后修改日期
- -E: 设置账户的过期日期
- -m: 设置密码的最小使用天数
- -M: 设置密码的最大使用天数
- -W: 设置密码过期前的警告天数
- -I: 设置密码过期后的账户禁用天数
- -R: 指定远程系统的名称

使用说明： - chage 命令需要 root 权限才能修改其他用户的密码过期信息 - 普通用户只能查看自己的密码过期信息 - 日期格式可以使用 YYYY-MM-DD 或从 1970 年 1 月 1 日开始的天数 - 设置密码的最后修改日期为 0 (-d 0) 会强制用户下次登录时修改密码 - 显示用户的密码过期信息 (-l 选项) 不需要 root 权限

注意事项：

- 修改密码过期信息前，应先了解当前的密码策略
- 合理设置密码的最大使用天数，提高系统安全性
- 适当设置警告天数，给用户足够的时间修改密码
- 避免设置过长的密码使用期限，定期更换密码有助于提高安全性
- 对于系统账户，应根据实际需求设置合适的密码过期策略

2.18 usermod 命令

usermod 命令用于修改用户账户的属性。该命令可以修改用户的 UID、GID、主目录、登录 shell、注释信息等属性，也可以将用户添加到附加组或从附加组中删除。usermod 命令需要 root 权限才能执行。

usermod 命令的功能：

- 修改用户的 UID
- 修改用户的主组
- 修改用户的附加组
- 修改用户的主目录
- 修改用户的登录 shell
- 修改用户的注释信息
- 修改用户的密码过期时间
- 锁定或解锁用户账户

修改用户的主组

```
usermod -g groupname username
```

修改用户的附加组

```
usermod -G group1,group2 username
```


修改用户的主目录

```
usermod -d /new/home username
```

修改用户的主目录并移动现有文件

```
usermod -d /new/home -m username
```

修改用户的登录shell

```
usermod -s /bin/zsh username
```

修改用户的注释信息

```
usermod -c "New Description" username
```

修改用户的UID

```
usermod -u 1001 username
```

将用户添加到附加组

```
usermod -aG groupname username
```

锁定用户账户

```
usermod -L username
```

解锁用户账户

```
usermod -U username
```

修改用户的密码过期时间

```
usermod -e 2026-12-31 username
```

usermod 命令的常用选项:

- -g: 指定用户的主组
- -G: 指定用户的附加组, 多个组用逗号分隔
- -a: 与-G 选项一起使用, 将用户添加到附加组而不是替换
- -d: 指定用户的主目录
- -m: 与-d 选项一起使用, 移动现有文件到新的主目录
- -s: 指定用户的登录 shell

- -c: 添加或修改用户的注释信息
- -u: 指定用户的 UID
- -L: 锁定用户账户
- -U: 解锁用户账户
- -e: 指定账户的过期日期
- -f: 指定密码过期后多少天账户被禁用
- -l: 修改用户的登录名称
- -p: 指定用户的加密密码

使用说明: - usermod 命令需要 root 权限才能执行 - 修改用户的 UID 时, 应确保新 UID 未被其他用户使用 - 修改用户的主目录时, 建议使用 -m 选项移动现有文件 - 将用户添加到附加组时, 应使用 -aG 选项, 否则会替换现有附加组 - 锁定用户账户后, 用户无法登录, 但可以使用 SSH 密钥等其他方式 - 解锁用户账户后, 用户可以正常登录 - 使用 usermod 命令锁定账户时, 密码字段前面会加上"! " - 使用 passwd 命令锁定账户时, 密码字段前面会加上"! " - 可以使用 grep 命令查看 /etc/shadow 文件来确认账户是否被锁定 - 也可以通过修改 /etc/passwd 或 /etc/shadow 文件, 在密码字段的第一个字符前加上"! " 来锁定账户 - 在需要恢复时, 只要删除"! " 即可 - 如果只是禁止用户账户登录系统, 可以将其启动 shell 设置为 /bin/false 或 /dev/null

注意事项:

- 修改用户的 UID 可能会影响文件的所有权
- 修改用户的主目录时, 应确保新目录存在且有适当的权限
- 不要在用户登录时修改其 UID 或主目录
- 修改用户的登录 shell 时, 应确保指定的 shell 存在
- 锁定用户账户前, 应先确认该用户没有重要的进程在运行
- 修改用户的附加组时, 应避免删除用户的必要组

2.19 who 命令

who 命令用于查看当前登录主机的用户终端信息。可以快速显示出所有正在登录本机的用户名称以及他们正在开启的终端信息。

who 命令的功能:

- 显示当前登录的用户
- 显示用户登录的终端
- 显示用户登录的时间
- 显示用户登录的来源 IP 或主机名
- 显示用户的空闲时间

显示当前登录的所有用户

`who`

显示当前登录用户的详细信息

`who -a`

显示用户登录的终端和登录时间

`who -b`

显示当前登录用户的数量

`who -q`

显示用户登录的来源IP或主机名

`who -H`

显示用户的空闲时间

`who -i`

`who` 命令的常用选项:

- `-a`: 显示所有信息，包括空闲时间和进程 ID
- `-b`: 显示系统最后启动的时间
- `-d`: 显示已死掉的进程
- `-H`: 显示列标题
- `-i`: 显示空闲时间
- `-l`: 显示登录进程
- `-m`: 仅显示当前终端的用户

- -q: 显示当前登录用户的数量
- -r: 显示当前运行级别
- -s: 显示特殊字符
- -T: 显示终端类型
- -u: 显示已登录用户的详细信息

使用说明: - who 命令显示当前登录到系统的所有用户 - who 命令从/var/run/utmp 文件中读取信息 - who 命令可以显示用户的登录时间、终端和来源 - who 命令可以帮助管理员了解当前系统的使用情况

输出字段说明: - LOGIN_NAME: 登录用户名 - TTY: 用户登录的终端 - TIME: 用户登录的时间 - LINE: 终端线路 - ID: 用户 ID - PID: 用户进程 ID - FROM: 用户登录的来源 IP 或主机名 - LOGIN@: 用户登录的时间 - IDLE: 用户的空闲时间

2.20 last 命令

last 命令用于查看本机的登录记录。但是，由于这些信息都是以日志文件的形式保存在系统中的，所以黑客可以很容易地对内容进行篡改。因此，不能单纯以此来判定是否遭黑客攻击。

last 命令的功能：

- 显示用户登录历史记录
- 显示系统重启历史
- 显示登录时间、退出时间
- 显示登录来源 IP 或主机名
- 显示登录持续时间

显示所有登录记录

```
last
```

显示最近10条登录记录

```
last -n 10
```

显示指定用户的登录记录

```
last username
```

显示指定终端的登录记录

```
last tty1
```

显示系统重启记录

```
last reboot
```

显示系统关机记录

```
last shutdown
```

显示完整的主机名

```
last -a
```

显示登录记录的IP地址

```
last -i
```

显示登录记录的详细时间

```
last -F
```

显示登录记录，不显示主机名

```
last -R
```

显示指定时间范围内的登录记录

```
last -t 20240101000000
```

显示指定时间之后的登录记录

```
last -s 20240101
```

显示指定时间之前的登录记录

```
last -t 20240101
```

last 命令的常用选项：

- -n: 显示指定数量的记录
- -a: 显示完整的主机名
- -d: 将 IP 地址解析为主机名
- -i: 显示 IP 地址而不是主机名

- -F: 显示完整的时间格式
- -R: 不显示主机名
- -x: 显示系统关机和重启记录
- -t: 显示指定时间之前的记录
- -s: 显示指定时间之后的记录
- -w: 显示完整格式的用户名和主机名

使用说明: - last 命令从/var/log/wtmp 文件中读取登录记录 - last 命令显示用户的登录时间、退出时间和持续时间 - last 命令可以显示系统重启和关机记录 - last 命令可以帮助管理员了解系统的使用历史

注意事项:

- 登录记录保存在/var/log/wtmp 文件中
- 由于这些信息都是以日志文件的形式保存在系统中的，所以黑客可以很容易地对内容进行篡改
- 因此，不能单纯以此来判定是否遭黑客攻击
- last 命令的记录可能会被管理员或黑客修改
- 建议结合其他安全工具（如 fail2ban、auditd）来检测攻击
- 定期备份登录记录文件，以便在发生安全事件时进行分析

输出字段说明: - USER: 登录用户名 - TTY: 用户登录的终端 - FROM: 用户登录的来源 IP 或主机名 - LOGIN@: 用户登录的时间 - LOGOUT@: 用户退出的时间 - DURATION: 用户登录的持续时间 - START: 会话开始时间 - END: 会话结束时间

与 lastlog 命令的比较:

- last: 显示用户登录历史记录
- lastlog: 显示所有用户的最后登录信息
- last: 从/var/log/wtmp 文件读取
- lastlog: 从/var/log/lastlog 文件读取
- last: 显示每次登录的详细信息
- lastlog: 显示每个用户的最后登录时间

安全建议:

- 定期检查登录记录，发现异常登录行为
- 配置登录失败锁定策略，防止暴力破解
- 使用 SSH 密钥认证，提高安全性
- 禁用 root 用户的远程登录
- 配置防火墙，限制访问来源
- 使用安全增强工具（如 fail2ban、denyhosts）
- 定期备份登录记录文件

第 3 节 文件权限机制

- 权限表示方法（rwx，数字表示法）
- 文件权限与目录权限的区别
- 特殊权限（SUID，SGID，Sticky Bit）
- ACL（访问控制列表）

第五章 Linux 软件包管理

第 1 节 软件包管理概述

- 软件包的概念
- 依赖关系管理
- 主流软件包管理系统

第 2 节 RPM 包管理系统

- RPM 命令基本用法 (rpm)
- YUM 包管理器 (CentOS/RHEL)
- DNF 包管理器 (新一代 YUM)

2.1 rpm 命令

rpm 命令主要用于对 RPM 软件包进行管理。RPM 软件包是 Linux 的各种发行版中应用最为广泛的软件包格式之一。

rpm 命令的常用选项：

- -i: 安装 RPM 软件包
- -e: 卸载 RPM 软件包
- -nodeps: 卸载时不检查依赖性
- -U: 升级 RPM 软件包
- -F: freshen 模式，只升级已安装的软件包
- -v: 详细模式，显示详细的执行过程
- -h: 显示安装进度条
- -q: 查询 RPM 软件包

- -a: 查询所有已安装的软件包
- -l: 列出软件包中的文件
- -p: 对未安装的软件包进行操作
- -i: 与-q 配合使用, 显示软件包的详细信息 (-qi)
- -l: 与-q 配合使用, 列出软件包中的文件 (-ql)
- -f: 与-q 配合使用, 查询文件属于哪个软件包 (-qf)
- -p: 与-q 配合使用, 查询未安装软件包的信息 (-qp)

安装RPM软件包

```
rpm -i package.rpm
```

详细模式安装 (使用-v选项)

```
rpm -iv package.rpm
```

显示进度条安装 (使用-vh选项)

```
rpm -ivh package.rpm
```

升级RPM软件包

```
rpm -U package.rpm
```

详细模式升级 (使用-vh选项)

```
rpm -Uvh package.rpm
```

Freshen模式升级 (只升级已安装的包, 使用-F选项)

```
rpm -Fvh package.rpm
```

卸载RPM软件包 (注意: 卸载时不需要加.rpm扩展名)

```
rpm -e package_name
```

详细模式卸载 (使用-v选项)

```
rpm -ev package_name
```

不检查依赖性卸载 (使用--nodeps选项)

```
rpm -e --nodeps package_name
```

```
# 示例：卸载network-scripts软件包且不检查依赖性
# 注意：软件包名称会因系统版本而稍有差异，不要机械照抄
rpm -e network-scripts-10.00.6-1.el8.x86_64 --nodeps
```

```
# 查询已安装的软件包
rpm -q package_name
```

```
# 查询所有已安装的软件包
rpm -qa
```

```
# 列出软件包中的文件
rpm -ql package_name
```

```
# 查询未安装软件包的信息
rpm -qp package.rpm
```

```
# 显示软件包的详细信息（使用-qi选项）
rpm -qi package_name
```

```
# 列出软件包中的文件（使用-ql选项）
rpm -ql package_name
```

```
# 查询文件属于哪个软件包（使用-qf选项）
rpm -qf /path/to/file
```

```
# 查询未安装软件包的详细信息（使用-qpi选项）
rpm -qpi package.rpm
```

尽管 RPM 命令能够帮助用户查询软件相关的依赖关系，但具体问题还是要运维人员自己来解决。而有些大型软件可能与数十个程序都有依赖关系，在这种情况下安装软件是非常痛苦的。yum 软件仓库便是为了进一步降低软件安装难度和复杂度而设计的软件。

2.2 yum 软件仓库

RHEL 先将发布的软件存放到 yum 服务器内，再分析这些软件的依赖属性问题，将软件内的记录信息写下来，然后将这些信息分析后记录成软件相关的清单列表。这些列表数据与软件所在的位置可以称为容器（repository）。当 Linux 客户端有软件安装的需求时，Linux 客户端主机主动向网络上的 yum 服务器的容器网址请求下载清单列

表，然后通过清单列表的数据与本机 RPM 数据库已存在的软件数据相比较，就能够一次性安装所有需要的具有依赖属性的软件了。

当 Linux 客户端有升级、安装的需求时，会向容器要求更新清单列表，使清单列表更新到本机的 `/var/cache/yum` 中。当 Linux 客户端实施更新、安装时，会用清单列表的数据与本机的 RPM 数据库进行比较，这样就知道该下载什么软件了。接下来会到 yum 服务器下载所需要的软件，然后通过 RPM 的机制开始安装软件。这就是整个流程，仍然离不开 RPM。

RHEL 8 提供了基于 Fedora 28 中 DNF 的包管理系统 yum v4，兼容 RHEL 7 的 yum v3。

2.3 常见 DNF 命令列表

第 3 节 DPKG 包管理系统

- DPKG 命令基本用法 (dpkg)
- APT 包管理器 (Debian/Ubuntu)
- 软件源配置

第 4 节 源码编译安装

- 编译环境准备
- 源码获取与解压
- configure, make, make install 流程
- 软件卸载

功能描述	命令
安装软件	<code>dnf install 软件包名</code> (示例: <code>dnf install firefox</code>)
自动安装软件 (无需确认)	<code>dnf install -y 软件包名</code>
卸载软件	<code>dnf remove 软件包名</code> (示例: <code>dnf remove firefox</code>)
自动卸载软件 (无需确认)	<code>dnf remove -y 软件包名</code>
重新安装软件	<code>dnf reinstall 软件包名</code> (示例: <code>dnf reinstall firefox</code>)
更新软件	<code>dnf update 软件包名</code>
更新所有软件	<code>dnf update</code>
仅更新安全补丁	<code>dnf update --security</code>
检查更新	<code>dnf check-update</code>
搜索软件	<code>dnf search 关键词</code> (示例: <code>dnf search web browser</code>)
查看软件信息	<code>dnf info 软件包名</code> (示例: <code>dnf info firefox</code>)
列出已安装软件	<code>dnf list installed</code>
列出可用软件	<code>dnf list available</code>
列出所有软件包 (包括已安装和可用的)	<code>dnf list all</code>
查看软件依赖	<code>dnf deplist 软件包名</code>
清理缓存	<code>dnf clean all</code>
重新生成缓存	<code>dnf makecache</code>
查看软件组	<code>dnf grouplist</code>
查看软件组详细信息	<code>dnf groupinfo 软件包组</code>
安装软件组	<code>dnf groupinstall 软件组名</code>
卸载软件组	<code>dnf groupremove 软件组名</code>
查看仓库	<code>dnf repolist</code>
查看所有仓库 (包括禁用的)	<code>dnf repolist all</code>
启用仓库	<code>dnf config-manager --set-enabled 仓库名</code>
禁用仓库	<code>dnf config-manager --set-disabled 仓库名</code>

表 5.1: 常见 DNF 命令列表

第六章 Linux 系统管理

第 1 节 进程管理

- 进程的概念与状态
- 进程管理命令：ps, top, htop, pstree
- 进程控制：kill, killall, pkill
- 作业管理：bg, fg, jobs, nohup
- 系统负载查看：uptime, w

1.1 ps 命令

ps 命令主要用于查看系统的进程。它是 Process Status 的缩写，是 Linux 系统中最常用的进程查看工具之一。

ps 命令的功能：

- 查看当前系统中运行的进程
- 查看进程的详细信息，如 PID、PPID、CPU 使用率、内存使用率等
- 查看指定用户的进程
- 查看指定终端的进程
- 查看进程的层级关系

ps 命令的常用选项：

- -a: 显示所有用户的进程
- -u: 显示进程的详细信息（包括用户、CPU、内存等）
- -x: 显示没有控制终端的进程
- -e: 显示所有进程，等同于-A

- -f: 显示完整格式的进程信息
- -l: 显示长格式的进程信息
- -o: 自定义输出格式
- -forest: 以树状结构显示进程间的关系
- -p: 显示指定 PID 的进程
- -t: 显示指定终端的进程，后面可跟终端设备名
- -u: 显示指定用户的进程
- -w: 设置输出宽度，允许更宽的输出显示，可多次使用以增加宽度

显示当前终端的进程

```
ps
```

显示所有进程的详细信息

```
ps aux
```

显示所有进程的完整格式信息

```
ps -ef
```

以树状结构显示进程间的关系

```
ps aux --forest
```

显示指定PID的进程

```
ps -p 1234
```

显示指定用户的进程

```
ps -u username
```

显示没有控制终端的进程

```
ps ax
```

自定义输出格式（PID、用户、命令）

```
ps -eo pid,user,cmd
```

按CPU使用率排序显示进程


```
ps aux --sort=-%cpu
```

按内存使用率排序显示进程

```
ps aux --sort=-%mem
```

显示指定终端的进程并使用宽输出格式

```
ps -w -t pts/0
```

显示所有终端的进程并使用更宽的输出格式（使用多个-w选项）

```
ps -ww -t
```

ps 命令输出字段说明：

- PID: 进程 ID
- PPID: 父进程 ID
- USER: 进程所有者
- %CPU: CPU 使用率
- %MEM: 内存使用率
- VSZ: 虚拟内存大小
- RSS: 常驻内存大小
- TTY: 控制终端
- STAT: 进程状态（R: 运行, S: 睡眠, D: 不可中断睡眠, Z: 僵尸进程, T: 停止）
- START: 进程启动时间
- TIME: 进程占用 CPU 的时间
- COMMAND: 进程命令

使用说明：ps 命令是一个静态查看进程的工具，它只显示命令执行时的进程状态。如果需要实时查看进程状态，可以使用 top 命令。

参数风格说明：ps 命令支持两种参数风格，这是由 Unix 的历史发展造成的：

- **BSD 风格：**参数前面不加 '-'，例如 'ps aux'
- **System V 风格：**参数前面加 '-'，例如 'ps -ef'

历史原因： - BSD 风格源自 Berkeley Software Distribution Unix 的传统 - System V 风格源自 AT&T System V Unix 的传统

ps 命令为了兼容这两种 Unix 分支的用户习惯，同时支持两种参数形式。在实际使用中，两种风格的参数可以混合使用，但为了可读性和一致性，建议选择一种风格并坚持使用。

与管道和重定向配合使用： ps 命令通常和重定向、管道等命令一起使用，用于查找出所需的进程。

使用管道过滤包含特定关键字的进程

```
ps aux | grep ssh
```

使用管道和grep查找特定用户的进程

```
ps aux | grep username
```

使用管道和awk提取特定字段

```
ps aux | awk '{print $1, $2, $11}'
```

使用管道和sort按CPU使用率排序

```
ps aux | sort -nrk 3,3 | head -10
```

使用管道和sort按内存使用率排序

```
ps aux | sort -nrk 4,4 | head -10
```

将ps命令的输出重定向到文件

```
ps aux > process_list.txt
```

使用管道和wc统计进程数量

```
ps aux | wc -l
```

查找特定命令的进程

```
ps aux | grep "nginx"
```

查找并排除grep自身的进程

```
ps aux | grep nginx | grep -v grep
```

这些组合使用方式可以帮助用户更高效地筛选和分析系统中的进程信息，特别是在系统进程数量较多的情况下。

1.2 top 命令

top 命令是一个实时的进程监控工具，它可以动态显示系统中运行的进程及其资源占用情况。与 ps 命令不同，top 命令会持续更新显示内容，默认每 5 秒刷新一次。

top 命令的功能：

- 实时监控系统中运行的进程
- 显示进程的 CPU 使用率、内存使用率等资源占用情况
- 支持按不同字段排序进程
- 支持交互式操作
- 可以调整刷新闻隔

启动top命令，默认每5秒刷新一次

```
top
```

设置刷新闻隔为20秒

```
top -d 20
```

以批处理模式运行，输出一次后退出

```
top -b -n 1
```

只显示指定用户的进程

```
top -u username
```

只显示指定PID的进程

```
top -p 1234
```

top 命令的常用选项：

- -d: 指定刷新闻隔，单位为秒
- -b: 批处理模式，适合重定向到文件
- -n: 指定刷新次数，完成后退出
- -u: 只显示指定用户的进程
- -p: 只显示指定 PID 的进程
- -H: 显示线程信息

- -c: 显示完整的命令行

使用说明: 和 ps 命令不同, top 命令可以实时监控进程的状况。top 命令界面自动每 5s 刷新一次, 也可以用"top -d 20", 使得 top 命令界面每 20s 刷新一次。

在 top 命令界面中, 可以使用以下按键进行交互式操作:

- P: 按 CPU 使用率排序
- M: 按内存使用率排序
- N: 按 PID 排序
- T: 按 CPU 累计时间排序
- k: 终止指定进程
- q: 退出 top 命令

1.3 htop 命令

htop 是一个交互式的进程查看器, 是 top 命令的增强版, 提供了更友好的界面和更多功能。

htop 命令的功能:

- 彩色界面: 使用不同颜色显示不同类型的进程和资源使用情况
- 交互式操作: 支持鼠标点击和键盘导航
- 实时监控: 动态显示系统资源使用情况
- 进程管理: 可以直接在界面中终止、调整优先级等操作
- 树形视图: 可以以树状结构显示进程间的关系
- 多列显示: 可以同时显示多个 CPU 核心的使用情况

启动htop

htop

以树形视图启动

htop -t

只显示指定用户的进程

htop -u username

只显示指定PID的进程

```
htop -p 1234
```

htop 命令的常用选项:

- -t: 以树形视图显示进程
- -u: 只显示指定用户的进程
- -p: 只显示指定 PID 的进程
- -s: 指定排序字段
- -d: 指定刷新闻隔（单位为秒）

安装方法: 在大多数 Linux 发行版中, 可以通过包管理器安装:

Ubuntu/Debian

```
sudo apt install htop
```

CentOS/RHEL

```
sudo yum install htop
```

Fedora

```
sudo dnf install htop
```

Arch Linux

```
sudo pacman -S htop
```

常用快捷键:

- F1: 显示帮助信息
- F2: 进入设置界面
- F3: 搜索进程
- F4: 过滤器设置
- F5: 切换树形视图
- F6: 选择排序字段
- F7: 降低进程优先级（增加 nice 值）
- F8: 提高进程优先级（减少 nice 值）

- F9: 终止进程
- F10: 退出 htop
- 空格: 标记/取消标记进程
- u: 显示指定用户的进程
- P: 按 CPU 使用率排序
- M: 按内存使用率排序
- T: 按时间排序

与 top 命令的对比:

- 界面: htop 提供彩色界面, top 默认是单色
- 交互性: htop 支持鼠标操作, top 主要依赖键盘
- 功能: htop 提供更多内置功能, 如直接调整优先级
- 显示: htop 默认显示所有 CPU 核心的使用情况
- 启动速度: top 启动速度通常比 htop 快

1.4 pidof 命令

pidof 命令用于查询某个指定服务进程的进程号码值 (Process Identifier, PID)。它可以快速查找指定进程名的所有运行实例的 PID。

pidof 命令的功能:

- 查找指定进程名的所有运行实例的 PID
- 支持同时查找多个进程名
- 可以与 kill 命令配合使用, 快速终止指定进程

查找sshd进程的PID

```
pidof sshd
```

查找nginx进程的PID

```
pidof nginx
```

同时查找多个进程的PID

```
pidof sshd nginx
```

与kill命令配合使用，终止指定进程

```
kill $(pidof sshd)
```

强制终止指定进程

```
kill -9 $(pidof nginx)
```

pidof 命令的常用选项：

- -s: 只返回一个 PID
- -c: 只返回当前运行在相同 root 目录下的进程
- -x: 同时查找 shell 脚本的进程
- -o: 排除指定的 PID

只返回一个PID

```
pidof -s sshd
```

只返回当前运行在相同root目录下的进程

```
pidof -c nginx
```

同时查找shell脚本的进程

```
pidof -x script.sh
```

排除指定的PID

```
pidof -o 1234 sshd
```

使用说明：pidof 命令是一个简单但实用的工具，它可以快速获取指定进程的 PID，特别适合与 kill 命令配合使用来终止进程。与 ps 命令相比，pidof 命令更加专注于获取 PID，输出更加简洁。

1.5 kill 命令

kill 命令用于向指定进程发送信号，通常用于终止进程。它是 Linux 系统中最常用的进程控制命令之一。

kill 命令的功能：

- 向指定 PID 的进程发送信号
- 支持发送不同类型的信号

- 可以与其他命令配合使用，如 `pidof`、`ps` 等

常用信号：

- 1 (SIGHUP)：重新加载进程配置
- 2 (SIGINT)：中断进程（相当于 `Ctrl+C`）
- 9 (SIGKILL)：强制终止进程，进程无法捕获此信号
- 15 (SIGTERM)：终止进程（默认信号），进程可以捕获此信号并进行清理
- 18 (SIGCONT)：继续暂停的进程
- 19 (SIGSTOP)：暂停进程

使用默认信号（SIGTERM）终止进程

```
kill 1234
```

强制终止进程（使用SIGKILL信号）

```
kill -9 1234
```

```
kill -KILL 1234
```

重新加载进程配置（使用SIGHUP信号）

```
kill -1 1234
```

```
kill -HUP 1234
```

中断进程（使用SIGINT信号）

```
kill -2 1234
```

```
kill -INT 1234
```

暂停进程（使用SIGSTOP信号）

```
kill -19 1234
```

```
kill -STOP 1234
```

继续暂停的进程（使用SIGCONT信号）

```
kill -18 1234
```

```
kill -CONT 1234
```

与`pidof`命令配合使用，终止指定进程

```
kill $(pidof sshd)
```


与ps和grep配合使用，终止匹配的进程

```
kill $(ps aux | grep "nginx" | grep -v grep | awk '{print $2}')
```

kill 命令的常用选项：

- -l: 列出所有可用的信号
- -s: 指定要发送的信号
- -n: 指定信号的数字编号

列出所有可用的信号

```
kill -l
```

使用-s选项指定信号

```
kill -s TERM 1234
```

使用-n选项指定信号编号

```
kill -n 9 1234
```

注意事项：

- 默认情况下，kill 命令发送 SIGTERM 信号（编号 15），允许进程进行清理操作
- 使用 SIGKILL 信号（编号 9）可以强制终止进程，但进程无法进行清理操作
- 只有进程的所有者或 root 用户才能向进程发送信号
- 某些系统进程可能受到保护，即使是 root 用户也无法终止
- 对于僵尸进程（Z 状态），kill 命令通常无效，因为这些进程已经终止但尚未被父进程回收

1.6 killall 命令

killall 命令用于通过进程名终止所有匹配的进程。与 kill 命令不同，killall 命令不需要指定进程的 PID，而是直接使用进程名。

killall 命令的功能：

- 通过进程名终止所有匹配的进程
- 支持发送不同类型的信号
- 可以指定用户，只终止特定用户的进程

- 支持交互式操作，确认后再终止进程

终止所有名为sshd的进程

```
killall sshd
```

强制终止所有名为nginx的进程

```
killall -9 nginx
```

```
killall -KILL nginx
```

交互式终止所有名为apache2的进程

```
killall -i apache2
```

只终止特定用户的进程

```
killall -u username sshd
```

向所有名为java的进程发送SIGHUP信号（重新加载配置）

```
killall -HUP java
```

终止所有与指定模式匹配的进程（使用正则表达式）

```
killall -r "^java.*"
```

killall 命令的常用选项：

- -i: 交互式操作，在终止进程前确认
- -9: 强制终止进程，使用 SIGKILL 信号
- -u: 只终止指定用户的进程
- -r: 使用正则表达式匹配进程名
- -s: 指定要发送的信号
- -v: 详细模式，显示终止的进程
- -w: 等待所有被终止的进程真正终止

详细模式终止进程

```
killall -v nginx
```

等待进程终止

```
killall -w sshd
```

使用指定信号

```
killall -s TERM apache2
```

使用场景：通常来讲，复杂软件的服务程序会有多个进程协同为用户提供服务，如果逐个结束这些进程会比较麻烦，此时可以使用 killall 命令来批量结束某个服务程序带有的全部进程。

注意事项：

- killall 命令默认区分大小写，进程名必须完全匹配
- 只有进程的所有者或 root 用户才能终止进程
- 使用 killall 命令时要特别小心，避免误终止重要进程
- 在某些系统上，killall 命令可能会终止与进程名部分匹配的进程，因此建议使用精确的进程名

1.7 nice 命令

nice 命令用于调整进程的优先级，通过设置进程的 nice 值来影响 CPU 分配。nice 值的范围通常为-20 到 19，其中-20 表示最高优先级，19 表示最低优先级。默认情况下，新进程的 nice 值为 0。

Linux 操作系统有两个和进程有关的优先级：

- PRI 值：进程实际的优先级，它是由操作系统动态计算的。
- NI 值：进程的 nice 值，由用户通过 nice 命令设置。

NI 值的特性：

- NI 值可以被用户更改
- NI 值越大，优先级越低
- 一般用户只能增大 NI 值（降低优先级），只有超级用户才可以减小 NI 值（提高优先级）
- NI 值被改变后，会影响 PRI 值
- 优先级高的进程被优先运行
- 默认时进程的 NI 值为 0

PRI 值的计算与 NI 值有关。使用“ps -l”命令可以查看这两个优先级值。

nice 命令的功能：

- 调整新启动进程的优先级
- 通过设置 nice 值来影响 CPU 时间分配
- 普通用户只能降低进程优先级（增加 nice 值）
- root 用户可以任意调整进程优先级

以默认优先级启动进程

```
nice -n 0 ./myprogram
```

降低进程优先级（增加nice值）

```
nice -n 10 ./myprogram
```

以更高优先级启动进程（需要root权限）

```
sudo nice -n -5 ./myprogram
```

使用简写形式（-10 表示 nice 值为 10）

```
nice -10 ./myprogram
```

nice 命令的常用选项：

- -n: 指定 nice 值，范围为-20 到 19
- -adjustment=N: 与-n 选项相同，指定 nice 值的调整量

使用--adjustment选项

```
nice --adjustment=5 ./myprogram
```

注意事项：

- 普通用户只能将进程的 nice 值增加（降低优先级），不能减少（提高优先级）
- root 用户可以任意调整进程的 nice 值
- nice 命令只能在启动进程时设置优先级，不能调整已运行进程的优先级
- 要调整已运行进程的优先级，需要使用 renice 命令

1.8 renice 命令

renice 命令是根据进程的进程号来改变进程优先级的。与 nice 命令不同，renice 命令可以调整已运行进程的优先级，而不仅仅是启动新进程时设置优先级。

renice 命令的功能：

- 调整已运行进程的优先级
- 通过进程号（PID）指定要调整的进程
- 可以批量调整多个进程的优先级
- 支持调整指定用户或组的所有进程优先级

调整单个进程的优先级（增加nice值，降低优先级）

```
renice +10 1234
```

调整多个进程的优先级

```
renice +5 1234 5678 9012
```

降低nice值，提高进程优先级（需要root权限）

```
sudo renice -5 1234
```

调整指定用户的所有进程优先级

```
renice +10 -u username
```

调整指定组的所有进程优先级

```
renice +10 -g groupname
```

调整指定终端的所有进程优先级

```
renice +10 -t pts/0
```

renice 命令的常用选项：

- -n: 指定要设置的 nice 值
- -u: 指定用户名，调整该用户的所有进程优先级
- -g: 指定组名，调整该组的所有进程优先级
- -t: 指定终端，调整该终端的所有进程优先级

注意事项：

- 普通用户只能将进程的 nice 值增加（降低优先级），不能减少（提高优先级）
- root 用户可以任意调整进程的 nice 值
- renice 命令会影响进程的 PRI 值，优先级高的进程会被优先运行
- 可以使用“ps -l”命令查看进程的当前优先级

1.9 作业管理命令（jobs、bg、fg）

作业管理命令用于控制和管理 shell 中的作业（后台运行的进程）。在 Linux 系统中，当你在终端中运行命令时，可以通过这些命令来控制作业的运行状态。

6.1.9.1 jobs 命令

jobs 命令用于查看当前 shell 中运行的作业列表，包括作业号、状态和命令。

查看当前shell中的作业

```
jobs
```

查看详细的作业信息（包括进程组ID）

```
jobs -l
```

只显示运行中的作业

```
jobs -r
```

只显示停止的作业

```
jobs -s
```

6.1.9.2 bg 命令

bg 命令用于将前台作业或停止的作业放入后台运行。

将最后一个停止的作业放入后台运行

```
bg
```

将指定作业号的作业放入后台运行

```
bg %1
```

6.1.9.3 fg 命令

fg 命令用于将后台作业切换到前台运行。

将最后一个后台作业切换到前台运行

```
fg
```

将指定作业号的作业切换到前台运行

```
fg %1
```

作业管理操作流程示例：

启动一个后台作业（在命令末尾添加&符号）

```
find . -name "*.txt" -type f | xargs grep "error" &
```

查看作业状态

```
jobs
```

假设作业号为1，将其切换到前台

```
fg %1
```

按下Ctrl+Z暂停作业

```
^Z
```

查看暂停的作业

```
jobs
```

将暂停的作业放入后台继续运行

```
bg %1
```

再次查看作业状态

```
jobs
```

作业状态说明：

- Running：作业正在运行
- Stopped：作业被暂停（通常是通过 Ctrl+Z）
- Done：作业已完成

注意事项：

- 作业管理命令只对当前 shell 中的作业有效
- 当退出 shell 时，后台作业可能会被终止（除非使用 nohup 命令）
- 可以使用 kill 命令终止后台作业，例如：kill

第 2 节 服务管理

- 系统服务概念
- SysV init, Upstart, Systemd
- Systemd 服务管理 (systemctl)
- 主机名管理 (hostnamectl)
- 服务自启动配置
- 日志管理 (journalctl)

2.1 hostnamectl 命令

hostnamectl 是 Linux 系统中用于查看和修改主机名的命令，属于 systemd 工具集的一部分。它提供了一种统一的方式来管理系统的主机名设置，适用于使用 systemd 的现代 Linux 发行版。

查看主机名信息

```
hostnamectl
```

修改静态主机名（需要root权限）

```
sudo hostnamectl set-hostname new-hostname
```

修改Pretty主机名（用于显示的友好名称）

```
sudo hostnamectl set-hostname "My Server" --pretty
```

查看特定信息

```
hostnamectl status
```

```
hostnamectl hostname # 只显示主机名
```

```
hostnamectl kernel # 只显示内核版本
```

hostnamectl 命令的优势在于它会自动更新相关的配置文件，确保主机名的持久化设置，并且不需要重启系统即可生效。相比传统的 hostname 命令，它提供了更全面的主机名管理功能。

2.2 shutdown 命令

shutdown 命令用于在指定时间关闭系统。它是 Linux 系统中安全关机和重启的标准命令，可以通知所有登录用户系统即将关闭，并给予他们保存工作的时间。

shutdown 命令的功能：

- 安全关闭系统
- 安全重启系统
- 在指定时间执行关机或重启操作
- 向所有登录用户发送警告消息
- 取消已计划的关机操作

立即关闭系统

```
shutdown now
```

立即重启系统

```
shutdown -r now
```

在10分钟后关闭系统

```
shutdown +10
```

在指定时间关闭系统（24小时制）

```
shutdown 22:00
```

在指定时间重启系统

```
shutdown -r 22:00
```

关闭系统并发送警告消息

```
shutdown +15 "系统将在15分钟后关闭，请保存您的工作"
```

取消已计划的关机操作

```
shutdown -c
```

强制关闭系统（不发送警告）

```
shutdown -h now
```

shutdown 命令的常用选项：

- **-h**: 关闭系统 (halt), 默认选项
- **-r**: 重启系统 (reboot)
- **-H**: 停止系统 (halt), 不关闭电源
- **-P**: 关闭系统并切断电源 (poweroff)
- **-c**: 取消已计划的关机操作
- **-k**: 只发送警告消息, 不实际关机
- **-t**: 指定在发送警告后等待的时间 (秒)
- **-f**: 强制关机, 不调用 init 程序
- **-F**: 强制关机, 不检查文件系统
- **-no-wall**: 不发送警告消息给登录用户

时间格式说明: shutdown 命令支持多种时间格式:

- **now**: 立即执行
- **+m**: 在 m 分钟后执行 (如 +10 表示 10 分钟后)
- **HH:MM**: 在指定时间执行 (24 小时制, 如 22:00 表示晚上 10 点)
- **HH:MM:SS**: 在指定时间执行 (包含秒数)

使用说明: - shutdown 命令会向所有登录用户发送警告消息, 告知系统即将关闭 - 系统管理员可以指定关机原因或警告消息 - 使用 shutdown -c 命令可以取消已计划的关机操作 - shutdown 命令会正确关闭所有服务和进程, 确保数据安全

注意事项:

- 只有 root 用户才能执行 shutdown 命令
- 关机前应通知所有登录用户保存工作
- 在生产环境中, 建议在非高峰时段执行关机操作
- 使用 shutdown 命令比直接使用 poweroff 或 halt 命令更安全
- 如果系统有多个用户登录, 建议使用 shutdown 命令而不是 poweroff

与其他关机命令的比较:

- **shutdown**: 最安全的关机命令, 会通知用户并等待指定时间

- **poweroff**: 立即关闭系统并切断电源
- **halt**: 停止系统，但不一定切断电源
- **reboot**: 立即重启系统
- **systemctl poweroff**: systemd 的关机命令
- **systemctl reboot**: systemd 的重启命令

2.3 halt 命令

halt 命令用于立即停止系统，但该命令不自动关闭电源，需要手动关闭电源。它是 Linux 系统中停止系统的传统命令，通常在系统维护或调试时使用。

halt 命令的功能：

- 立即停止系统运行
- 停止所有进程
- 同步文件系统
- 不自动关闭电源（需要手动操作）

立即停止系统

```
halt
```

停止系统并关闭电源

```
halt -p
```

强制停止系统

```
halt -f
```

停止系统前将内存内容写入磁盘

```
halt -w
```

停止系统并显示详细信息

```
halt -v
```

halt 命令的常用选项：

- **-p**: 停止系统并关闭电源（poweroff）
- **-f**: 强制停止系统，不调用 init 程序

- **-w**: 不实际停止系统，只将内存内容写入磁盘
- **-d**: 不将 wtmp 记录写入 /var/log/wtmp
- **-n**: 停止前不同步文件系统
- **-i**: 停止前关闭所有网络接口
- **-v**: 显示详细信息

使用说明: - **halt** 命令会立即停止系统，不会向登录用户发送警告 - 默认情况下，**halt** 命令不会自动关闭电源，需要手动按电源按钮 - 使用 **halt -p** 命令可以在停止系统后自动关闭电源 - **halt** 命令会尝试同步文件系统，确保数据安全

注意事项:

- 只有 root 用户才能执行 **halt** 命令
- **halt** 命令不会向登录用户发送警告，应确保没有其他用户在使用系统
- 在生产环境中，建议使用 **shutdown** 命令而不是 **halt** 命令
- 使用 **halt** 命令前应确保所有重要数据已保存
- **halt** 命令不会关闭电源，需要手动关闭系统电源

halt、poweroff 与 shutdown 的区别:

- **halt**: 停止系统，不自动关闭电源
- **poweroff**: 停止系统并自动关闭电源
- **shutdown**: 最安全的关机命令，会通知用户并等待指定时间
- **halt -p**: 等同于 **poweroff** 命令
- **shutdown -h now**: 等同于 **halt** 命令
- **shutdown -P now**: 等同于 **poweroff** 命令

历史背景: **halt** 命令是传统的 Unix/Linux 关机命令，在现代系统中，它通常是 **systemctl halt** 或 **shutdown -H** 的别名。在 **systemd** 系统中，**halt** 命令会调用 **systemd** 的 **halt.target**。

2.4 reboot 命令

reboot 命令用于重新启动系统，相当于“shutdown -r now”。它是 Linux 系统中重启系统的标准命令，可以安全地重启系统。

reboot 命令的功能：

- 立即重新启动系统
- 停止所有进程
- 同步文件系统
- 重新启动系统内核

立即重新启动系统

```
reboot
```

强制重新启动系统

```
reboot -f
```

重新启动系统并显示详细信息

```
reboot -v
```

重新启动系统并记录到/var/log/wtmp

```
reboot -w
```

reboot 命令的常用选项：

- -f: 强制重启系统，不调用 init 程序
- -p: 在重启前关闭电源（某些硬件支持）
- -w: 不实际重启系统，只将重启记录写入/var/log/wtmp
- -d: 不将 wtmp 记录写入/var/log/wtmp
- -n: 重启前不同步文件系统
- -i: 重启前关闭所有网络接口
- -v: 显示详细信息

使用说明： - reboot 命令会立即重启系统，不会向登录用户发送警告 - reboot 命令会尝试同步文件系统，确保数据安全 - reboot 命令等同于“shutdown -r now” - 在 systemd 系统中，reboot 命令会调用 systemd 的 reboot.target

注意事项：

- 只有 root 用户才能执行 reboot 命令
- reboot 命令不会向登录用户发送警告，应确保没有其他用户在使用系统
- 在生产环境中，建议使用 shutdown -r 命令而不是 reboot 命令
- 使用 reboot 命令前应确保所有重要数据已保存
- 重启前应通知所有登录用户系统即将重启

reboot 与其他重启命令的比较：

- **reboot**：立即重启系统
- **shutdown -r now**：立即重启系统（推荐使用）
- **shutdown -r +10**：10 分钟后重启系统
- **systemctl reboot**：systemd 的重启命令
- **init 6**：传统的重启命令（SysV init 系统）
- **telinit 6**：init 命令的符号链接

历史背景：reboot 命令是传统的 Unix/Linux 重启命令，在现代系统中，它通常是 systemctl reboot 或 shutdown -r 的别名。在 systemd 系统中，reboot 命令会调用 systemd 的 reboot.target。

与 shutdown 命令的关系：reboot 命令实际上等同于“shutdown -r now”，但 shutdown 命令提供了更多功能，如：- 可以指定重启时间 - 可以向登录用户发送警告消息 - 可以取消已计划的重启操作 - 更适合在生产环境中使用

2.5 poweroff 命令

poweroff 命令用于立即停止系统，并关闭电源，相当于“shutdown -h now”。它是 Linux 系统中关闭系统并切断电源的标准命令。

poweroff 命令的功能：

- 立即停止系统
- 停止所有进程
- 同步文件系统
- 自动关闭系统电源

立即停止系统并关闭电源

```
poweroff
```

强制停止系统并关闭电源

```
poweroff -f
```

停止系统并关闭电源，显示详细信息

```
poweroff -v
```

停止系统并关闭电源，记录到/var/log/wtmp

```
poweroff -w
```

poweroff 命令的常用选项：

- -f: 强制关机，不调用 init 程序
- -w: 不实际关机，只将关机记录写入/var/log/wtmp
- -d: 不将 wtmp 记录写入/var/log/wtmp
- -n: 关机前不同步文件系统
- -i: 关机前关闭所有网络接口
- -v: 显示详细信息

使用说明： - poweroff 命令会立即停止系统并关闭电源，不会向登录用户发送警告
- poweroff 命令等同于“shutdown -h now” - poweroff 命令会尝试同步文件系统，确保数据安全
- 在 systemd 系统中，poweroff 命令会调用 systemd 的 poweroff.target

注意事项：

- 只有 root 用户才能执行 poweroff 命令
- poweroff 命令不会向登录用户发送警告，应确保没有其他用户在使用系统
- 在生产环境中，建议使用 shutdown 命令而不是 poweroff 命令
- 使用 poweroff 命令前应确保所有重要数据已保存
- 关机前应通知所有登录用户系统即将关闭

poweroff 与其他关机命令的比较：

- **poweroff:** 立即停止系统并关闭电源

- **halt**: 停止系统，不自动关闭电源
- **halt -p**: 等同于 poweroff 命令
- **shutdown -h now**: 立即停止系统（等同于 halt）
- **shutdown -P now**: 立即停止系统并关闭电源（等同于 poweroff）
- **shutdown -H now**: 停止系统，不关闭电源（等同于 halt）
- **systemctl poweroff**: systemd 的关机命令

历史背景: poweroff 命令是传统的 Unix/Linux 关机命令，在现代系统中，它通常是 systemctl poweroff 或 shutdown -P 的别名。在 systemd 系统中，poweroff 命令会调用 systemd 的 poweroff.target。

与 halt 命令的区别: - **poweroff**: 停止系统并自动关闭电源 - **halt**: 停止系统，但不自动关闭电源（需要手动按电源按钮） - **halt -p**: 等同于 poweroff 命令

与 shutdown 命令的关系: poweroff 命令实际上等同于“shutdown -P now”，但 shutdown 命令提供了更多功能，如：- 可以指定关机时间 - 可以向登录用户发送警告消息 - 可以取消已计划的关机操作 - 更适合在生产环境中使用

第 3 节 磁盘管理

- 磁盘分区: fdisk, gdisk
- 文件系统创建: mkfs
- 磁盘挂载: mount, umount, /etc/fstab 配置
- 磁盘使用情况查看: df, du
- 磁盘性能测试: dd, hdparm
- 逻辑卷管理 (LVM)

第 4 节 系统监控与性能优化

- 系统资源监控: vmstat, iostat, sar
- 内存管理: free, top
- 网络监控: netstat, ss
- 系统日志查看: /var/log 目录, dmesg
- 性能瓶颈分析与优化策略

4.1 free 命令

free 命令主要用来查看系统内存、虚拟内存的大小及占用情况。

free 命令的常用选项：

- -b: 以字节为单位显示
- -k: 以 KB 为单位显示（默认）
- -m: 以 MB 为单位显示
- -g: 以 GB 为单位显示
- -h: 以人类可读的格式显示（自动选择合适的单位）
- -t: 显示总计行
- -s: 持续显示，指定间隔秒数
- -c: 持续显示指定次数后退出
- -V: 显示版本信息

查看内存使用情况（默认KB单位）

```
free
```

以人类可读的格式查看内存使用情况

```
free -h
```

显示总计行

```
free -h -t
```

持续显示内存使用情况（每2秒一次，共10次）

```
free -h -s 2 -c 10
```

以MB单位查看内存使用情况

```
free -m
```

输出说明： - total: 总内存大小 - used: 已使用的内存大小 - free: 空闲内存大小 - shared: 共享内存大小 - buff/cache: 缓冲区和缓存大小 - available: 可用内存大小（考虑了可回收的缓存）

注意事项： - available 值比 free 值更能准确反映系统的实际可用内存 - Linux 系统会使用部分内存作为缓存来提高性能，这是正常现象

4.2 uname 命令

uname 命令用于显示系统信息，包括内核版本、硬件架构、主机名等。它是一个常用的系统信息查询工具。

显示系统内核版本

```
uname
```

显示所有系统信息

```
uname -a
```

显示内核版本

```
uname -r
```

显示硬件架构

```
uname -m
```

显示主机名

```
uname -n
```

显示操作系统名称

```
uname -s
```

显示内核版本号

```
uname -v
```

显示处理器类型

```
uname -p
```

显示硬件平台

```
uname -i
```

uname 命令的常用选项：

- -a: 显示所有系统信息
- -r: 显示内核版本
- -m: 显示硬件架构
- -n: 显示主机名

- -s: 显示操作系统名称
- -v: 显示内核版本号
- -p: 显示处理器类型
- -i: 显示硬件平台

使用说明: `uname` 命令是一个简单但实用的工具，它可以快速获取系统的基本信息，特别适合在脚本中使用。通过不同的选项组合，可以获取系统的各种信息，帮助用户了解系统的配置和状态。

示例输出:

```
# uname -a输出示例
```

```
Linux localhost 5.4.0-100-generic x86_64 x86_64 x86_64 GNU/Linux
```

各字段含义:

Linux: 操作系统名称

localhost: 主机名

5.4.0-100-generic: 内核版本

x86_64: 硬件架构

x86_64: 处理器类型

x86_64: 硬件平台

GNU/Linux: 操作系统类型

4.3 dmesg 命令

`dmesg` 命令用实例名称和物理名称来标识连到系统上的设备。`dmesg` 命令也用于显示系统诊断信息、操作系统版本号、物理内存大小以及其他信息。

`dmesg` 命令的常用选项:

- -c: 显示信息后清除缓冲区
- -T: 显示人类可读的时间戳
- -L: 使用彩色输出
- -l: 限制显示的日志级别（如 `err`, `warn`, `info` 等）
- -k: 只显示内核消息
- -t: 不显示时间戳
- -x: 显示详细的内核消息

查看所有系统消息

```
dmesg
```

查看系统消息并显示人类可读的时间戳

```
dmesg -T
```

只查看错误消息

```
dmesg -l err
```

查看最近的系统消息（结合tail）

```
dmesg | tail -n 50
```

查看特定设备的相关消息

```
dmesg | grep -i usb
```

```
dmesg | grep -i eth
```

实时监控系统消息

```
dmesg -w
```

注意事项: - dmesg 命令输出的信息可能会很多，可以使用管道和 grep 命令来过滤感兴趣的内容 - 对于较新的 Linux 系统，系统日志也可能存储在 /journal 或 /var/log 目录中 - 使用 dmesg -c 会清除内核环缓冲区的内容，谨慎使用

4.4 sosreport 命令

sosreport 命令用于收集系统配置及架构信息并输出诊断文档。它是一个系统诊断工具，可以帮助管理员快速收集系统信息，用于故障排除和问题分析。

sosreport 命令的功能：

- 收集系统配置信息
- 收集系统架构信息
- 收集系统日志信息
- 生成诊断报告
- 支持插件扩展
- 支持自定义配置

生成系统诊断报告

```
sosreport
```

生成诊断报告并指定报告名称

```
sosreport --batch --name=myreport
```

生成诊断报告并指定输出目录

```
sosreport --tmp-dir=/tmp/sos
```

生成诊断报告并启用所有插件

```
sosreport --all-plugins
```

生成诊断报告并禁用特定插件

```
sosreport --disable-plugin=plugin_name
```

生成诊断报告并只启用特定插件

```
sosreport --enable-plugin=plugin_name
```

生成诊断报告并设置压缩级别

```
sosreport --compress-level=9
```

生成诊断报告并设置报告格式

```
sosreport --report-type=html
```

生成诊断报告并显示详细输出

```
sosreport --verbose
```

sosreport 命令的常用选项:

- `-batch`: 批处理模式, 不提示用户输入
- `-name`: 指定报告名称
- `-tmp-dir`: 指定临时目录
- `-all-plugins`: 启用所有插件
- `-enable-plugin`: 启用指定插件
- `-disable-plugin`: 禁用指定插件

- `-list-plugins`: 列出所有可用插件
- `-compress-level`: 设置压缩级别 (0-9)
- `-report-type`: 设置报告格式 (html、json 等)
- `-verbose`: 显示详细输出
- `-quiet`: 安静模式, 不输出信息
- `-help`: 显示帮助信息
- `-version`: 显示版本信息

使用说明: `- sosreport` 命令会收集系统的配置信息、日志文件和状态信息 - 生成的报告会打包成 `tar.gz` 格式 - 报告中包含系统硬件、软件、网络、存储等信息 - `sosreport` 命令需要 `root` 权限才能执行 - 生成的报告可以发送给技术支持人员进行问题分析

报告内容: `sosreport` 生成的诊断报告通常包含以下信息:

- 系统基本信息 (主机名、操作系统版本、内核版本等)
- 硬件信息 (CPU、内存、磁盘、网络设备等)
- 网络配置 (网络接口、路由、DNS 等)
- 存储配置 (磁盘分区、文件系统、LVM 等)
- 系统服务 (运行的服务、服务配置等)
- 系统日志 (系统日志、应用程序日志等)
- 软件包信息 (已安装的软件包、软件源等)
- 安全配置 (防火墙规则、SELinux 状态等)

注意事项:

- `sosreport` 命令需要 `root` 权限才能执行
- 生成的报告可能包含敏感信息, 发送前要进行脱敏处理
- 报告生成可能需要较长时间, 取决于系统配置和数据量
- 在生产环境中, 建议在非高峰时段执行 `sosreport` 命令
- 生成的报告文件较大, 可能需要足够的磁盘空间
- 报告中可能包含密码、密钥等敏感信息, 要谨慎处理

常用场景：

- **故障排除：**系统出现问题时，使用 `sosreport` 收集诊断信息
- **技术支持：**向技术支持人员提供系统诊断报告
- **系统审计：**定期生成报告用于系统审计和合规检查
- **问题分析：**分析系统配置和日志，找出问题根源
- **迁移前检查：**系统迁移前，使用 `sosreport` 记录系统状态

与类似工具的比较：

- **sosreport：**专注于系统诊断，信息全面
- **systemd-analyze：**分析 `systemd` 启动性能
- **perf：**系统性能分析工具
- **strace：**跟踪系统调用
- **lsof：**列出打开的文件

安装方法：在大多数 Linux 发行版中，`sosreport` 命令可以通过包管理器安装：

```
# RHEL/CentOS/Fedora
sudo dnf install sos
```

```
# Ubuntu/Debian
sudo apt install sosreport
```

```
# Arch Linux
sudo pacman -S sos
```

报告位置：生成的诊断报告默认保存在以下位置：

- **RHEL/CentOS：** `/var/tmp/sosreport-<hostname>-<date>-<id>/`
- **Ubuntu/Debian：** `/tmp/sosreport-<hostname>-<date>-<id>/`
- **Arch Linux：** `/tmp/sosreport-<hostname>-<date>-<id>/`

安全建议：

- 发送报告前要检查并删除敏感信息

- 使用加密方式传输报告
- 定期删除旧的报告文件
- 限制报告的访问权限
- 记录报告的发送和接收情况

第七章 Linux 网络配置与管理

第 1 节 网络基础概念

- TCP/IP 协议栈
- IP 地址与子网掩码
- 网关与 DNS
- 端口与服务

第 2 节 网络配置

网络配置通常包括主机名、IP 地址、子网掩码、默认网关、DNS 服务器等的设置。

- 主机名配置
- 网络接口配置 (ifconfig, ip 命令)
- 静态 IP 配置
- DHCP 配置
- 路由配置 (route, ip route)
- DNS 配置 (/etc/resolv.conf)

2.1 主机名配置

RHEL 8 有以下 3 种形式的主机名：

1. 静态(static)主机名:静态主机名也称为内核主机名,是系统在启动时从/etc/hostname 自动初始化的主机名。
2. 瞬态(transient)主机名:瞬态主机名是在系统运行时临时分配的主机名,由内核管理。例如,通过 DHCP 或 DNS 服务器分配的 localhost 就是瞬态主机名。

3. 灵活 (pretty) 主机名: 灵活主机名是 UTF8 格式的自由主机名, 以展示给终端用户。

使用 nmtui 修改主机名。使用 NetworkManager 的 nmtui 接口修改静态主机名后 (/etc/hostname 文件), 不会通知 hostnamectl。要想强制让 hostnamectl 知道静态主机名已经被修改, 需要重启 hostnamed 服务:

```
systemctl restart systemd-hostnamed
```

使用 hostnamectl 修改主机名:

查看当前主机名

```
hostnamectl
```

修改静态主机名

```
hostnamectl set-hostname new-hostname
```

修改灵活主机名

```
hostnamectl set-hostname "New Hostname" --pretty
```

修改瞬态主机名

```
hostnamectl set-hostname transient-hostname --transient
```

重启 hostnamed 服务, 让 hostnamectl 知道静态主机名已经被修改:

```
systemctl restart systemd-hostnamed
```

使用 NetworkManager 的命令行接口 nmcli 修改主机名:

查看当前主机名

```
nmcli general hostname
```

修改主机名

```
nmcli general hostname new-hostname
```

2.2 使用 nmtui 和 nmcli 设置网络

7.2.2.1 nmtui 设置网络

nmtui 是 NetworkManager 的文本用户界面工具, 可以通过交互式方式设置网络配置。

```
# 启动nmtui
```

```
nmtui
```

在 nmtui 界面中，可以进行以下操作：

- 编辑连接：修改现有网络连接的配置
- 激活连接：启用或禁用网络连接
- 设置系统主机名：修改系统主机名

7.2.2.2 nmcli 设置网络

nmcli 是 NetworkManager 的命令行接口，可以通过命令行方式设置网络配置。

```
# 查看网络连接
```

```
nmcli connection show
```

```
# 查看网络设备
```

```
nmcli device status
```

```
# 创建新的以太网连接
```

```
nmcli connection add con-name eth0 type ethernet ifname eth0
```

```
# 配置静态IP地址
```

```
nmcli connection modify eth0 ipv4.method manual ipv4.addresses 192.168.1.100/24 ipv4.
```

```
# 配置DHCP
```

```
nmcli connection modify eth0 ipv4.method auto
```

```
# 激活连接
```

```
nmcli connection up eth0
```

```
# 禁用连接
```

```
nmcli connection down eth0
```

```
# 删除连接
```

```
nmcli connection delete eth0
```

```
# 查看连接详细信息
```

```
nmcli connection show eth0
```

创建无线连接

```
nmcli connection add con-name wlan0 type wifi ifname wlan0 ssid "WiFi名称" password "
```

配置IPv6地址

```
nmcli connection modify eth0 ipv6.method manual ipv6.addresses 2001:db8::1/64 ipv6.ga
```

配置多个DNS服务器

```
nmcli connection modify eth0 ipv4.dns "8.8.8.8 8.8.4.4"
```

修改连接名称

```
nmcli connection modify eth0 con-name eth0-new
```

重启NetworkManager服务

```
sudo systemctl restart NetworkManager
```

2.3 传统网络配置文件目录

在 RHEL/CentOS 7 及之前的版本中,网络配置文件存储在 ‘/etc/sysconfig/network-scripts/’ 目录中。每个网络接口对应一个配置文件,命名格式为 ‘ifcfg-接口名称’。

查看网络配置文件

```
ls /etc/sysconfig/network-scripts/ifcfg-*
```

典型的网络配置文件示例 (ifcfg-eth0):

```
TYPE=Ethernet
PROXY_METHOD=none
BROWSER_ONLY=no
BOOTPROTO=none
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
IPV6_ADDR_GEN_MODE=stable-privacy
NAME=eth0
```

```
UUID=5fb06bd0-0bb0-7ffb-45f1-d6edd65f3e03
DEVICE=eth0
ONBOOT=yes
IPADDR=192.168.1.100
PREFIX=24
GATEWAY=192.168.1.1
DNS1=8.8.8.8
DNS2=8.8.4.4
IPV6_PRIVACY=no
```

在 RHEL 8 及之后的版本中，默认使用 NetworkManager 的 keyfile 格式存储网络配置，配置文件位于 ‘/etc/NetworkManager/system-connections/’ 目录。但传统的 ‘/etc/sysconfig/network-scripts/’ 目录仍然被支持，用于兼容旧的配置。

查看新的网络配置文件

```
ls /etc/NetworkManager/system-connections/
```

第 3 节 网络服务

- SSH 服务（远程登录）
- FTP/SFTP 服务（文件传输）
- Web 服务（Apache, Nginx）
- DNS 服务（BIND）
- 邮件服务（Postfix, Dovecot）

第 4 节 防火墙配置

- iptables 基础
- firewalld 服务（新一代防火墙）
- 端口开放与访问控制

4.1 firewalld 服务详解

RHEL 8 集成了多款防火墙管理工具，其中 firewalld 提供了支持在网络/防火墙区域（zone）定义网络连接以及接口安全等级的动态防火墙管理工具—Linux 操作系统的动态防火墙管理器（Dynamic Firewall Manager of Linux Systems）。Linux 操作系统的

动态防火墙管理器拥有基于命令行界面（Command Line Interface, CLI）和基于图形用户界面（Graphical User Interface, GUI）的两种管理方式。

相较于传统的防火墙管理配置工具，firewalld 支持动态更新技术，并加入了区域的概念。简单来说，区域就是 firewalld 预先准备了几套防火墙策略集合（策略模板），用户可以根据生产场景的不同选择合适的策略集合，从而实现防火墙策略之间的快速切换。

例如，我们有一台笔记本电脑，每天都要在办公室、咖啡厅和家里使用。按常理来讲，这三者的安全性按照由高到低的顺序排列，应该是家里、办公室、咖啡厅。当前，我们希望为这台笔记本电脑指定如下防火墙策略：在家中允许访问所有服务；在办公室内仅允许访问文件共享服务；在咖啡厅仅允许上网浏览。以往，我们需要频繁地手动设置防火墙策略，而现在只需要预设好区域集合，然后轻点鼠标就可以自动切换了，从而极大地提升了防火墙策略的应用效率。

7.4.1.1 常见的区域名称及默认策略

firewalld 提供了以下常见的区域，每个区域都有其默认的防火墙策略：

1. **drop（丢弃）**：默认策略是丢弃所有传入的连接，不响应任何请求，只允许传出的连接。适用于最高安全性的场景，如完全不可信的网络。
2. **block（阻塞）**：默认策略是拒绝所有传入的连接，响应 ICMP 错误消息，只允许传出的连接。适用于需要明确拒绝连接的场景。
3. **public（公共）**：默认策略是只允许选定的传入连接，适用于公共网络。这是默认的区域。
4. **external（外部）**：默认策略是只允许选定的传入连接，适用于作为 NAT 网关的外部网络接口。
5. **dmz（非军事区）**：默认策略是只允许选定的传入连接，适用于放置公开服务的非军事区。
6. **work（工作）**：默认策略是只允许选定的传入连接，适用于工作网络。
7. **home（家庭）**：默认策略是只允许选定的传入连接，适用于家庭网络。
8. **internal（内部）**：默认策略是只允许选定的传入连接，适用于内部网络。
9. **trusted（信任）**：默认策略是允许所有传入的连接，适用于完全信任的网络。

7.4.1.2 使用 firewall-cmd 命令管理防火墙

firewall-cmd 命令是 firewalld 防火墙配置管理工具的 CLI 版本。它的参数一般都是以“长格式”来提供的，但幸运的是，RHEL 8 系统支持部分命令的参数补齐。

基本语法 firewall-cmd 命令的基本语法如下：

```
firewall-cmd [选项] [命令]
```

配置模式说明 与 Linux 操作系统中其他防火墙策略配置工具一样，使用 firewalld 配置的防火墙策略默认为运行时（Runtime）模式，又称为当前生效模式，而且系统重启后会失效。如果想让配置策略一直存在，就需要使用永久（Permanent）模式，方法是在用 firewall-cmd 命令正常设置防火墙策略时添加 `--permanent` 参数，这样配置的防火墙策略就可以永久生效了。但是，永久生效模式有一个“不近人情”的特点，就是使用它设置的策略只有在系统重启之后才能自动生效。如果想让配置的策略立即生效，则需要手动执行 `firewall-cmd --reload` 命令。

常用命令示例

- 查看当前默认区域：

```
firewall-cmd --get-default-zone
```

- 查看所有可用区域：

```
firewall-cmd --get-zones
```

- 查看所有可用服务：

```
firewall-cmd --get-services
```

- 查看活动区域：

```
firewall-cmd --get-active-zones
```

- 添加源地址到指定区域：

```
firewall-cmd --zone=public --add-source=192.168.1.0/24 --permanent
```

- 从指定区域移除源地址：

```
firewall-cmd --zone=public --remove-source=192.168.1.0/24 --permanent
```

- 添加网络接口到指定区域:

```
firewall-cmd --zone=public --add-interface=eth0 --permanent
```

- 修改网络接口所属区域:

```
firewall-cmd --zone=work --change-interface=eth0 --permanent
```

- 查看所有区域的规则:

```
firewall-cmd --list-all-zones
```

- 启用应急模式（拒绝所有网络连接）:

```
firewall-cmd --panic-on
```

- 禁用应急模式:

```
firewall-cmd --panic-off
```

- 查看当前区域的所有规则:

```
firewall-cmd --list-all
```

- 查看指定区域的所有规则:

```
firewall-cmd --zone=public --list-all
```

- 永久设置默认区域:

```
firewall-cmd --set-default-zone=public --permanent
```


- 永久添加服务：

```
firewall-cmd --add-service=http --permanent
```

- 永久添加端口：

```
firewall-cmd --add-port=8080/tcp --permanent
```

- 永久添加富规则（允许特定 IP 访问特定端口）：

```
firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0
```

- 重新加载防火墙规则：

```
firewall-cmd --reload
```

- 查看已开放的服务：

```
firewall-cmd --list-services
```

- 查看已开放的端口：

```
firewall-cmd --list-ports
```

常用参数说明

- --zone: 指定操作的区域
- --permanent: 永久生效，需要重新加载
- --add-service: 添加服务
- --remove-service: 移除服务
- --add-port: 添加端口
- --remove-port: 移除端口

- `--add-rich-rule`: 添加富规则
- `--remove-rich-rule`: 移除富规则
- `--reload`: 重新加载规则
- `--panic-on`: 启用应急模式（拒绝所有网络连接）
- `--panic-off`: 禁用应急模式
- `--change-interface=`: 修改网络接口所属区域
- `--list-all-zones`: 查看所有区域的规则
- `--get-default-zone`: 获取默认区域
- `--set-default-zone`: 设置默认区域
- `--get-zones`: 查看所有可用区域
- `--get-active-zones`: 查看活动区域
- `--get-services`: 查看所有可用服务
- `--add-source=`: 添加源地址到指定区域
- `--remove-source=`: 从指定区域移除源地址
- `--add-interface=`: 添加网络接口到指定区域
- `--remove-interface=`: 从指定区域移除网络接口
- `--list-all`: 查看所有规则
- `--list-services`: 查看已开放的服务
- `--list-ports`: 查看已开放的端口

实际使用场景 1. 开放 Web 服务:

永久开放 HTTP 和 HTTPS 服务

```
firewall-cmd --add-service=http --permanent  
firewall-cmd --add-service=https --permanent  
firewall-cmd --reload
```

2. 开放自定义端口:

永久开放 8080 端口 (TCP)

```
firewall-cmd --add-port=8080/tcp --permanent
```

```
firewall-cmd --reload
```

3. 限制特定 IP 访问:

只允许 192.168.1.0/24 网段访问 SSH 服务

```
firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" serv
```

```
firewall-cmd --reload
```

4. 切换区域:

将 eth0 接口切换到 work 区域

```
firewall-cmd --zone=work --change-interface=eth0 --permanent
```

```
firewall-cmd --reload
```

NAT 功能 RHEL 8 的防火墙 (firewall) 利用 nat 表能够实现 NAT 功能, 将内网地址与外网地址进行转换, 完成内、外网的通信。nat 表支持以下 3 种操作。

- **SNAT:** 改变数据包的源地址。防火墙会使用外部地址替换数据包的本地网络地址。这样使网络内部主机能够与网络外部通信。
- **DNAT:** 改变数据包的目的地址。防火墙接收到数据包后, 会替换该包的目的地址, 重新转发到网络内部主机。当应用服务器处于网络内部时, 防火墙接收到外部请求, 会按照规则设定, 将访问重定向到指定的主机上, 使外部主机能够正常访问网络内部主机。
- **MASQUERADE:** MASQUERADE 的作用与 SNAT 完全一样, 改变数据包的源地址。因为对每个匹配的包, MASQUERADE 都要自动查找可用的 IP 地址, 而不像 SNAT 用的 IP 地址是配置好的, 所以会加重防火墙的负担。当然, 如果接入外网的地址不是固定地址, 而是 ISP 随机分配的, 则使用 MASQUERADE 将会非常方便。

使用 firewall-cmd 命令配置 SNAT 示例:

添加 SNAT 规则, 将内网地址 192.168.1.0/24 转换为外网地址 203.0.113.1

```
firewall-cmd --zone=external --add-rich-rule='rule family="ipv4" source address="192.
```

```
firewall-cmd --reload
```

使用 firewall-cmd 命令配置 DNAT 示例:

添加 DNAT 规则, 将外部访问 203.0.113.1:80 转发到内部服务器 192.168.1.10:80

```
firewall-cmd --zone=external --add-rich-rule='rule family="ipv4" destination address=
```

```
firewall-cmd --reload
```

使用 firewall-cmd 命令配置 MASQUERADE 示例：

```
# 添加 MASQUERADE 规则，用于动态 IP 地址的网络
firewall-cmd --zone=external --add-masquerade --permanent
firewall-cmd --reload
```

删除 NAT 配置 要删除已配置的 NAT 规则，可以使用以下命令：

1. **** 删除 SNAT 或 DNAT 规则 ****：使用 `--remove-rich-rule` 参数，与添加时的规则内容相同

删除 SNAT 规则

```
firewall-cmd --zone=external --remove-rich-rule='rule family="ipv4" source address="1
```

删除 DNAT 规则

```
firewall-cmd --zone=external --remove-rich-rule='rule family="ipv4" destination address="1
```

2. **** 删除 MASQUERADE 规则 ****：使用 `--remove-masquerade` 参数

删除 MASQUERADE 规则

```
firewall-cmd --zone=external --remove-masquerade --permanent
```

3. **** 使删除生效 ****：执行 reload 命令

```
firewall-cmd --reload
```

GUI 配置工具 `firewall-config` 命令是 `firewalld` 防火墙配置管理工具的 GUI 版本，几乎可以实现所有以命令行来执行的操作。毫不夸张地说，即使读者没有扎实的 Linux 命令基础，也完全可以通过它来妥善配置 RHEL 8 中的防火墙策略。

`firewall-config` 默认没有安装，需要手动安装：

在 RHEL/CentOS 系统上安装

```
yum install firewalld-config
```

在 Debian/Ubuntu 系统上安装

```
apt install firewall-config
```

安装完成后，可以通过以下命令启动：

```
firewall-config
```

第 5 节 网络工具

- 网络下载工具 (wget, curl)
- 网络测试工具 (ping, traceroute, nslookup)
- 网络监控工具 (netstat, ss, iftop)

5.1 wget 命令

wget 命令用于在终端中下载网络文件。它是一个功能强大的命令行下载工具，支持 HTTP、HTTPS、FTP 等协议，可以递归下载、断点续传等功能。

wget 命令的功能：

- 下载网络文件
- 支持 HTTP、HTTPS、FTP 等协议
- 支持断点续传
- 支持递归下载
- 支持后台下载
- 支持限速下载
- 支持代理服务器

下载单个文件

```
wget http://example.com/file.zip
```

下载文件并指定保存名称

```
wget -O newname.zip http://example.com/file.zip
```

下载文件到指定目录

```
wget -P /path/to/directory http://example.com/file.zip
```

断点续传下载

```
wget -c http://example.com/largefile.iso
```

后台下载

```
wget -b http://example.com/largefile.iso
```

限制下载速度（每秒200KB）

```
wget --limit-rate=200k http://example.com/file.zip
```

递归下载网站

```
wget -r http://example.com/
```

递归下载网站，不创建父目录

```
wget -r -np http://example.com/
```

下载多个文件（从文件列表中读取）

```
wget -i urls.txt
```

下载文件并显示进度条

```
wget --progress=bar http://example.com/file.zip
```

下载文件并重试3次

```
wget -t 3 http://example.com/file.zip
```

下载文件，超时时间为30秒

```
wget -T 30 http://example.com/file.zip
```

模拟浏览器下载

```
wget -U "Mozilla/5.0" http://example.com/file.zip
```

下载文件并使用代理服务器

```
wget -e "http_proxy=http://proxy.example.com:8080" http://example.com/file.zip
```

下载文件并验证证书

```
wget --no-check-certificate https://example.com/file.zip
```

下载文件并保存到指定文件

```
wget -o download.log http://example.com/file.zip
```

wget 命令的常用选项：

- -O：指定下载文件的保存名称
- -P：指定下载文件的保存目录
- -c：断点续传

- -b: 后台下载
- -r: 递归下载
- -np: 不创建父目录
- -l: 设置递归深度
- -t: 设置重试次数
- -T: 设置超时时间（秒）
- -i: 从文件中读取 URL 列表
- -q: 安静模式，不输出信息
- -v: 详细模式，输出详细信息
- -nc: 不覆盖已存在的文件
- -N: 只下载比本地文件新的文件
- --limit-rate: 限制下载速度
- --progress: 设置进度显示方式
- -U: 设置用户代理字符串
- -e: 设置环境变量
- --no-check-certificate: 不验证 SSL 证书
- -o: 将日志信息写入指定文件

使用说明: - wget 命令支持 HTTP、HTTPS、FTP 等多种协议 - wget 命令可以递归下载整个网站 - 使用 -c 选项可以实现断点续传 - 使用 -b 选项可以在后台下载 - 使用 -i 选项可以从文件中读取多个 URL 进行批量下载 - wget 命令会自动处理重定向

注意事项:

- 递归下载时要注意不要下载过多内容，避免对服务器造成压力
- 下载大文件时建议使用 -c 选项以支持断点续传
- 使用 --no-check-certificate 选项可以跳过 SSL 证书验证，但不推荐在生产环境中使用
- 下载敏感信息时要注意保护下载的文件

- 在受限网络环境中，可能需要配置代理服务器

与 curl 命令的比较:

- **wget**: 专注于下载文件，支持递归下载
- **curl**: 功能更全面，支持多种协议和操作
- **wget**: 更适合下载文件和网站
- **curl**: 更适合 API 调用和测试
- **wget**: 默认保存到文件
- **curl**: 默认输出到标准输出

常用场景:

- **下载软件包**: 从官方网站下载软件安装包
- **下载网站**: 递归下载整个网站用于离线浏览
- **批量下载**: 从 URL 列表文件中批量下载多个文件
- **断点续传**: 下载大文件时支持断点续传
- **后台下载**: 在后台下载大文件
- **限速下载**: 限制下载速度以避免占用过多带宽

第八章 Linux 脚本编程

第 1 节 Shell 脚本基础

- Shell 脚本的概念
- 脚本文件格式与执行方式
- 注释与变量
- 输入输出 (echo, read)

1.1 变量的定义和引用

shell 支持具有字符串值的变量。shell 变量不需要专门的说明语句，可通过赋值语句完成变量说明并予以赋值。在命令行或 shell 脚本文件中使用 *name* *name*

变量定义 变量定义的基本语法：

定义变量

```
variable_name=value
```

定义包含空格的变量

```
variable_name="value with spaces"
```

定义包含特殊字符的变量

```
variable_name='value with $special chars'
```

变量引用 变量引用的几种方式：

基本引用方式

```
echo $variable_name
```

推荐使用大括号包围变量名，避免歧义

```
echo ${variable_name}
```

在字符串中引用变量

```
echo "Hello, ${variable_name}!"
```

特殊变量 Shell 中存在一些特殊变量，具有特定的含义：

命令行参数

\$0 # 脚本名称

\$1, \$2, ..., \$n # 脚本参数

\$# # 参数个数

\$* # 所有参数作为一个字符串

@ # 所有参数作为单独的字符串

退出状态

\$? # 上一个命令的退出状态码

进程ID

\$\$ # 当前shell进程ID

环境变量 环境变量是指由 shell 定义和赋初值的 shell 变量。shell 用环境变量来确定查找路径、注册目录、终端类型、终端名称、用户名等。所有环境变量都是全局变量，并可以由用户重新设置。

环境变量是系统级别的变量，对所有进程可见：

查看环境变量

```
env
```

```
printenv
```

设置环境变量

```
export VARIABLE_NAME=value
```

取消环境变量

```
unset VARIABLE_NAME
```

常见的环境变量：

查看PATH环境变量（命令搜索路径）

```
echo $PATH
```

查看HOME环境变量（用户主目录）

```
echo $HOME
```

```
# 查看USER环境变量（当前用户名）
```

```
echo $USER
```

```
# 查看SHELL环境变量（当前使用的shell）
```

```
echo $SHELL
```

```
# 查看LANG环境变量（语言环境）
```

```
echo $LANG
```

变量的作用范围 与程序设计语言中的变量一样，shell 变量有其规定的作用范围。shell 变量分为局部变量和全局变量。

- **局部变量：**作用范围仅限制在其命令行所在的 shell 或 shell 脚本文件中。
- **全局变量：**作用范围则包括本 shell 进程及其所有子进程。
- **变量转换：**可以使用 export 内置命令将局部变量设置为全局变量。

示例：

```
# 定义局部变量
```

```
local_var="local value"
```

```
# 尝试在子shell中访问（无法访问）
```

```
bash -c "echo $local_var"
```

```
# 将局部变量设置为全局变量
```

```
export global_var="global value"
```

```
# 在子shell中访问（可以访问）
```

```
bash -c "echo $global_var"
```

常用的环境变量 Linux 系统中常见的环境变量及其说明：

- **PATH：**命令搜索路径，包含系统执行命令时查找的目录列表
- **HOME：**当前用户的主目录路径
- **SHELL：**当前使用的 shell 类型

- **USER:** 当前登录的用户名
- **HOSTNAME:** 主机名
- **PWD:** 当前工作目录
- **LANG:** 语言环境设置
- **TZ:** 时区设置
- **TERM:** 终端类型
- **EDITOR:** 默认编辑器
- **MAIL:** 用户邮件存储位置
- **LOGNAME:** 登录名
- **PS1:** 命令提示符格式
- **PS2:** 二级命令提示符格式
- **HISTFILE:** 历史命令文件路径
- **HISTSIZE:** 历史命令缓冲区大小
- **OLDPWD:** 上一个工作目录
- **FCEDIT:** 命令历史编辑器（fc 命令）使用的默认编辑器
- **SECONDS:** 记录 shell 会话已经运行的秒数

查看环境变量的值:

查看单个环境变量

```
echo $PATH
```

```
echo $HOME
```

查看所有环境变量

```
env
```

```
printenv
```

```
set
```

删除变量:

```
# 删除局部变量
local_var="value"
unset local_var

# 删除环境变量
export global_var="value"
unset global_var
```

注意：unset 命令可以删除局部变量和环境变量，但无法删除只读变量。

工作环境设置文件 shell 环境依赖于多个文件的设置。用户并不需要每次登录后都对各种环境变量进行手动设置，通过环境设置文件，用户工作环境的设置可以在登录的时候由系统自动完成。环境设置文件有两种，一种是系统中的用户环境设置文件，另一种是用户设置的环境设置文件。

- 系统中的用户环境设置文件：

- 登录环境设置文件：/etc/profile

- 用户设置的环境设置文件：

- 登录环境设置文件：*HOME/.bash_profile* *HOME/.bashrc*

注意：只有在特定的情况下才读取 profile 文件，确切地说是在用户登录的时候读取。运行 shell 脚本以后，就无须再读 profile 文件了。

系统中的用户环境设置文件对所有用户均生效，而用户设置的环境设置文件仅对用户自身生效。

用户可以修改自己的用户环境设置文件来覆盖系统环境设置文件中的全局设置。例如，用户可以将自定义的环境变量存放在 *\$HOME/.bash_profile* *\$HOME/.bashrc* *shell*

命令执行方式 连续命令执行（不考虑命令相关性）：

- # 使用分号分隔多个命令，按顺序执行，不考虑前一个命令是否成功

```
command1 ; command2 ; command3
```

命令回传值与逻辑操作符 命令执行完成后，会返回一个回传值（退出状态），通过特殊变量 \$? 可以获取这个值：

```
# 执行命令并查看回传值
echo "Hello"
echo $? # 显示 0，表示命令执行成功
```

执行一个不存在的命令并查看回传值

```
nonexistent_command
```

```
echo $? # 显示非 0 值，表示命令执行失败
```

使用逻辑操作符（与）和 ||（或）可以根据前一个命令的回传值来决定是否执行后续命令：

&&：前一个命令成功（回传值为 0）时，才执行后续命令

```
command1 && command2 # 只有 command1 成功，才执行 command2
```

||：前一个命令失败（回传值非 0）时，才执行后续命令

```
command1 || command2 # 只有 command1 失败，才执行 command2
```

组合使用

```
command1 && command2 || command3 # command1 成功执行 command2，失败执行 command3
```

和 || 的详细执行情况 操作符执行机制： - 语法：‘command1 command2’ - 执行逻辑：1. 首先执行 command1 2. 检查 command1 的回传值 3. 如果回传值为 0（成功），则执行 command2 4. 如果回传值非 0（失败），则不执行 command2 - 应用场景：用于确保只有前一个命令成功完成后，才执行后续的依赖命令

|| 操作符执行机制： - 语法：‘command1 || command2’ - 执行逻辑：1. 首先执行 command1 2. 检查 command1 的回传值 3. 如果回传值为 0（成功），则不执行 command2 4. 如果回传值非 0（失败），则执行 command2 - 应用场景：用于提供备选命令，当前一个命令失败时执行备用操作

实际应用示例：

示例1：备份文件，只有文件存在才备份

```
[ -f file.txt ] && cp file.txt file.txt.bak
```

示例2：创建目录，目录已存在则提示

```
mkdir -p testdir || echo "Directory already exists"
```

示例3：组合使用，实现简单的错误处理

```
cp source.txt dest.txt && echo "Copy successful" || echo "Copy failed"
```

示例4：检查命令是否存在，不存在则安装

```
type git > /dev/null 2>&1 || sudo apt install git # Debian/Ubuntu
```

```
type git > /dev/null 2>&1 || sudo yum install git # RHEL/CentOS
```

注意事项： - 和 || 的优先级相同，按照从左到右的顺序执行 - 组合使用时，要注意逻辑的正确性，避免因中间命令的回传值导致意外结果 - 可以使用括号来改变执行顺序，例如：‘command1 (command2 || command3)’

第 2 节 Shell 编程结构

- 条件判断 (if-elif-else)
- 循环结构 (for, while, until)
- 函数定义与调用
- 数组与字符串处理

第 3 节 脚本调试与优化

- 脚本调试技巧
- 错误处理机制
- 性能优化建议
- 脚本安全考虑

第 4 节 实用脚本示例

- 系统监控脚本
- 备份脚本
- 自动化部署脚本
- 日志分析脚本

第 5 节 常用命令行工具

5.1 grep 命令

grep 是一个强大的文本搜索工具，用于在文件中搜索匹配指定模式的行。

基本语法：

grep [选项] 模式 [文件...]

常用选项：

- **-i**: 忽略大小写
- **-n**: 显示行号
- **-v**: 反向搜索，显示不匹配的行
- **-c**: 显示匹配的行数
- **-A n**: 显示匹配行及其后 n 行
- **-B n**: 显示匹配行及其前 n 行
- **-C n**: 显示匹配行及其前后 n 行
- **-r**: 递归搜索目录
- **-l**: 只显示包含匹配的文件名
- **-E**: 使用扩展正则表达式
- **-F**: 固定字符串匹配，不使用正则表达式

使用示例：

在单个文件中搜索

```
grep "error" log.txt
```

忽略大小写搜索

```
grep -i "error" log.txt
```

显示行号

```
grep -n "error" log.txt
```

反向搜索

```
grep -v "error" log.txt
```

递归搜索目录

```
grep -r "error" /var/log/
```

使用正则表达式

```
grep -E "[0-9]+" file.txt
```

显示匹配行及其前后2行


```
grep -C 2 "error" log.txt
```

只显示包含匹配的文件名

```
grep -l "error" *.txt
```

高级用法：

从标准输入读取

```
cat file.txt | grep "error"
```

组合使用多个选项

```
grep -rni "error" /var/log/
```

使用管道与其他命令组合

```
ps aux | grep "nginx"
```

搜索多个模式

```
grep -E "error|warning" log.txt
```

5.2 正则表达式基础

正则表达式是一种用于匹配字符串的模式描述语言，在 `grep`、`sed`、`awk` 等命令中广泛使用。

基本元字符：

- `.`：匹配任意单个字符（除换行符外）
- `*`：匹配前面的字符零次或多次
- `+`：匹配前面的字符一次或多次（需使用 `-E` 选项）
- `?`：匹配前面的字符零次或一次（需使用 `-E` 选项）

•

常用正则表达式示例：

• # 匹配包含 "error" 的行

```
grep "error" file.txt
```

匹配以 "ERROR" 开头的行

```
grep "^ERROR" file.txt
```

匹配以数字结尾的行

```
grep "[0-9]$" file.txt
```

匹配包含至少一个数字的行

```
grep -E "[0-9]+" file.txt
```

匹配包含邮箱地址的行

```
grep -E "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" file.txt
```

匹配包含 IP 地址的行

```
grep -E "([0-9]{1,3}\.){3}[0-9]{1,3}" file.txt
```

匹配包含 "test" 或 "demo" 的行

```
grep -E "test|demo" file.txt
```

在 `grep` 中使用正则表达式的注意事项： - 默认情况下，`grep` 使用基本正则表达式（BRE），某些元字符（如 `+`、`?`、`|`、`()`）需要转义 - 使用 `-E` 选项可以启用扩展正则表达式（ERE），无需转义这些元字符 - 使用 `-F` 选项可以禁用正则表达式，按字面意思匹配字符串 - 正则表达式区分大小写，可使用 `-i` 选项忽略大小写 - 对于复杂的正则表达式，建议使用单引号包围，避免 `shell` 解释特殊字符

5.3 输入输出重定向

重定向就是不使用系统的标准输入端口、标准输出端口或标准错误端口，而进行重新指定，所以重定向分为输入重定向、输出重定向和错误重定向。通常情况下，是重定向到一个文件。在 `shell` 中，要实现重定向主要依靠重定向符，即 `shell` 通过检查命令行中有无重定向符来决定是否需要实施重定向。

标准文件描述符：

- 0：标准输入（`stdin`）
- 1：标准输出（`stdout`）
- 2：标准错误（`stderr`）

输出重定向：

将标准输出重定向到文件（覆盖模式）

```
echo "Hello" > output.txt
```

将标准输出重定向到文件（追加模式）

```
echo "World" >> output.txt

# 将标准错误重定向到文件
echo "Error" 2> error.txt

# 将标准输出和标准错误都重定向到文件
echo "Output" > all.txt 2>&1
# 或 (bash 4.0+)
echo "Output" &> all.txt

# 丢弃标准输出
echo "Silent" > /dev/null

# 丢弃标准错误
echo "Error" 2> /dev/null

# 丢弃所有输出
echo "Quiet" > /dev/null 2>&1
# 或
echo "Quiet" &> /dev/null
```

输入重定向:

```
# 从文件读取输入
cat < input.txt

# 从文件读取输入并输出到另一个文件
cat < input.txt > output.txt

# 多行输入重定向 (here document)
cat > file.txt << EOF
Line 1
Line 2
Line 3
EOF

# 多行输入重定向 (here string)
echo "Hello" | cat
```

```
# 或 (bash 4.0+)
cat <<< "Hello"
```

管道重定向：管道 (|) 用于将一个命令的标准输出重定向到另一个命令的标准输入：

```
# 将 ls 的输出传递给 grep 过滤
ls -l | grep "txt"
```

```
# 多个管道组合
ps aux | grep "nginx" | wc -l
```

```
# 结合 tee 命令同时输出到文件和终端
ls -l | tee output.txt | grep "txt"
```

第九章 Linux 高级应用

第 1 节 虚拟化技术

- KVM 虚拟化
- Docker 容器技术
- 容器编排（Kubernetes）

第 2 节 存储管理

- RAID 技术
- NFS 网络文件系统
- Samba 服务
- iSCSI 存储

第 3 节 安全加固

- 系统安全基线配置
- SSH 安全配置
- 入侵检测与防御
- 安全审计

第十章 Linux 学习资源与进阶建议

第 1 节 推荐书籍

第 2 节 在线教程与文档

第 3 节 社区与论坛

第 4 节 实践项目建议

第 5 节 认证考试（RHCE, LPIC 等）

注：此提纲仅作为学习 *Linux* 的框架，具体内容将在后续逐步补充。